Equilibrium Propagation Algorithms for Graph Structured, Dynamic, and Noisy Data

Imed BELAROUCI

Univ. Lille, CNRS, Inria, Centrale Lille, UMR 9189 – CRIStAL
59000 Lille, France

Email: imed.belarouci.etu@univ-lille.fr

M2 ISN, Internship

Supervised by Debabrota Basu

September 25, 2026

Contents

1 Introduction **11**

1.1 A primer on Equilibrium Propagation 11

1.1.1 Schwarz commutation formula of total derivatives and the fundamental Lemma of EP 12

1.1.2 Extension to LEP and equivalence of LEP PFVP with RHEL 13

1.2 A primer on Neural ODEs, LNNs and HNNs 15

1.3 A primer on GNNs, GCNs and GATs 16

1.4 Overview of the thesis 17

1.4.1 Chapter 2: LEP/RHEL for GNNs 17

1.4.2 Chapter 3: Large error signals for EP (Fourrier EP and stochastic EP) 17

1.4.3 Chapter 4: Stochastic LEP/HEP (discrete and continuous time) . . 18

1.4.4 Chapter 5: EP and LEP for RL and control 19

1.4.5 Chapter 6: Predictive single phase EP and novel DL architecture . . 20

2 Lagrangian Equilibrium Propagation for Graph Neural Networks **21**

2.1 GraphCON (Graph-Coupled Oscillator Networks) 21

2.1.1 Deriving LEP for GraphCON 23

2.1.2 Hamiltonian for GraphCON-GCN 25

2.1.3 Case 1: The Caldirola-Kanai Hamiltonian with linear activation function ($\sigma(x) = x$ and symmetric weight matrix $W = W^\top$) 25

2.1.4 Experiment for GraphCON (linear case) with LEP PFVP 27

2.1.5 Case 2: The Caldirola-Kanai Hamiltonian (Non linear activation function and symmetric weight matrix $W = W^\top$) 28

2.1.6 Experiment for Graph CON (non linear case) 29

2.1.7 Real dataset : Zachary's Karate Club 29

2.2 Quadratic HANG Formulation 30

2.2.1 Application of LEP For Quadratic HANG 31

2.2.2 Experiment for HANG quad 32

3 EP with Large error signals **35**

3.1 Fourier EP 35

3.1.1 Alternative to Holomorphic EP via Fourier transform 35

3.1.2 Approach with Fourier Transform: Fourier EP 35

3.1.3 Fourier dissipative PFVP LEP 36

3.1.4 Proposal for numerical implementation . . . 37
3.2 Stochastic EP . . . 38
3.2.1 Linking Litman [15], Massar and Mognetti [30], and Scellier [14] . . . 39
3.2.2 Proof of the gradient formula with Gibbs Boltzman distribution . . . 39
3.3 Fisher EP for Gibbs Boltzmann distributions . . . 41
3.4 Tsallis EP: Generalizing Litman's approach . . . 44
3.4.1 Extension to Tsallis distributions for EP . . . 44
3.4.2 Advantage of Tsallis distributions . . . 46
3.4.3 Extending Massar and Mognetti [30] with Tsallis distributions . . . 47

4 Stochastic LEP and HEP for Dynamic Data 48
4.1 About the Schwarz formula . . . 48
4.1.1 Deterministic EP and LEP . . . 48
4.1.2 Stochastic EP and analogy for LEP . . . 49
4.2 Alternative Derivation of the EP Learning Rule without the Helmholtz free energy . . . 50
4.3 Approach for dynamical stochastic EP with discrete time . . . 51
4.3.1 Modified Approach for stochastic damped Hamiltonian IVP EP . . . 51
4.4 Limitless HEP . . . 58
4.5 Fisher HEP: Limitless single phase . . . 59
4.6 Framework for stochastic first order dynamical systems . . . 60
4.6.1 Approach for discrete SDEs . . . 60
4.6.2 Stochastic EP for 1st order dynamical noisy systems . . . 61
4.6.3 Dynamical stochastic EP with continuous time . . . 62
4.6.4 EP-like Learning Rule for Continuous-Time SDEs . . . 63
4.7 2nd order stochastic dynamical systems . . . 66
4.7.1 Score function estimator for Stochastic 2nd order dynamical systems in continuous time . . . 66
4.7.2 EP for stochastic 2nd order dynamical systems in continuous time . . . 67

5 EP and LEP for reinforcement learning and control 69
5.1 Closed loop deterministic control with LEP PFVP . . . 69
5.1.1 Setup: substitute the policy into the Lagrangian . . . 70
5.1.2 Closed loop LEP PFVP control . . . 70
5.2 Energy-Based EP actor(stochastic)-critic RL . . . 71
5.2.1 Critic Optimization . . . 71
5.2.2 Actor network "stochastic EP" . . . 72
5.3 Distributional RL: Matching Return Distributions . . . 73
5.3.1 Gradient derivation . . . 75
5.4 Stochastic deterministic control: Discrete time . . . 77
5.5 Stochastic deterministic control: Continuous time . . . 79
5.5.1 Functional derivative . . . 79

6 Single phase EP and novel DL architecture **81**
6.1 Limitless, single phase, EP based on Fisher divergence 81
6.1.1 Fisher Divergence and Score Matching 81
6.1.2 The Formal Derivation 81
6.1.3 Single-Phase EP 82
6.1.4 Deterministic single phase static EP 83
6.1.5 Deterministic single phase LEP with Large error signal and no boundary residuals for any Lagrangian 84
6.1.6 Numerical experimentation 85
6.2 DL Architecture: Classical Paradigm and Forward Only Proposal 87
6.2.1 Proposed Approach 87
6.2.2 The Cheat Cost Concept 90
6.2.3 Approach without bijectivity constraint 91

7 Discussions, Limitations, and Future Works **94**

A Some proofs of Chapter 4 **97**
A.1 Alternative Derivation of the EP Learning Rule (without the Helmholtz free energy for the Gibbs-Boltzmann distribution) (see Chapter 4) 97
A.1.1 Proof of the EP formula derivation from Chapter 4 97
A.2 Approach for dynamical stochastic EP with discrete time (see Chapter 4) . . 99
A.2.1 Modified Approach for stochastic damped Hamiltonian IVP EP 99
A.2.2 Proof of the EP formula 99
A.3 About Litman's objective (Limitless HEP) (see Chapter 4) 102
A.3.1 Proof of the EP-like formula 102
A.4 Fisher HEP (limitless single phase) (see Chapter 4) 103
A.4.1 Proof of the EP-like formula 103

Abstract

Equilibrium Propagation (EP) is a learning algorithm that applies to a class of Machine learning (ML) models called Energy-Based Models (EBMs). EP is based on a philosophy of single physical circuit for learning and inference, and forward only local learning rules. EP enables cheaper computational complexity in both time and memory and is a biologically plausible ML algorithm unlike Backpropagation. Lagrangian Equilibrium Propagation (LEP) is the extension of EP to handle dynamical data (time dependent inputs and outputs). Some open questions related to this algorithm consist in it's implementation for Graph structured data, and handling dynamical noisy data. Among the contributions made in this work are the application of EP to graph dynamical data, and also, Reinforcement learning (RL) and optimal control. We also propose an EP framework for dynamical noisy data. Other contributions consists on enabling large error signals and single phase EP.

Context

The majority of the work done in this internship has been related to contributions for EP and LEP, this was done following the guidelines of D. Basu who was the Internship supervisor. The topics considered where :

1) Application of LEP to Graph neural networks (GNNs), Graph convolutional networks (GCNs), and also Graph Attention networks GATs (see Chapter 2).

2) Considering large error signals for EP and LEP (see Chapter 3).

3) application of EP and LEP in RL and control (see Chapter 5).

4) Advantage of Hamiltonian/ Lagrangian formalism (HNNs/LNNs) over differential equations (Neural ODEs).

5) Contributions to stochastic EP and LEP to handle noisy data (see Chapter 4).

All the topics expect the one regarding "Advantage of Hamiltonian/ Lagrangian formalism (HNNs/LNNs) over differential equations (Neural ODEs)" will be developed in this work, because contributions have been achieved for each. For the exception topic (not developed), research in Literature have been made, and one main advantage which was highlighted for LNNs (Lagrangian neural nets) by Cranmer et al. [1] and HNNs (Hamiltonian neural nets) by Greydanus, Dzamba, and Yosinski [2], is that the Hamiltonian/ Lagrangian formalism for deep learning offers the advantage of learning energy conservation more efficiently than standard Neural ODEs, nevertheless this characteristic is suited mostly for PINNs (see Raissi, Perdikaris, and Karniadakis [3]) (Physics informed neural nets)) and not general DL. Actually, the property of energy conservation is not suited for general ML applications as stated by Pourcel et al. [4], For conservative systems where Energy is conserved, DL systems can't forget, and dissipative systems like dissipative LEP that was proposed in that paper, enables dissipation of energy, which makes it more preferable for general purpose ML. An exploration for quantization has been considered when dealing with this topic (indeed one of the efficient way to quantize a differential system is based on canonical quantization which is suited only for Hamitlonian's [5]), quantization would be suited for quantum neural nets (see Biamonte et al. [6]), and may also offer an alternative to augmented Neural ODEs [7] (due to quantization, a natural discretization property occur, which is linked to quantum correlations and non locality, nevertheless this is just a hypothesis) , but it has been estimated that the work required to make contributions in this point was too consequent for the time available at this internship, and the results if positive wouldn't be that significant, thus this point has been abandoned.

Contribution for another topic : single phase EP (see Chapter 6). During the exploration related to the previous points, some contributions related to single phase EP were made (inspired by the Framework of Luczak, McNaughton, and Kubo [8]). A heuristic approach for a variant of Fisher EP (see Chapter 3) with a single phase deterministic EP has been derived (where the nudge parameters are considered "frozen"). This heuristic EP proposal would require future work for a more rigorous baseline. Another framework not directly related to EP but inspired for single phase, forward only and biological plausibility ML has been considered. It has some connections to "Target propagation" [9],[10], this is still a heuristic proposal and further work for a rigorous basis is required.

Contributions

I. Application of LEP to GNNs, GCNs, and also GATs (see Chapter 2)

Here the main contribution is to implement LEP PFVP (proposed by Pourcel et al. [4]) to GNNs, GCNs and GATs. The work consisted in identifying from the literature the intersection between GNNs, GCNs, GAts and Neural ODEs, furthermore, because not all dynamical systems are Lagrangian or Hamiltonian systems, we further selected the proposals which where compatible with Hamiltonian and Lagrangian formalism, and satisfied also the property of time reversibility (key requirement of LEP PFVP to avoid costly boundary residuals "which is against the philosophy of computational efficiency of EP"). We consider the application of LEP PFVP to Graph CON which was proposed by Rusch et al. [11], and we consider the application of RHEL to Quadratic HANG which was proposed by Zhao et al. [12]

II. Considering large error signals for EP and LEP (see Chapter 3)

In this part, we address the fundamental requirement of infinitesimal error signal required by theory for EP. There is a contribution in a novel framework of Fourier EP, which is based on a Fourier transform, which enables large error signals, and is inspired by the framework of Holomorphic EP [13]. Compared to Holomorphic EP, Fourier EP has an advantage of broader and more realistic applicability (energy functions and cost functions are not Holomorphic in practice).

There are also contributions in stochastic EP, mainly establishing a link between Scellier's classical stochastic EP framework [14], and Litman's limitless EP framework [15]. Furthermore Litman's framework is generalized to any nudge parameter value, also a generalization to Tsallis distributions (which generalizes Gibbs Boltzmann distributions and describe more general stochastic systems) called Tsallis EP has been derived. Litman's framework is also generalized from KL divergence based divergences to Fisher based divergences which allows a single (nudge) phase limitless learning and called Fisher EP, and also for the Cramer distance for application in RL (by considering the parameters as frozen for the target "nudge" distribution).

III. Contributions to stochastic EP and LEP to handle noisy data (see Chapter 4)

We propose a framework of EP which handles dynamical and noisy data simultaneously. The work relates to Chapter 3 where stochastic contributions have been made. A framework for stochastic Hamiltonian EP has been established by considering discrete time dynamics and also continuous time dynamics. Furthermore Litman's framework [15], is generalized to dynamical noisy data in discrete time, and we also consider a generalization of Fisher EP (from Chapter 3) as well.

IV. Application of EP and LEP in RL and control (see Chapter 5)

This work relates to application of EP in RL which have been considered only by Han and Sengupta [16] and also Kubo, Chalmers, and Luczak [17]. We consider a Litman based EP framework (similar, but different) with Cramer distance, and a target nudge distribution (where the parameters are frozen). Furthermore, a framework for LEP in closed loop deterministic control has been established. Also a general EP based actor-critic framework is established.

V. Contribution for another topic : single phase EP (see Chapter 6)

In this final chapter, we address the question of single phase EP.

We consider a true single phase framework without prediction (unlike the proposal of Luczak, McNaughton, and Kubo [8]) similar to the stochastic Fisher EP proposal in Chapter 3, where the parameters are frozen in the nudge distribution, with a stochastic objective function defined solely as the Fisher divergence, this results in a different gradient rule, which enjoys a remarkable property of unlike any other stochastic framework : the deterministic limit, results in an exact gradient formula, which requires only a single nudge phase. Nevertheless, this proposal is still a heuristic (unlike, the others), and further work is still required to make it more rigorous (see Chapter 6 for details). This approach is compatible with both EP and LEP. Furthermore it offers a potential great advantage for LEP : having an approach based on a single phase, limitless, and without costly boundary residuals while being implementable for any Lagrangian (this potentially answers the limitations stated by Pourcel et al. [4]), nevertheless this proposal is still a heuristic and further work is required. We call it Heuristic EP for now.

Finally, taking inspiration from the motivation of EP, which were, single circuit and forward only DL models for biological plausibility, we propose a heuristic novel DL architecture, which enjoys this properties. Further work is required to obtain a stronger foundation.

Acknowledgments

I want to thank the advisor Debabrota Basu, for giving me the opportunity to work in this exciting and modern topic. His guidelines helped me to reach these contributions which are developed in this work. I want to thank Guillaume Pourcel for insightful discussions.

I also want to thank Nicolas Wicker for sharing this internship opportunity, and i also want to thank Tanmoy Mondal who also approved my application to this internship.

List of Abbreviations

EP	Equilibrium propagation
LEP	Lagrangian Equilibrium propagation
CIVP	Constant Initial Value Problem
IVP	Initial Value Problem
CBPVP	Constant Boundary Position Value Problem
PFVP	Parametric Final Value Problem
RHEL	Recurrent Hamiltonian Echo Learning
HEP	Hamiltonian Equilibrium propagation
HEB	Hamiltonian Echo Backpropagation
BPTT	Backpropagation Through Time
BP	Backpropagation
ML	Machine learning
DL	Deep learning
RL	Reinforcement learning
GCNs	Graph Convolutional Networks
GATs	Graph attention networks
HANG	HamiltoniAN Graph diffusion
CNNs	Convolutional neural networks
GraphCON	Graph-Coupled Oscillator Networks
GNNs	Graph Neural Networks
PINNs	Physics-informed Neural Networks
ODEs	Ordinary differential equations
SDEs	Stochastic differential equations
HNNs	Hamiltonian neural nets
LNNs	Lagrangian neural nets
EBMs	Energy-Based Models

Chapter 1

Introduction

1.1 A primer on Equilibrium Propagation

Equilibrium propagation (EP) is a machine learning (ML) algorithm introduced by Scellier and Bengio [18], which offers low computational complexity in comparison to backpropagation (BP), and is suited for low consumption ML hardware (shown in Table 1.1 from Pourcel et al. [4]) , specifically analog computers. Furthermore it is a biologically plausible ML algorithm, due the use of a single circuit for both inference and learning, unlike BP. Nevertheless, EP can only be suited for ML Energy based models (EBMs) which relies on relaxation/optimization of a scalar energy function E, to produce the output, which represent the relaxed/optimal state.

Method	**Time Complexity**		**Memory**		**Forward-only**	**Streaming**	**Bottleneck**
	Dynamics	**Gradient**	**Dynamics**	**Gradient**			
CBPVP	$\mathcal{O}(KNd_s^2)$	$\mathcal{O}(Nd_\theta)$	$\mathcal{O}(Nd_s)$	$\mathcal{O}(d_\theta)$	yes	no	BVP iterations
EP (static data)	$\mathcal{O}(Kd_s^2)$	$\mathcal{O}(d_\theta)$	$\mathcal{O}(d_s)$	$\mathcal{O}(d_\theta)$	yes	irrelevant	None
CIVP	$\mathcal{O}(Nd_s^2)$	$\mathcal{O}(Nd_s^3)$	$\mathcal{O}(d_s)$	$\mathcal{O}(Nd_s)$	no	yes	BPTT memory
PFVP/RHEL	$\mathcal{O}(Nd_s^2)$	$\mathcal{O}(Nd_\theta)$	$\mathcal{O}(d_s)$	$\mathcal{O}(d_\theta)$	yes	yes	None

Table 1.1: Computational complexity comparison for EP (static data) and LEP (dynamic data). Red indicates the dominant cost that makes the method impractical. Green indicates efficient scaling. K is the number of iterations required for the convergence of the gradient descent on the energy (static case) or the action (dynamics case), N is the number of discrete time steps, d_θ is the dimension of the learnable parameters θ, d_s is the dimension of the state s. See Pourcel et al. [4]) for detailed derivation.

In EP, the scalar energy function $E_\beta(\theta, \beta, s, x, y) = E_0(\theta, 0, s, x_0) + \beta C(s, y_0)$ is the fundamental object of the algorithm, it describes the whole ML EBM, where the state is $s \in \mathbb{R}^{dim(s)}$, and $s^\beta = argmin_s E_\beta(\theta, \beta, s)$ is the optimal state of the energy function which represents the prediction in supervised learning framework. The static input is x_0 and the static output is y_0. The cost/loss function which we aim to optimize is $C(s, y_0)$ (usually the least squares for regression tasks, and Cross-entropy for classification). The scalar parameter β is a hyperparameter proper to EP, and central to the algorithm. For $\beta = 0$, we have the 0

error signal case, in which the EBM : is $E_\beta(\theta, \beta, s, x, y)|_{\beta=0} = E_0(\theta, 0, s, x)$, this represents the EBM which would be used for inference after training is done. θ is the learnable parameters vector and the goal of EP would thus be to optimize via gradient descent $C(s^0, y_0)$, thus we have to compute $d_\theta C(s^0, y_0)$. For notational simplicity, we omit the dependence of the energy E_β on inputs x and targets/outputs y whenever the context is clear.

1.1.1 Schwarz commutation formula of total derivatives and the fundamental Lemma of EP

We present the fundamental lemma of the equilibrium propagation framework, from Scellier and Bengio [18]

Let $E_\beta(\theta, \beta, s)$ be a twice differentiable function of the three variables θ, β, and s. For fixed θ and β, let s_θ^β be a point that satisfies the stationarity condition

$$\frac{\partial E_\beta}{\partial s}(\theta, \beta, s_\theta^\beta) = 0, \tag{1.1}$$

and suppose that $\frac{\partial^2 E_\beta}{\partial s^2}(\theta, \beta, s_\theta^\beta)$ is invertible. Then, in the neighborhood of this point, we can define a continuously differentiable function $(\theta, \beta) \mapsto s_\theta^\beta$ such that Eq. (1.1) holds for any (θ, β) in this neighborhood. Furthermore, we have the following identity:

$$\frac{d}{d\theta}\frac{\partial E_\beta}{\partial \beta}(\theta, \beta, s_\theta^\beta) = \frac{d}{d\beta}\frac{\partial E_\beta}{\partial \theta}(\theta, \beta, s_\theta^\beta). \tag{1.2}$$

The first statement follows from the implicit function theorem [18]. The second statement is obtained using the chain rule of differentiation and the stationary condition of Eq. (1.1). We have:

$$\frac{d}{d\beta}E_\beta(\theta, \beta, s_\theta^\beta) = \frac{\partial E_\beta}{\partial \beta}(\theta, \beta, s_\theta^\beta) + \underbrace{\frac{\partial E_\beta}{\partial s}(\theta, \beta, s_\theta^\beta)}_{=0} \cdot \frac{\partial s_\theta^\beta}{\partial \beta}. \tag{1.3}$$

Similarly, we have:

$$\frac{d}{d\theta}E_\beta(\theta, \beta, s_\theta^\beta) = \frac{\partial E_\beta}{\partial \theta}(\theta, \beta, s_\theta^\beta) + \underbrace{\frac{\partial E_\beta}{\partial s}(\theta, \beta, s_\theta^\beta)}_{=0} \cdot \frac{\partial s_\theta^\beta}{\partial \theta}. \tag{1.4}$$

Combining these equations and using the symmetry of second derivatives, we get:

$$\frac{d}{d\theta}\frac{\partial E_\beta}{\partial \beta}(\theta, \beta, s_\theta^\beta) = \frac{d}{d\theta}\frac{d}{d\beta}E_\beta(\theta, \beta, s_\theta^\beta) = \frac{d}{d\beta}\frac{d}{d\theta}E_\beta(\theta, \beta, s_\theta^\beta) = \frac{d}{d\beta}\frac{\partial E_\beta}{\partial \theta}(\theta, \beta, s_\theta^\beta). \tag{1.5}$$

This yields the fundamental result of EP [18]

$$d_\theta C(s_\theta^\beta, y_0) = \frac{d}{d\beta}\frac{\partial E_\beta}{\partial \theta}(\theta, \beta, s_\theta^\beta) \tag{1.6}$$

Thus

$$d_\theta C(s_\theta^0, y_0) = \frac{d}{d\beta}\bigg|_{\beta=0} \frac{\partial E_\beta}{\partial \theta}(\theta, \beta, s_\theta^\beta) = \lim_{\beta \to 0} \frac{1}{\beta} \left[\partial_\theta E_\beta(s_\theta^0, \theta, x_0) - \partial_\theta E_0(s_\theta^0, \theta, x_0)\right] \tag{1.7}$$

This suggests a two-phase procedure for gradient estimation via a finite difference method [18]:

1. **Free phase:** Compute the output value of the implicit function s_θ^0 by finding a minimum of the energy function $E_0(s, \theta, x_0)$.
2. **Nudged phase:** Compute the output value of the implicit function s_θ^β for a small positive value of β by finding a slightly perturbed minimum of the augmented energy $E_\beta(s, \theta, x_0, y_0)$.

In practice, these implicit function values can be found either by applying EP with an analog computer, and waiting for the EBM to relax at the optimal state which would represent the prediction, or by applying EP with a digital computer, and implement a root-finding algorithm, such as gradient descent dynamics over the energy function for example to obtain the optimal relaxed state of the EBM. Even though the data processed is static, there is a temporal dimension related to the relaxed state [18].

In the free phase ($\beta = 0$), the system evolves according to :

$$\frac{d}{dt} s_t = -\partial_s E(s_t, \theta, x_0), \tag{1.8}$$

until convergence to $s^0(\theta)$, i.e., $\lim_{t\to\infty} s_t = s^0(\theta)$. In the nudged phase ($\beta > 0$), starting from the free equilibrium, the system follows :

$$\frac{d}{dt} s_t = -\partial_s E(s_t, \theta, x_0) - \beta \partial_s C(s_t, y_0), \tag{1.9}$$

until convergence to $s^\beta(\theta)$, i.e., $\lim_{t\to\infty} s_t = s^\beta(\theta)$.

1.1.2 Extension to LEP and equivalence of LEP PFVP with RHEL

EP has been generalized to describe entire trajectories through a variational problem (in [4], and [14] enabling the training of dynamical systems that map time-varying inputs to time-varying outputs. this extension is refereed to as Lagrangian EP (LEP). For this extension the scalar augmented energy E_β is promoted to a scalar augmented action functional A_β that integrates over a time-varying "energy-like" quantity : the augmented Lagrangian density L_β [4].

$$\begin{aligned} \underbrace{A_\beta[s, \theta, x, y]}_{\text{augmented action}} &\int_0^T \underbrace{\Big(L_0(s_t, \dot{s}_t, \theta, x_t) + \beta c(s_t, y_t)\Big)}_{L_\beta(s_t, \dot{s}_t, \theta, x_t, y_t)} dt \\ &= \underbrace{\int_0^T L_0(s_t, \dot{s}_t, \theta, x_t)\, dt}_{\text{free action } A_0[s,\theta,x]} + \beta \underbrace{\int_0^T c(s_t, y_t)\, dt}_{\text{cost } C[s,y]}. \end{aligned} \tag{1.10}$$

The augmented action functional A_β is the temporal analogue of E_β. It integrates the augmented Lagrangian L_β (which takes as input the state s_t, its temporal derivative (velocity) $\dot{s}_t$, and the time-varying input x_t) that extends the Lagrangian by including an additional nudging term $\beta c(s_t, y_t)$. The augmented action functional $A_\beta[s]$ maps a trajectory $s\{s_t : t \in [0, T]\}$ to a scalar value, generalizing the scalar-valued energy functions of EP to functional-valued quantities that capture temporal dynamics. For notational simplicity, we omit the dependence on inputs x and targets y (or their time-indexed versions x_t and y_t) whenever the context is clear [4].

Unlike EP, where the relaxation dynamics are typically solved through gradient descent dynamics (for digital computing), these functional variational problems can be solved more directly using the Euler-Lagrange equations. The corresponding Euler-Lagrange (EL) expression is defined as [4] :

$$\begin{aligned} \mathrm{EL}(t, \theta, \beta)\partial_s L_\beta(s_t, \dot{s}_t, \theta) - d_t \partial_{\dot{s}} L_\beta(s_t, \dot{s}_t, \theta) \\ = \partial_s L_0(s_t, \dot{s}_t, \theta) - d_t \partial_{\dot{s}} L_0(s_t, \dot{s}_t, \theta) + \beta \partial_s c(s_t). \end{aligned} \tag{1.11}$$

The stationary "optimal" trajectory corresponds to the trajectory which makes the action stationary (see [19]), it is obtained by solving the EL equation : $\mathrm{EL}(t, \theta, \beta) = 0$. When the Lagrangian depends only on position and velocity, the resulting EL equations is a 2nd order differential equation, thus to be fully solved, two degrees of freedom have to specified (boundary specifications) [4].

The first natural choice is the causal boundary conditions which corresponds to a Constant Initial Value Problem (CIVP). This corresponds to fixing initial position and velocity : s_0 and $\dot{s}_0$. As shown by [4] and [20], in this configuration, costly boundary residuals are present, which makes the method incompatible with the EP philosophy (computational efficiency and single physical circuit). The costly boundary residuals are the jacobians with respect to the learnable parameters of the final position and final velocity : $d_\theta s_T^0$, $d_\theta \dot{s}_T^0$ (see [4] for detail).

The second other natural choice, is the one considered originally for LEP by [14], with non causal boundary conditions which corresponds to Constant Boundary Position Value Problem (CBPVP). This corresponds to fixing the initial and final positions : s_0 , s_T. This approach fully cancels the boundary value terms, and makes the more straightforward generalization of EP to dynamical data (see Table 1.1 from Pourcel et al. [4] : the computational complexity of EP corresponds to the values of CBPVP with one time step $N = 1$). Nevertheless this makes the approach incompatible with streaming, as the trajectory relaxation has to be achieved, via gradient descent on the action (implemented directly by a digital computer, or "implicitly" by an analog computer).

For LEP CBPVP, the gradient formula is derived from the Schwarz formula for the action $d_\beta d_\theta A_\beta(\theta, s) = d_\theta d_\beta A_\beta(\theta, s)$, and considering the CBPVP framework, it gives [14]:

$$d_\theta C[s^{0\leftrightarrow}(\theta, (\alpha_0, \alpha_T))] = \lim_{\beta \to 0} \frac{1}{\beta} \int_0^T \left[\partial_\theta L_\beta(s_t^{\beta\leftrightarrow}, \dot{s}_t^{\beta\leftrightarrow}, \theta) - \partial_\theta L_0(s_t^{0\leftrightarrow}, \dot{s}_t^{0\leftrightarrow}, \theta) \right] dt \tag{1.12}$$

LEP PFVP (Parametrized final value problem) : When the model is described by a Lagrangian which enjoys the property of time reversibility [4] :

$$L_0(s_t, \dot{s}_t) = L_0(s_t, -\dot{s}_t)$$

The forward nudge phase (forward in time) performed in the CIVP case, can be performed instead in reverse by a backward nudge phase (backward in time). This would still keep the philosophy of EP : froward only/single circuit, and the backward phase, would performed by a trick of velocity flip : $\dot{s}_t \to -\dot{s}_t$. This results in the LEP PFVP (parametrized final value problem) algorithm which is more efficient than the CBPVP method, while still being compatible with streaming [4].

Furthermore, LEP PFVP has been showed to be equivalent to RHEL (Recurrent Hamiltonian Echo Learning) [4]. Indeed RHEL, like HEB (Hamiltonian Echo Backprop) is based on Hamiltonian formalism, and Hamiltonian formalism can be transformed into Lagrangian formalism via the Legendre transform, and the opposite is also true. It was shown that RHEL [21], like HEB [22], can approximate the adjoint state method [23] in a manner making the algorithm compatible with EP philosophy, Nevertheless for the sake of simplicity it is easy to consider that RHEL as just a version of LEP where the scalar Lagrangian density is replaced by the Hamiltonian scalar energy function.

When the initial conditions $\begin{pmatrix} \alpha_0 \\ \mu_0 \end{pmatrix}$ are independent of the parameters θ (i.e., $\partial_\theta \begin{pmatrix} \alpha_0 \\ \mu_0 \end{pmatrix} = 0$), the RHEL gradient formula is (similar form with LEP PFVP after applying Legendre transform, see [4]) :

$$d_\theta C[\Phi(\theta, (\alpha_0(\theta), \mu_0(\theta))^\top)] = \lim_{\beta \to 0} -\frac{1}{\beta} \int_0^T [\partial_\theta H(\Phi_t^e, \theta, x_{T-t}) - \partial_\theta H(\Phi_t, \theta, x_t)]\, dt \tag{1.13}$$

where Φ_t^e is the echo trajectory at time t, and Φ_t represents the forward trajectory evaluated at time t.

Remark 1.1.1. *Here the augmented state ϕ describes both the vanilla position state s and it's associated momentum p.*

1.2 A primer on Neural ODEs, LNNs and HNNs

Neural ODEs. Neural ODEs [24] offer an approach for modeling continuous dynamics on hidden representations, where the dynamics are characterized through an ODE parameterized by a neural network. However, these methods can only deal with unstructured data, where different inputs are independent [25].

Continuous Graph Neural Networks extends Neural ODEs to graph-structured data.

Thorpe et al. [26] states that training neural ODEs using the adjoint method [23] is more memory efficient than training other neural networks using backpropagation. Recall that RHEL approximates the adjoint state method only for Hamiltonian based models (when the

Hamiltonian satisfy time reversal symmetry), which is not necessarily the case with Neural ODEs.

Lagrangian Neural networks (LNNs, see [1]) and Hamiltonian neural networks (HNNs, see [2]) are compatible with LEP (CBPVP and PFVP) and RHEL. Specifically LEP CBPVP can be applied to any LNNs, and LEP PFVP as well as RHEL can be applied to LNNs and HNNs that satisfy the time reversal property respectively.

1.3 A primer on GNNs, GCNs and GATs

Graph Neural Networks (GNNs). Graph neural networks [27] are an effective approach for learning node representations in graphs [25]. Typically, GNNs model discrete dynamics of node representations with multiple propagation layers, where in each layer the representation of each node is updated according to messages from neighboring nodes [25]. The majority of GNNs learn the relevant information from the graph structure A by learning finite polynomial filters g to apply to the eigenvalues Λ of the graph Laplacian $L = P\Lambda P^{-1}$ in each propagation layer [27]. GCNs as proposed by Kipf and Welling [27], uses a first-order Chebyshev polynomial approximation.

Advantage of Graph ODEs over GNNs

GNNs (e.g., GCNs) have been shown to suffer from over-smoothing, i.e., node representations start to converge to the same value [25]. However, it has been showed that Graph ODEs like for example Graph-Coupled Oscillator Networks (GraphCON) which is proposed by Rusch et al. [11] mitigates the oversmoothing problem. There is also the continuous graph neural networks (CGNN) architecture proposed by Xhonneux, Qu, and Tang [25] which does not suffer from the over-smoothing problem.

LEP for Graph ODEs

Most graph neural networks (GNNs) are built as a stack of discrete layers: node features are updated layer by layer, each layer mixing a node's own state with its neighbors' states through a learned linear map, usually followed by a nonlinearity. GraphCON [11] takes a different view: it treats the node features as the state of a physical system (a network of coupled, damped oscillators sitting on the graph) and defines the network's computation as *integrating this system forward in time* from $t = 0$ to some horizon T. "Depth" becomes "how long we let the system evolve", not "how many layers we stack". This continuous-depth view is part of a broader family (Neural ODEs) and has two attractive properties: it naturally avoids some of the numerical pathologies (vanishing/exploding updates, oversmoothing) that afflict very deep discrete GNNs, and it opens the door to *physics-inspired training rules* that do not require backpropagating through every integration step.

Training a system like this normally uses backpropagation-through-time (BPTT): unroll the ODE solver, then backpropagate the loss through every single step. BPTT is exact but requires storing the whole trajectory and differentiating through it, but it is expensive in memory, and arguably this is not how a physical or biological system could plausibly

learn. LEP PFVP (Physics-inspired/Principle-of-Least-Action-based Learning, closely related to Equilibrium Propagation), is a two-phase training rule: run the system forward once undisturbed (the *free phase*), then run a second, cheaply related simulation nudged gently towards the target (the *nudged/echo phase*), and read the parameter gradient off the difference between the two. There is no backward pass through the dynamics at all.

1.4 Overview of the thesis

1.4.1 Chapter 2: LEP/RHEL for GNNs

This chapter aims at applying the LEP PFVP and/or RHEL algorithms to Graph ODEs models which implement the convolution architecture (GCNs based on differential equations) or the attention mechanism (GATs based on differential equation).

To do so, we first identified among the various proposals of Graph ODEs, the ones which are based on Hamiltonian or Lagrangian dynamics (usually the requirement is to have a 2nd order differential equation), also the Hamiltonian or Lagrangian have to satisfy the time reversal symmetry property, and the convolution architecture or the attention mechanism have to be included in the construction of the Graph architecture.

We have selected the GraphCON Framework (Graph-Coupled Oscillator Networks) proposed by Rusch et al. [11]. Following the GraphCON framework, we implemented two architecture that are based on convolutions (see Chapter 2).

We also considered the Quadratic HANG (HANG-quad) Framework proposed by Zhao et al. [28] for implementing the attention mechanism.

1.4.2 Chapter 3: Large error signals for EP (Fourrier EP and stochastic EP)

We confront the issue of infinitesimal error signal, which is fundamentally inherent to the EP framework. Indeed , the EP algorithm is based on taking the limit to 0 of the nudge parameter β to obtain the gradient of the loss function $d_\theta C(s^0)$. Nevertheless, due to the operating of this framework, the requirement of a small nudge parameter β introduce two problems :

1) In practise, it's impossible to take $\beta \to 0$, and a small β value like for example $\beta \approx 0.1$ is considered instead. The choice of the nudge parameter, is similar to a hyper-parameter. This choice of finite small nudge parameter introduce a biais in the EP gradient estimation formula. As mentioned in [29], some strategies relied on considering random nudge parameters which would follow a Gaussian distribution with zero mean for example, nevertheless such strategy hasn't deliver satisfactory result. As shown by Laborieux et al. [29] the working approach is to reduce the bias size from $O(\beta)$ to $O(\beta^2)$ by modifying the EP formula with two symmetric nudge phases ($+\beta$ and $-\beta$) instead of the original free and nudge phase. This result in the following gradient formula :

$$d_\theta C(s^0_\theta, y_0) = \lim_{\beta \to 0} \frac{1}{2\beta} \left[\partial_\theta E_\beta(s^\beta_\theta, \beta, \theta, x_0) - \partial_\theta E_0(s^{-\beta}_\theta, -\beta, \theta, x_0) \right] \tag{1.14}$$

Nevertheless, this approach still relies on the requirement of small nudge parameter β, which give a 2nd problem :

2) The learning signal in each nudge phase, has to be small, which makes the learning slow, and makes EP incompatible with big DL architectures.

To address the large error signal problem (related to the size of the nudge parameter), we identified the framework of Holomorphic EP proposed by Laborieux and Zenke [13] where the Cauchy formula is used to re-express the β derivative at 0, and is engineered in a way that allows the consideration of large finite nudge signals. It has been shown that this approach offers an advantage over EP to handle noisy data in [13]. Nevertheless, it wasn't compared with the stochastic version of EP (which would handle noisy data), and the Cauchy formula requires that the cost functions, as well as the energy functions are Holomorphic, which in practice is not the case (in particular the cost function is usually real valued, thus not complex valued, and not Holomorphic). Following the construction of Holomorphic EP, we identify that the main idea that allows to consider large error signals, is the re-expression of the β derivative, thus we consider an alternative approach based on Fourier transform, and particularly, it's properties with the derivative operator, to derive a novel EP gradient rule, which inherently allow large error signals, without the requirement of Holomorphicity, but with a weaker requirement of compatibility with Fourier transform (satisfied in practice). The formula is derived in Chapter 3.

Another EP related framework, which enables the consideration of finite and large error signals, has recently been developed by Litman [15], in which he constructs a different objective function than the original one considered by Scellier [14] in the context of stochastic EP. In chapter 3, we highlight the relation between the two approaches of Litman [15] and Scellier [14]. Furthermore, we extend the applicability of the framework proposed by Litman [15] to Tsallis distributions, indeed like the original stochastic EP framework proposed by Scellier [14], the framework of Litman [15] is applicable only to Gibbs-Boltzmann distributions, which represents a particular case of the Tsallis distribution, which is more general and represents the stationary distribution of more general SDEs (see Chapter 3 for details).

Another proposal related to stochastic EP has been proposed by Massar and Mognetti [30], which relies on the log-derivative trick, and enables a gradient rule for EP without any nudge signal, thus relying only the free phase. This result has been derived only for a particular energy function, namely the Hopfield energy function and for Gibbs Boltzmann distribution, we extend in Chapter 3, the result of Massar and Mognetti [30] to arbitrary energy functions, and also to the more general Tsalis distribution.

Furthermore, we consider a variant of the framework used by Litman [15], where the KL divergence is replaced by the Fisher divergence, which results on a learning rule based on a single phase, which is the nudge phase (opposite to the single free phase of Massar and Mognetti [30]). We call this approach : Fisher EP.

1.4.3 Chapter 4: Stochastic LEP/HEP (discrete and continuous time)

The goal of this chapter is to extend HEP (Hamiltonian EP) to stochastic dynamical systems, to handle dynamical noisy data. Indeed such approach has not been yet considered.

We start by highlighting an important statement, which is related to Chapter 3 (static stochastic EP), which states that unlike in the deterministic case, where the EP framework relies on the Schwarz commutation formula of total derivatives (d_θ and d_β), it is not the case in the stochastic framework, indeed we show that the original stochastic EP (see [14]) formula can be derived by considering only the Schwarz commutation formula of partial derivatives (∂_θ and ∂_β) (this is not true in the deterministic case).

This remark is important because the requirement of time reversal symmetry of the LEP PFVP construction was to eliminate the costly boundary residuals of the LEP CIVP approach, which emerge due to the application of the Schwarz commutation formula with total derivatives (d_θ and d_β) to the action A_β [4]. Thus by following the right framework, there would be no boundary residuals in stochastic dynamical EP.

Furthermore, the original stochastic EP formula derived by Scellier [14] is built upon the Helmholtz free energy, which is associated to the Gibbs Boltzmann distribution. We show that the stochastic EP gradient formula can be derived from the expected energy instead. This would offer a straightforward generalization to the dynamical case (similar to promoting the energy to an action to move from EP to LEP).

Based upon all those results and remarks, we consider a stochastic LEP/HEP gradient rule, by considering the expected integrated augmented "energy". We start by considering a simplified framework of discrete time, and then consider the more general continuous time setting (see Chapter 4 for details).

1.4.4 Chapter 5: EP and LEP for RL and control

In the 5th chapter, we address the applicability of EP and LEP to RL and control. EP has been considered mostly for supervised learning applications, nevertheless RL applications of EP has been considered by Kubo, Chalmers, and Luczak [17] and Han and Sengupta [16]. We first make a proposal of an EP actor critic framework similarly to the proposal of Kubo, Chalmers, and Luczak [17], where EP is used to train both the actor and critic (unlike using backpropagation to train the critic like in [17]).

Furthermore we consider an EP-like framework for distributional RL, where we modify the framework proposed by Litman [15], and instead of using KL or Fisher divergence, we use the Cramer distance, and consider frozen parameter on the nudge distribution, which would represent the target distribution in a distributional RL framework.

We also propose a framework to implement RHEL for deterministic closed loop control (open loop control requires BPTT memory even for RHEL when considering functional derivative over the dynamical control).

Finally, we propose a framework, to apply LEP or Hamiltonian based EP (HEP) for stochastic optimal control, where we generalize the framework proposed by Massar and Mognetti [30] to functional derivative (we replace the parameters θ by the dynamical control variable u_t), this construction is related to the Chapter 4, which addresses stochastic LEP and HEP. We call this method for now : Heuristic EP.

1.4.5 Chapter 6: Predictive single phase EP and novel DL architecture

In this final chapter, we address the question of single phase EP. Indeed it has been mentioned in Pourcel et al. [4], that among the limitations of LEP PFVP and RHEL was the requirement, of two separate phases.

An approach of single phase predictive EP has been considered by Luczak, McNaughton, and Kubo [8]. This approach has been consider for the EP RL framework in [17], nevertheless this approach is not truly a single phase approach, in the fact that it requires a prediction phase, and thus is not truly compatible with the single physical circuit philosophy of EP. Following this remark, we propose, a true single phase framework similar to the stochastic Fisher EP proposal in Chapter 3, where the parameters are frozen in the nudge distribution, with a stochastic objective function defined solely as the Fisher divergence, this results in a different gradient rule, which enjoys a remarkable property of unlike any other stochastic framework, the deterministic limit, results on an exact gradient formula, which requires only a single nudge phase. Nevertheless, this proposal is still a heuristic (unlike, the others), and further work is still required to make it more rigorous (see Chapter 6 for details). This approach is compatible with both EP and LEP.

Finally, taking inspiration from the motivations of EP, which were, single circuit and forward only ML models for biological plausibility, we propose a heuristic novel DL architecture, which enjoys this properties. Further work is required for a more solid basis.

Chapter 2

Lagrangian Equilibrium Propagation for Graph Neural Networks

Graph Convolutional Networks (GCNs) can model relational data and capture both the local and global structure of the graphs. The attention mechanics can also be incorporated in Graph neural nets (GNNs), which yields the Graph Transformer model (GATs). The goal is to apply LEP PFVP and/or RHEL for the graph dynamics to learn data.

2.1 GraphCON (Graph-Coupled Oscillator Networks)

We consider the framework of GraphCON (Graph-Coupled Oscillator Networks) proposed by Rusch et al. [11].

Let $G = (V, E \subseteq V \times V)$ be an undirected graph with $|V| = v$ nodes and $|E| = e$ edges consisting of unordered pairs of nodes $\{i, j\}$, denoted by $i \sim j$. We label the nodes by the index $i \in V = \{1, 2, \ldots, v\}$. For any node $i \in V$, we denote its 1-neighborhood as:

$$N_i = \{j \in V : i \sim j\} \tag{2.1}$$

Furthermore, let $X \in \mathbb{R}^{v \times m}$ be given by $X = \{X_i\}$ for $i \in V$, denoting the m-dimensional feature vector at each node i.

Central to the GraphCON framework [11] is a graph dynamical system represented by the following nonlinear system of second-order ordinary differential equations (ODEs):

$$X'' = \sigma\big(F_\theta(X, t)\big) - \gamma X - \alpha X' \tag{2.2}$$

Here, $X(t)$ denotes the time-dependent $v \times m$ matrix of node features, σ is the activation function, and γ, α are positive hyperparameters governing the potential and damping forces respectively. The function F_θ is a general learnable (possibly time-dependent) 1-neighborhood coupling function of the form:

$$\big(F_\theta(X, t)\big)_i = F_\theta\big(X_i(t), X_j(t), t\big) \quad \forall j \in N_i \tag{2.3}$$

which is parameterized by a set of learnable parameters θ.

Remark

Time steps of the discretized ODE can be interpreted as layers of a deep neural network [24]. Indeed in this framework, the graph neural network is not defined as a stack of discrete layers but as a *continuous-time dynamical system* on a graph.

First-Order Dynamical Formulation. By introducing the auxiliary velocity variable $Y(t) = X'(t) \in \mathbb{R}^{v \times m}$, we can rewrite the second-order ODE system as an equivalent first-order system:

$$X' = Y \tag{2.4}$$

$$Y' = \sigma\big(F_\theta(X, t)\big) - \gamma X - \alpha Y \tag{2.5}$$

The central operational principle of this framework is to treat the input node features $X(0)$ as an initial condition for the dynamical system, and to utilize the state matrix solution $X(T)$ evaluated at a terminal time T as the final output. More generally, linear embedding transformations can be applied to both the initial state $X(0)$ and the terminal state $X(T)$. As demonstrated in [11], the underlying solution space of this system forms a rich class of functions capable of solving a wide variety of graph-structured learning tasks.

Remark 2.1.1. *The output which would be evaluated by the cost function is only the final state matrix $X(T)$ not the whole trajectory. In the original LEP PFVP derivation, the cost considered, evaluated the whole trajectory, nevertheless LEP PFVP is equivalent to RHEL as shown in [4], and RHEL can be reduce to HEB (Hamiltonian echo backpropagation is the original proposal of López-Pastor and Marquardt [31]), which is specifically designed for cost functions that evaluate only the final state as shown in [21], thus without further mathematical rigor, we consider that LEP PFVP can be applied to GraphCON (in short because it is equivalent to RHEL, and RHEL can be applied).*

Coupling Functions and Message Passing Mechanisms

The coupling function F_θ acts as the core message passing mechanism—frequently conceptualized as neighborhood aggregation or graph diffusion—analogous to traditional Graph Neural Networks (GNNs) [11]. The GraphCON framework natively accommodates any learnable 1-neighborhood coupling operator (including architectures such as GraphSAGE, SplineCNN, or MoNet). In particular, one of the configurations considered by Rusch et al. [11] is GraphCONGCN.

Convolutional Message Passing (GraphCONGCN)

Utilizing the localized spectral graph convolution operator introduced by Kipf and Welling [27], the coupling function is expressed as [11]:

$$F_\theta(X^n, t^n) = \hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} X^n W^n \tag{2.6}$$

where $\hat{A} = A + I_v$ represents the graph adjacency matrix modified with self-loops, and $\hat{D} \in \mathbb{R}^{v \times v}$ is the corresponding diagonal degree matrix with elements $\hat{D}_{kk} = \sum_{l=1}^{v} \hat{A}_{kl}$. The matrix $W^n \in \mathbb{R}^{m \times m}$ denotes the learnable layer-wise parameter weights.

Remark 2.1.2. *Considering the normalization of the adjacency matrix : $\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$, where $\hat{D}$ is the diagonal degree matrix of $\hat{A}$, keeps the scale of the aggregated signal comparable across nodes regardless of how many neighbors they have.*

Message passing. The term $\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}XW$ (with $W \in \mathbb{R}^{m \times m}$ a learnable weight matrix) is the network's *message-passing* operator: the row i of $\hat{A}_{\text{sym}}X$ is a weighted average of node i's own features and its neighbors' features, and right-multiplying by W mixes the m feature channels the same way at every node. This is exactly the propagation rule of a standard Graph Convolutional Network (GCN, Kipf and Welling [27]). The difference here is that instead of applying it once per discrete layer, it acts as a *continuous force* inside an ODE.

Comparison with Convolutional networks (CNNs). A CNN slides a small, fixed-size filter over a *regular grid* (an image), applying the *same* learned weights at every spatial location – this weight sharing is what gives CNNs translation invariance. The graph model here shares a similar spirit (the same W is applied at every node), but the "neighborhood" a node aggregates over is not a fixed geometric window; it is whatever the graph's edges say it is, and it can be a different size and shape at every node. The graph analogue of translation invariance is *permutation* invariance: relabeling the nodes and correspondingly permuting the rows of X and A does not change the computation, whereas shifting the input grid is what CNN weight sharing is built to handle.

Also a CNN (or a Transformer layer) does not have a notion of "time" or "momentum" – they are a fixed number of feedforward passes. This model is a genuine second-order ODE: node features have both a position (X) and a velocity ($Y = \dot{X}$), and the network's depth is a continuous integration time T rather than a layer count. That is what makes a physics-based training rule like PFVP applicable in the first place.

2.1.1 Deriving LEP for GraphCON

Physical interpretation. The term $\hat{A}_{\text{sym}}XW$ is the graph-coupling "spring" pulling each node's state towards a combination of its neighbors' states (message passing, as above).

$-\gamma X$ is an ordinary harmonic restoring force pulling every node back towards the origin, at a fixed rate $\gamma > 0$ (a hyperparameter here, not learned).

$-\alpha\dot{X}$ represents friction/damping, at rate $\alpha > 0$, which drains energy from the system over time and is what makes it eventually settle rather than oscillate forever.

W is the only learnable parameter of the dynamics and is constrained to be **symmetric** ($W = W^\top$). This constraint makes the coupling term a genuine *conservative force* (i.e. derivable from a scalar potential) which is what allows an exact Lagrangian/Hamiltonian description of the system and, in turn, what the LEP PFVP training rule needs to be valid (see below).

In code, W is stored as an unconstrained matrix and only ever read through its symmetrized version $\frac{1}{2}(W_{\text{raw}} + W_{\text{raw}}^\top)$, so the symmetry holds exactly at every step of training, with no extra projection needed.

The network's "output" is the terminal state $X(T)$ after integrating for a fixed time horizon T – the graph analogue of "the activation after the last layer".

Deriving the Hamiltonian for Graph-CON

The system is given by:

$$\begin{cases} X' = Y \\ Y' = \sigma(F_\theta(X,t)) - \gamma X - \alpha Y \end{cases} \tag{2.7}$$

From an analytical mechanics perspective, this system describes a collection of damped oscillators (via the $-\alpha Y$ term) that are forced and coupled through the graph topology (via the $\sigma(F_\theta)$ term). The presence of the damping term αY (when $\alpha \neq 0$) implies that the system is dissipative, meaning its total energy decreases over time. Consequently, a standard autonomous (time-independent) Hamiltonian cannot be constructed using the original state variables (X, Y). However, there is a rigorous theoretical framework that allows for a Hamiltonian formulation of this system. Indeed this is the same framework used by Pourcel et al. [4] to consider dissipative LEP.

The Caldirola-Kanai Hamiltonian (Time-Dependent)

This classical framework incorporates linear damping into the Hamiltonian formalism by introducing an explicit time dependence through an exponential scaling factor $e^{\alpha t}$ (see Caldirola [32] and Kanai [33]).

To apply this method, we assume that the graph coupling function $\sigma(F_\theta(X,t))$ is conservative, meaning it derives from a global interaction potential $V_{\text{inter}}(X,t)$ such that:

$$\nabla_X V_{\text{inter}}(X,t) = -\sigma(F_\theta(X,t)) \tag{2.8}$$

The total potential of the system (including the spring-like restoring term $-\gamma X$) is then expressed using the matrix trace (Tr) to sum over all node dimensions:

$$V(X,t) = \frac{\gamma}{2}\text{Tr}(X^\top X) + V_{\text{inter}}(X,t) \tag{2.9}$$

By defining the canonical conjugate momentum as $P = Ye^{\alpha t}$, the Caldirola-Kanai Hamiltonian is expressed as:

$$H(X,P,t) = e^{-\alpha t}\frac{1}{2}\text{Tr}(P^\top P) + e^{\alpha t}V(X,t) \tag{2.10}$$

Verification via Hamilton's Equations:

- **Position equation:**

$$X' = \frac{\partial H}{\partial P} = e^{-\alpha t}P = Y \implies X' = Y \tag{2.11}$$

- **Momentum equation:**

$$P' = -\frac{\partial H}{\partial X} = -e^{\alpha t}\nabla_X V(X,t) = -e^{\alpha t}\big(\gamma X - \sigma(F_\theta(X,t))\big) \tag{2.12}$$

Differentiating the definition $P = X'e^{\alpha t}$ with respect to time yields $P' = X''e^{\alpha t} + \alpha X'e^{\alpha t} = e^{\alpha t}(Y' + \alpha Y)$. Equating both expressions for P' and dividing through by $e^{\alpha t}$, we successfully recover the second-order equation of the system:

$$Y' = \sigma(F_\theta(X,t)) - \gamma X - \alpha Y \tag{2.13}$$

2.1.2 Hamiltonian for GraphCON-GCN

Linear activation function. In the case of GraphCON-GCN, the coupling function $F_\theta(X)$ is based on the localized graph convolution operator introduced by Kipf and Welling [27] :

$$F_\theta(X) = \hat{A}_{\text{sym}} X W \tag{2.14}$$

where $\hat{A}_{\text{sym}} = \hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$ represents the symmetrically normalized adjacency matrix incorporating self-loops.

Remark 2.1.3. *The GCN operator introduces a physical simplification: because the underlying graph G is undirected, the operator matrix $\hat{A}_{sym}$ is strictly symmetric ($\hat{A}_{sym} = \hat{A}_{sym}^\top$). This mathematical property enables the formulation of an exact Hamiltonian under specific structural constraints on the learnable parameters.*

2.1.3 Case 1: The Caldirola-Kanai Hamiltonian with linear activation function ($\sigma(x) = x$ and symmetric weight matrix $W = W^\top$)

By selecting a linear activation function ($\sigma(x) = x$) and constraining the weight matrix to be symmetric ($W = W^\top$) which is usually a requirement for EP and LEP [4], the GCN coupling reduces to a purely conservative force field. Consequently, it can be directly derived from a global interaction potential.

1. The GCN Interaction Potential Leveraging theproperties of the matrix trace (Tr), the potential energy associated with the GCN convolutional force is defined as:

$$V_{\text{inter}}(X) = -\frac{1}{2}\text{Tr}\big(X^\top \hat{A}_{\text{sym}} X W\big) \tag{2.15}$$

Proof via matrix gradient calculus: Using standard matrix derivative identities for bilinear forms, the gradient with respect to X is:

$$\nabla_X V_{\text{inter}}(X) = -\frac{1}{2}\big(\hat{A}_{\text{sym}} X W + \hat{A}_{\text{sym}}^\top X W^\top\big) \tag{2.16}$$

Since both $\hat{A}_{\text{sym}}$ and W are symmetric matrices ($\hat{A}_{\text{sym}}^\top = \hat{A}_{\text{sym}}$ and $W^\top = W$), this identity

simplifies directly to the expected driving force:

$$\nabla_X V_{\text{inter}}(X) = -\hat{A}_{\text{sym}} XW = -F_\theta(X) \tag{2.17}$$

2. The Total System Hamiltonian By incorporating the baseline harmonic restoring term γX and introducing the exponentially scaled canonical conjugate momentum $P = Ye^{\alpha t}$ (where $Y = X'$), the complete Caldirola-Kanai Hamiltonian for the GraphCON-GCN architecture is expressed as:

$$H(X, P, t) = e^{-\alpha t}\frac{1}{2}\text{Tr}(P^\top P) + e^{\alpha t}\left[\frac{\gamma}{2}\text{Tr}(X^\top X) - \frac{1}{2}\text{Tr}\big(X^\top \hat{A}_{\text{sym}} XW\big)\right] \tag{2.18}$$

Application of dissipative LEP (PFVP) for GraphCON-GCN with linear activation function ($\sigma(x)$ and $W = W^\top$)

For

$$H(X, P, t) = e^{-\alpha t}\frac{1}{2}\text{Tr}(P^\top P) + e^{\alpha t}\left[\frac{\gamma}{2}\text{Tr}(X^\top X) - \frac{1}{2}\text{Tr}\big(X^\top \hat{A}_{\text{sym}} XW\big)\right] \tag{2.19}$$

The Lagrangian associated with $H(X, P, t)$ is:

$$L(X, \dot{X}, t) = e^{\alpha t}\left[\frac{1}{2}\text{Tr}(\dot{X}^\top \dot{X}) - \frac{\gamma}{2}\text{Tr}(X^\top X) + \frac{1}{2}\text{Tr}\left(X^\top \hat{A}_{\text{sym}} XW\right)\right]$$

This is a matrix-valued Caldirola–Kanai type Lagrangian with time-dependent exponential damping $e^{\alpha t}$.

Here is the step-by-step derivation :

1. Compute Velocity from Hamilton's Equation :

Differentiating H with respect to the conjugate momentum matrix P yields the velocity $\dot{X}$:

$$\dot{X} = \frac{\partial H}{\partial P} = e^{-\alpha t} P$$

2. Express Momentum in Terms of Velocity : Inverting the velocity relation

$$P = e^{\alpha t}\dot{X}$$

3. Perform the Inverse Legendre Transformation:

The Lagrangian is obtained via $L(X, \dot{X}, t) = \text{Tr}(P^\top \dot{X}) - H(X, P, t)$

$$L(X, \dot{X}, t) = \text{Tr}\left((e^{\alpha t}\dot{X})^\top \dot{X}\right) - \left(e^{-\alpha t}\frac{1}{2}\text{Tr}\left((e^{\alpha t}\dot{X})^\top (e^{\alpha t}\dot{X})\right) + e^{\alpha t}\left[\frac{\gamma}{2}\text{Tr}(X^\top X) - \frac{1}{2}\text{Tr}\big(X^\top \hat{A}_{\text{sym}} XW\big)\right]\right)$$

4. Simplify Terms Combining the trace kinetic terms:

$$L(X, \dot{X}, t) = e^{\alpha t}\text{Tr}(\dot{X}^\top \dot{X}) - \frac{1}{2}e^{\alpha t}\text{Tr}(\dot{X}^\top \dot{X}) - e^{\alpha t}\left[\frac{\gamma}{2}\text{Tr}(X^\top X) - \frac{1}{2}\text{Tr}\big(X^\top \hat{A}_{\text{sym}} XW\big)\right]$$

Factoring out $e^{\alpha t}$ yields the final result:

$$L(X, \dot{X}, t) = e^{\alpha t} \left[\frac{1}{2} \mathrm{Tr}(\dot{X}^\top \dot{X}) - \frac{\gamma}{2} \mathrm{Tr}(X^\top X) + \frac{1}{2} \mathrm{Tr}\big(X^\top \hat{A}_{\mathrm{sym}} X W\big) \right]$$

2.1.4 Experiment for GraphCON (linear case) with LEP PFVP

We implement the graph CON model with linear activation function $\sigma(x) = x$ and symmetric learnable matrix parameters $W = W^\top$ by considering the Lagrangian :

$$L(X, \dot{X}, t) = e^{\alpha t} \Big[\underbrace{\tfrac{1}{2} \mathrm{Tr}(\dot{X}^\top \dot{X}) - \tfrac{\gamma}{2} \mathrm{Tr}(X^\top X) + \tfrac{1}{2} \mathrm{Tr}(X^\top \hat{A}_{\mathrm{sym}} X W)}_{L_0(X, \dot{X})} \Big]. \tag{2.20}$$

We consider the dissipative LEP PFVP algorithm [4].

Remark 2.1.4. *We follow the notations of the Graph CON paper of Rusch et al. [11], thus X represents the dynamical state.*

Free phase, echo phase, and the gradient estimate. Training proceeds in two passes, both using nothing but the forward ODE integrator:

1. **Free phase.** Integrate the system forward from a fixed initial condition $(X_0, Y_0{=}0)$ to T, with no reference to the target at all. Record the whole trajectory.

2. **Echo (nudged) phase.** Bounce off the end of the free trajectory by flipping the velocity, $(X(T), -Y(T))$, and integrate forward again, but with the sign of the damping term *flipped* (so friction becomes a gentle energy pump) and a small forcing term added, proportional to a nudging strength β and to the terminal error $X_{\mathrm{target}} - X(T)$, decaying exponentially as the echo moves away from the bounce point.

The parameter gradient is then read off as a simple, local, time-integrated difference between quantities evaluated along the two trajectories (no backward pass through the solver is ever taken). As $\beta \to 0$, this estimator converges to the true gradient of the loss with respect to W.

Only the matrix weight W represent the learnable parameters in Graph CON Furthermore. Due to the structure of this Lagrangian, we have :

$$\frac{\partial}{\partial W} \frac{\partial}{\partial \dot{X}} L(X, \dot{X}, W, t) = \frac{\partial}{\partial W} e^{\alpha t} \dot{X} = 0$$

Thus only the bulk term remains in the dissipative LEP PFVP gradient estimator, when considering fixed initial conditions. In our case, we take : $X(0) = X_0$ (a constant, W-independent) and initial velocity $Y(0) = 0$ (also W-independent). The gradient formula is

$$\nabla_W C \;\approx\; \frac{1}{\beta} \int_0^T e^{\alpha t} \Big[\partial_W L_\beta(X_t^\beta) - \partial_W L_0(X_t^0) \Big] \, dt$$

Simulated Dataset

We start first by considering a purely hand-written edge list which consists in a synthetic hexagon-plus-chord graph and a random target matrix, to validate the LEP PFVP algorithm for dynamical graphs with Graph CON. We consider a small, synthetic 6-node graph (a hexagon with one extra chord) with $m = 3$ features per node and an arbitrary random target matrix X_{target} to hit at T.

We perform a gradient check, the PFVP estimate of $\partial_W(\text{loss})$ is compared, entry by entry, to the true gradient from full backpropagation, across a range of β values. Cosine similarity peaks around 0.97–0.99 for a broad middle range of β (roughly 0.1 to 0.5 here), degrading gracefully for very large β (too much bias).

The loss curve is illustrated in Figure 2.1. It shows that LEP PFVP performs closely to BPTT. We considered 10 random initializations, and the average loss curve.

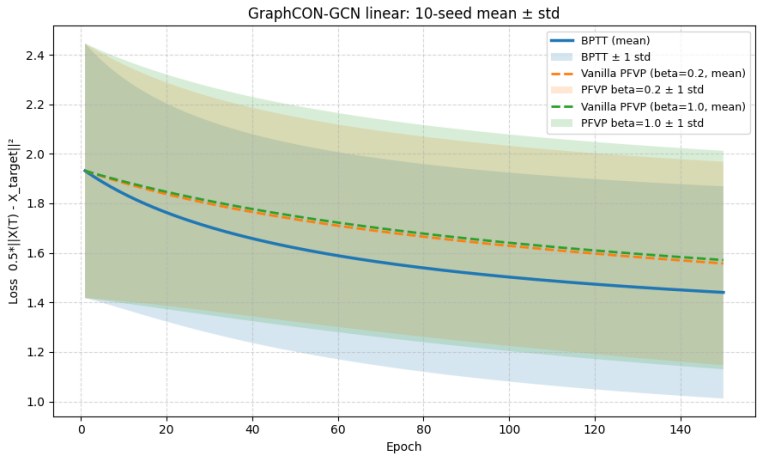


Figure 2.1: Cost evolution of LEP PFVP with Graph CON on simulated data(linear case).

The Python notebook (google colab) for this linear activation function version is accessible at :

https://colab.research.google.com/drive/1S-e3WvQrncZb3Ug0axwh1i_Z7strcVNK?usp=sharing

2.1.5 Case 2: The Caldirola-Kanai Hamiltonian (Non linear activation function and symmetric weight matrix $W = W^\top$)

We consider the following 2nd order differential equation with non linear activation function

$$\ddot{X} = \hat{A}_{\text{sym}} \tanh\big(\hat{A}_{\text{sym}} X W\big)\, W \;-\; \gamma X \;-\; \alpha \dot{X}.$$

The term $\hat{A}_{\text{sym}} \tanh\big(\hat{A}_{\text{sym}} X W\big)\, W$ represents the conservative force part. This is the gradient of a scalar potential:

$$\Phi(X) = \sum_{i,j} \log\cosh\left(\hat{A}_{\text{sym}} X W\right)_{ij} \quad \Longrightarrow \quad \nabla_X \Phi(X) = \hat{A}_{\text{sym}} \tanh(\hat{A}_{\text{sym}} X W) W$$

(using $\hat{A}_{\text{sym}} = \hat{A}_{\text{sym}}^\top$ and $W = W^\top$)

This represents a generalization of the UniCORNN for graphs of Rusch and Mishra [34].

The Lagrangian associated to this dynamics is (derived in the same manner as the case of linear activation function) :

$$L(X, \dot{X}, t) = e^{\alpha t}\left[\frac{1}{2}\mathrm{Tr}(\dot{X}^\top \dot{X}) - \frac{\gamma}{2}\mathrm{Tr}(X^\top X) + \sum_{ij} \log\cosh(\hat{A}_{\mathrm{sym}} X W)_{ij}\right]$$

2.1.6 Experiment for Graph CON (non linear case)

2.1.7 Real dataset : Zachary's Karate Club

Dataset. We consider a small supervised classification dataset : Zachary's Karate Club [35] which consists of 34 members of a university karate club, 78 edges of observed friendship, and a famous historical fact: the club split into two factions after a dispute between the instructor ("Mr. Hi") and the administrator ("Officer"). Every member's eventual faction is known, which turns the graph into a natural two-class **node classification** problem. It is real, small, and requires no data cleaning, making it the smallest standard benchmark for graph-based node classification.

Task and features. The task is to predict each node's faction from the graph structure alone. Karate Club carries no intrinsic per-node attributes (node features), there is no equivalent of "pixel values" here, only structure. Following standard practice for featureless graphs, each node is given a small random embedding as its initial state $X_0 \in \mathbb{R}^{34\times 4}$; any classification power has to come entirely from the graph-coupled dynamics reshaping these random vectors as they evolve (we let the dynamics — driven purely by $\hat{A}_{\mathrm{sym}}$ and the learned W — separate the two factions by the time we reach $X(T)$). After integrating to T, a learnable linear **readout** matrix $R \in \mathbb{R}^{4\times 2}$ turns the terminal state into class logits, $X(T)R$, trained with cross-entropy against the known faction labels. The readout sits outside the physical system entirely (it never appears in the Lagrangian), so it is trained with an ordinary local gradient step; only W, the dynamics' own parameter, is trained through the free/echo mechanism above. We use full supervision (all 34 nodes labeled), which gives a well-conditioned, well-distributed error signal.

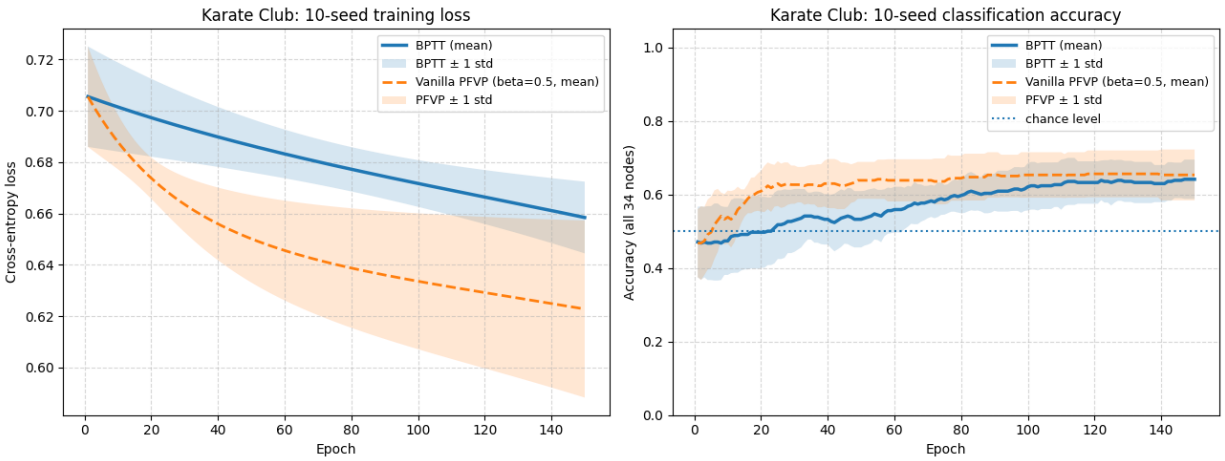


Figure 2.2: Cost evolution of LEP PFVP with Graph CON on Karate Club (non linear case).

As illustrated in Figure 2.2, the accuracy of LEP PFVP is close (slightly better here) to BPTT,

The Python notebook (google colab) for this non linear version is accessible at :

https://colab.research.google.com/drive/1w6PENl_FmsWw4GJFTSklRx94C4gg768n?usp=sharing

2.2 Quadratic HANG Formulation

Here we consider the Quadratic HANG framework proposed by Zhao et al. [28]. The Quadratic Hamiltonian Network (*HANG-quad*) models the system using a separable Hamiltonian structure where the kinetic energy $T(q,p)$ depends quadratically on the generalized momenta, and the potential energy $U(q)$ depends strictly on the generalized coordinates:

$$H = T(q,p) + U(q) \tag{2.21}$$

Remark 2.2.1. *We use the usual notation of Hamiltonian systems, where q represents the generalized position, and p represents the generalized momentum.*

Kinetic Energy Term $T(q,p)$

The kinetic energy function acts as a parameterized Riemannian metric across the graph topology, defined as:

$$T(q,p) = \sum_{k \in V} p_k^\top \left(A_G(q_k, q_k) A_G^\top(q_k, q_k) + \sigma I \right) p_k \tag{2.22}$$

Where the internal components are defined as follows:

- $p_k \in \mathbb{R}^r$ is the generalized momentum vector for node k.
- $A_G(q_k, q_k)$ represents a structural graph configuration matrix, which can either be instantiated as the static graph adjacency matrix A or as a dynamically learnable attention matrix.
- $\sigma > 0$ is a small positive regularization constant ensuring strict positive definiteness of the metric matrix, guaranteeing that $T(q,p) > 0$ for all $p \neq 0$.
- I is the identity matrix of appropriate dimensions.

In practice, $A_G(q_k) \in \mathbb{R}^{r \times r}$ is produced from the node's own current state q_k by a small learnable linear "hypernetwork" (a single matrix Θ reshaping q_k into an $r \times r$ matrix).

Potential Energy Term $U(q)$

Theoretically, the baseline potential landscape can be configured using standard geometric constraints on the coordinates:

$$U(q) = \|q\|_2 \quad \text{or} \quad U(q) = \|\sin(q)\|_2 \tag{2.23}$$

In practice, to capture higher-order relational dependencies across neighborhoods, the potential energy $U(q)$ is implemented using a specialized Graph Attention Network (GAT) architecture:

$$U(q) = \|\sin(\text{ggat}(q))\|_2 \tag{2.24}$$

Where:

- $\text{ggat} : \mathbb{R}^{r\times|V|} \to \mathbb{R}^{r\times|V|}$ represents a single-layer Graph Attention Network that projects the coordinate state matrix.
- $\sin(\cdot)$ denotes the element-wise sine activation function, injecting non-linear periodic geometric priors into the potential landscape.
- $\|\cdot\|_2$ is the global ℓ_2 norm tracking total coordinate-based potential energy across the graph.

In practice, for $U(q)$ we consider a standard recipe: a learned linear projection Wq, an attention logit $e_{ij} = \text{LeakyReLU}(a^\top[Wq_i \| Wq_j])$ computed *only for graph neighbors* j (all other pairs are masked out before the softmax), normalized per node with a softmax, and the neighbor features combined with these learned weights. The elementwise $\sin(\cdot)$ nonlinearity, followed by an overall norm, injects a periodic geometric prior into the energy landscape – again this follows the original HANG-quad construction directly.

2.2.1 Application of LEP For Quadratic HANG

We consider the Hamiltonian

$$H(q,p) = \sum_{k\in V} p_k^\top \left(A_G(q_k,q_k)A_G^\top(q_k,q_k) + \sigma I\right) p_k + \|\sin(\text{ggat}(q))\|_2$$

This Hamiltonian can be applied directly with RHEL.

The Lagrangian associated with $H(q,p)$ is:

$$L(q,\dot{q}) = \frac{1}{4}\sum_{k\in V} \dot{q}_k^\top \left(A_G(q_k,q_k)A_G^\top(q_k,q_k) + \sigma I\right)^{-1} \dot{q}_k - \|\sin(\text{ggat}(q))\|_2$$

Here is the step-by-step derivation

1. Define Metric Matrix and Potential

Let $G_k(q_k) = A_G(q_k,q_k)A_G^\top(q_k,q_k) + \sigma I$ and $V(q) = \|\sin(\text{ggat}(q))\|_2$. The Hamiltonian simplifies to:

$$H(q,p) = \sum_{k\in V} p_k^\top G_k(q_k) p_k + V(q)$$

2. Compute Velocity via Hamilton's Equation

Differentiating $H(q,p)$ with respect to momentum p_k:

$$\dot{q}_k = \frac{\partial H}{\partial p_k} = 2G_k(q_k)p_k$$

3. Invert to Express Momentum in Terms of Velocity

Since $G_k(q_k)$ is symmetric positive-definite for $\sigma > 0$, it is strictly invertible:

$$p_k = \frac{1}{2} G_k(q_k)^{-1} \dot{q}_k = \frac{1}{2} \left(A_G(q_k, q_k) A_G^\top(q_k, q_k) + \sigma I \right)^{-1} \dot{q}_k$$

4. Perform the Inverse Legendre Transformation

Applying $L(q, \dot{q}) = \sum_{k \in V} p_k^\top \dot{q}_k - H(q, p)$:

Legendre coupling term:

$$\sum_{k \in V} p_k^\top \dot{q}_k = \sum_{k \in V} \left(\frac{1}{2} G_k(q_k)^{-1} \dot{q}_k \right)^\top \dot{q}_k = \frac{1}{2} \sum_{k \in V} \dot{q}_k^\top G_k(q_k)^{-1} \dot{q}_k$$

Hamiltonian kinetic term:

$$\sum_{k \in V} p_k^\top G_k(q_k) p_k = \sum_{k \in V} \left(\frac{1}{2} G_k(q_k)^{-1} \dot{q}_k \right)^\top G_k(q_k) \left(\frac{1}{2} G_k(q_k)^{-1} \dot{q}_k \right) = \frac{1}{4} \sum_{k \in V} \dot{q}_k^\top G_k(q_k)^{-1} \dot{q}_k$$

5. Combine Terms

Subtracting the kinetic and potential terms yields the final Lagrangian:

$$L(q, \dot{q}) = \left(\frac{1}{2} - \frac{1}{4} \right) \sum_{k \in V} \dot{q}_k^\top G_k(q_k)^{-1} \dot{q}_k - V(q)$$

$$L(q, \dot{q}) = \frac{1}{4} \sum_{k \in V} \dot{q}_k^\top \left(A_G(q_k, q_k) A_G^\top(q_k, q_k) + \sigma I \right)^{-1} \dot{q}_k - \| \sin\left(\mathrm{ggat}(q)\right) \|_2$$

Remark on matrix inversion

It is more efficient to consider the Hamiltonian formalism, and thus to consider RHEL, rather than LEP PFVP, because, the Lagrangian formalism requires the inversion of the matrix $A_G(q_k, q_k) A_G^\top(q_k, q_k) + \sigma I$.

2.2.2 Experiment for HANG quad

The first two implementations (Graph CON, above) used *damped* graph oscillator networks: energy continually drains out of the system as it evolves, which is what makes the network's state settle into a well-defined answer by the time horizon T. In this approach we have something different: a graph ODE with *no damping at all*, a genuinely conservative Hamiltonian system, in which total energy is exactly constant over time.

We consider a **graph attention (GAT) layer**, of the kind popularized by Veličković et al. [36]. Where the first two implementations where based on GCNs which meant they aggregate neighbors with a single fixed weight per edge (from the normalized adjacency matrix), a GAT layer computes an *attention score* for each edge, learned from the current node states, and normalizes it (via a softmax) over each node's neighborhood.

Graphs and Features. As before, node states are stored as a matrix $q \in \mathbb{R}^{v \times r}$ (one r-dimensional row per node; we use the symbol q rather than X deliberately, borrowing the

"generalized position" notation of classical mechanics). What is new is that the state now comes in a *pair*: alongside the position-like q sits a momentum-like variable $p \in \mathbb{R}^{v \times r}$, and the pair (q, p) together, not q alone, is what evolves.

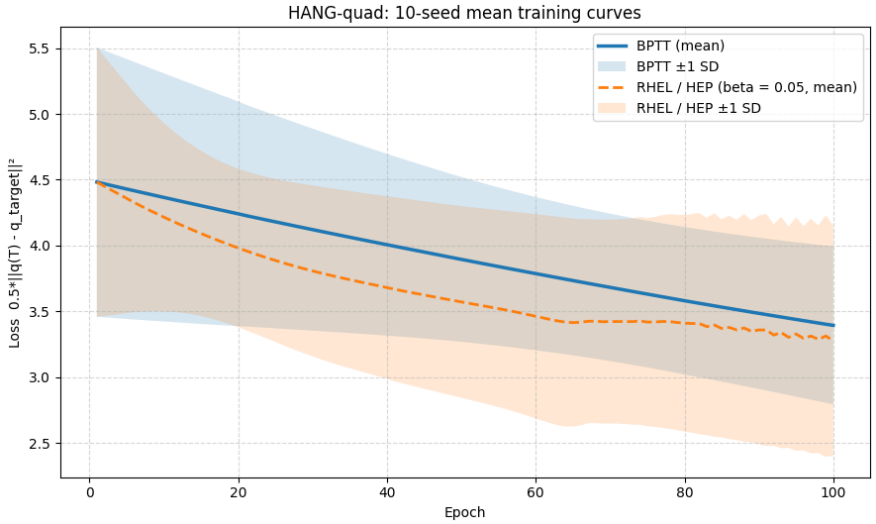


Figure 2.3: Cost evolution of RHEL with Hang-quad on simulated data.

As illustrated in Figure 2.3, RHEL offers a similar loss curve as BPTT on the simulated data.

We consider the same node-classification task as the other two previous implementations : Zachary's Karate Club, 34 nodes, two factions, full supervision with a linear readout $q(T)R$ feeding a cross-entropy loss.

As illustrated in Figure 2.4, RHEL applied to the Hangquad structure, reaches faster convergence with an average accuracy slightly above 55%, which makes it less performative than LEP PFVP applied with the Graph CON architecture above (see Figure 2.2). Interestingly, in this particular case, RHEL which represents an EP like rule performs better than BPTT. This result mirrors a similar finding by Kubo, Chalmers, and Luczak [17] , where EP based methods showed better performance than BP based method.

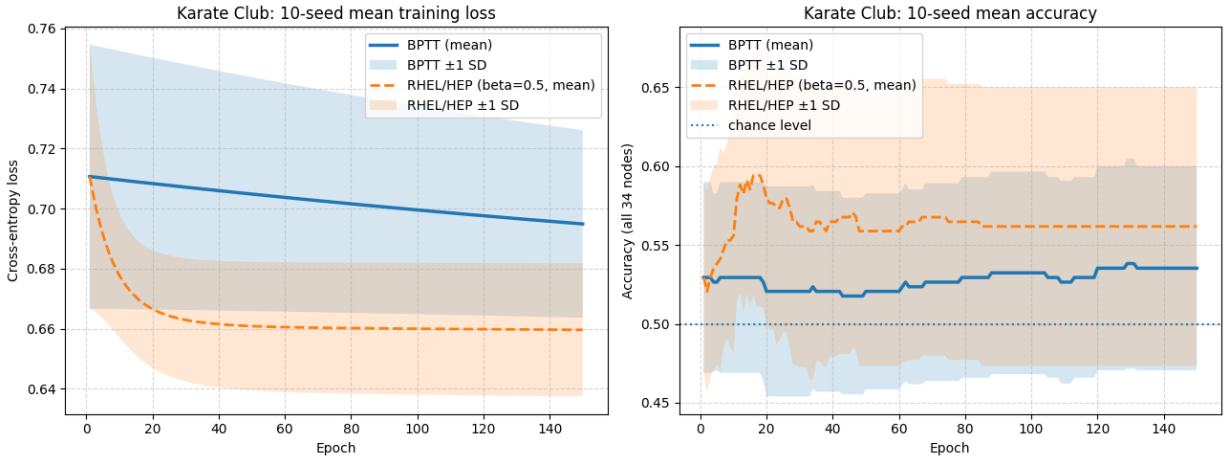


Figure 2.4: Cost evolution of RHEL with Hang-quad on Karate Club.

The Python notebook (google colab) is accessible at :

https://colab.research.google.com/drive/1MknYFtCeIYnWKBAt6tVn4vOoIZU7mKMI?usp=sharing

Chapter 3

EP with Large error signals

3.1 Fourier EP

3.1.1 Alternative to Holomorphic EP via Fourier transform

A proposal that solves the infinitesimal β issue is Holomorphic EP and is based on complex analysis using the holomorphic function approach. Indeed it was showed by Laborieux and Zenke [13] that if the augmented energy functions E_β is holomorphic, i.e., differentiable in the sense of complex, then holomorphic EP computes the gradient of the objective function C for finite β. They first generalized the result of Scellier and Bengio [18] in the complex case

$$\lim_{\beta \to 0} \frac{1}{\beta} \left(\frac{\partial E_\beta}{\partial \theta}(\theta, s^*_\beta, \beta, y) - \frac{\partial E_\beta}{\partial \theta}(\theta, s^*_0, 0, y) \right) = \frac{dC}{d\theta} \tag{3.1}$$

and then showed that for : $|\beta| > 0$ the radius of a circular path around 0 in $\mathbb{C}$ contained in the open set U on which the fixed point $s^*_{\theta,\beta}$ is defined, and further assume that this path is parameterized by $t \in [0, T] \mapsto \beta(t) = |\beta| e^{2i\pi t/T}$, where i is the imaginary unit. Then the loss gradient is given by:

$$\frac{dC}{d\theta} = \frac{1}{T|\beta|} \int_0^T \frac{\partial E_\beta}{\partial \theta} \left(\theta, s^*_{\theta,\beta(t)}, \beta(t) \right) e^{-2i\pi t/T} dt \tag{3.2}$$

This result is based on Cauchy's Integral Formula for Derivatives :

In complex analysis, if f is a holomorphic function on an open set $U \subseteq \mathbb{C}$, and γ is any piece-wise continuously differentiable closed curve in U that encloses a point $a \in U$ exactly once in the counterclockwise direction, then the n-th derivative of f at point a is given by:

$$f^{(n)}(a) = \frac{n!}{2i\pi} \oint_\gamma \frac{f(z)}{(z-a)^{n+1}} dz \tag{3.3}$$

3.1.2 Approach with Fourier Transform: Fourier EP

The key idea used with the Holomorphic EP is to avoid an explicit calculation of the derivative and to use instead Cauchy's formula, provided that the energy function is Holomorphic : $f'(a) = \frac{1}{2i\pi} \oint_\gamma \frac{f(z)}{(z-a)^2} dz$

Another similar approach would rely on the Fourier transform, indeed when considering the Fourier transform of the augmented energy function over the nudge parameter β :

$$\mathcal{T}(F)(k) = \hat{F}(k) = \int_{\mathbb{R}} E_\beta(\theta, s^*_{\theta,\beta}, \beta) e^{-2\pi i k\beta} d\beta$$

We can then use the Fourier transform property of the derivative

$$\mathcal{T}(d_\beta E_\beta)(k) = 2i\pi k \mathcal{T}(E_\beta)(k)$$

and thus the derivative can be obtained by the reverse Fourier transform

$$\boxed{d_\beta E_\beta = \mathcal{T}^{-1}(\mathcal{T}(d_\beta E_\beta)(k))(\beta) = \mathcal{T}^{-1}(2i\pi k \mathcal{T}(E_\beta)(k))(\beta) = \int_{\mathbb{R}} 2i\pi k \hat{E}_\beta(k) e^{2\pi i k\beta} dk}$$

particularly for $\beta = 0$ we have

$$\boxed{d_\beta E_\beta \mid_{\beta=0} = \int_{\mathbb{R}} 2i\pi k \hat{E}_\beta(k) dk = \int_{\mathbb{R}} 2i\pi k \left(\int_{\mathbb{R}} E_\beta(\theta, s^*_{\theta,\beta}, \beta) e^{-2\pi i k\beta} d\beta \right) dk}$$

The advantage of this approach is that we can theoretically consider all possible values of β with symmetry, thus the cost can be fully considered for all possible nudge values, and then we compute the derivative at $\beta = 0$ to use the classical EP rule.

Another advantage of such approach, is that the energy function F remains real valued, and thus has not be Holomorphic (the analyticity condition in $\mathbb{C}$) which is a strong requirement and is not satisfied in general, mainly because the cost function C is usually real valued and thus can't satisfy the Riemann Cauchy equations and be Holomorphic.

In application we would consider

$$\boxed{d_\beta \frac{\partial}{\partial_\theta} E_\beta \mid_{\beta=0} = \int_{\mathbb{R}} 2i\pi k \mathcal{T}\left(\frac{\partial}{\partial_\theta} E_\beta\right)(k) dk = \int_{\mathbb{R}} 2i\pi k \left(\int_{\mathbb{R}} \frac{\partial}{\partial_\theta} E_\beta(\theta, s^*_{\theta,\beta}, \beta) e^{-2\pi i k\beta} d\beta \right) dk}$$

3.1.3 Fourier dissipative PFVP LEP

Reminder of dissipative PFVP LEP Dissipative PFVP LEP is based on the following update formula [4] :

$$d_\theta \mathcal{C}[s^0_\leftarrow(\theta)] = \lim_{\beta \to 0} \Delta_{\text{PFVP}}(\beta, \alpha_0(\theta), \gamma_0(\theta))$$

$$\begin{aligned}
\Delta_{\text{PFVP}}(\beta, \alpha_0, \gamma_0) := \frac{1}{\beta} \Bigg[& \int_0^T \exp(\zeta t) \cdot \left(\partial_\theta L_\beta(s^\beta_{\leftarrow,t}, \dot{s}^\beta_{\leftarrow,t}, \theta) - \partial_\theta L_0(s^0_{\leftarrow,t}, \dot{s}^0_{\leftarrow,t}, \theta) \right) dt \\
& + \left(d_\theta \partial_{\dot{s}} L_0(\alpha_0, \gamma_0, \theta) \right)^\top \left(s^\beta_{\leftarrow,0} - \alpha_0 \right) \\
& - \left(\partial_\theta \alpha_0 \right)^\top \left(\partial_{\dot{s}} L_\beta(s^\beta_{\leftarrow,0}, \dot{s}^\beta_{\leftarrow,0}, \theta) - \partial_{\dot{s}} L_0(\alpha_0, \gamma_0, \theta) \right) \Bigg],
\end{aligned}$$

Remark 3.1.1. *The following term is time independent, which enables a procedure similar to the static EP case :*

$$\begin{aligned} K_{\mathrm{PFVP}}(\beta, \alpha_0, \gamma_0) = \\ \int_0^T \exp(\zeta t) \cdot \left(\partial_\theta L_\beta(s^\beta_{\leftarrow,t}, \dot{s}^\beta_{\leftarrow,t}, \theta) - \partial_\theta L_0(s^0_{\leftarrow,t}, \dot{s}^0_{\leftarrow,t}, \theta) \right) dt \\ + \left(d_\theta \partial_{\dot{s}} L_0(\alpha_0, \gamma_0, \theta)\right)^\top \left(s^\beta_{\leftarrow,0} - \alpha_0 \right) \\ -(\partial_\theta \alpha_0)^\top \left(\partial_{\dot{s}} L_\beta(s^\beta_{\leftarrow,0}, \dot{s}^\beta_{\leftarrow,0}, \theta) - \partial_{\dot{s}} L_0(\alpha_0, \gamma_0, \theta) \right) \end{aligned}$$

we thus start from the main equation of dissipative PFVP LEP

$$d_\theta C(s^0(\theta)) = d_\beta K_{PFVP}(\beta, \alpha_0, \gamma_0)|_{\beta=0}$$

with $K_{PFVP}(\beta, \alpha_0, \gamma_0) = \beta \cdot \Delta_{\mathrm{PFVP}}(\beta, \alpha_0, \gamma_0)$

Thus similarly to the static EP case we consider the formula using the properties of the Fourier transform

$$d_\beta K_{PFVP}\,|_{\beta=0} = \int_{\mathbb{R}} 2i\pi k \hat{K}_{PFVP}(k) dk = \int_{\mathbb{R}} 2i\pi k (\int_{\mathbb{R}} K_{PFVP}(\theta, s^*_{\theta,\beta}, \beta) e^{-2\pi i k\beta} d\beta) dk$$

By reformulating the point-derivative $\frac{dK_{\mathrm{PFVP}}}{d\beta}\Big|_{\beta=0}$ in the frequency domain, we convert a local, potentially ill-conditioned limit ($\beta \to 0$) into a global spectral integral over the nudging parameter β.

The identity follows directly from the inverse Fourier transform property. If we define the forward Fourier transform of $K_{\mathrm{PFVP}}(\beta)$ as:

$$\hat{K}_{\mathrm{PFVP}}(k) = \int_{\mathbb{R}} K_{\mathrm{PFVP}}(\beta)\, e^{-2\pi i k\beta} d\beta$$

The inverse transform expresses $K_{\mathrm{PFVP}}(\beta)$ as:

$$K_{\mathrm{PFVP}}(\beta) = \int_{\mathbb{R}} \hat{K}_{\mathrm{PFVP}}(k)\, e^{2\pi i k\beta} dk$$

Differentiating both sides with respect to β and evaluating at $\beta = 0$:

$$\frac{dK_{\mathrm{PFVP}}}{d\beta}(\beta) = \int_{\mathbb{R}} (2\pi i k) \hat{K}_{\mathrm{PFVP}}(k)\, e^{2\pi i k\beta} dk \implies \frac{dK_{\mathrm{PFVP}}}{d\beta}\Big|_{\beta=0} = \int_{\mathbb{R}} 2\pi i k \hat{K}_{\mathrm{PFVP}}(k)\, dk$$

Since $K_{\mathrm{PFVP}}(\beta)$ is real-valued, $\hat{K}_{\mathrm{PFVP}}(-k) = \hat{K}^*_{\mathrm{PFVP}}(k)$, which guarantees that the imaginary components cancel out and the integral yields a purely real gradient vector.

3.1.4 Proposal for numerical implementation

Approach A: Spectral Differentiation on a Uniform β-Grid (FFT)

With this approach, we sample $K_{\text{PFVP}}(\beta)$ at N evenly spaced points across a symmetric interval $\beta \in [-B, B]$ with step size $\Delta\beta = \frac{2B}{N}$:Compute $K_{\text{PFVP}}(\beta_n)$ for $\beta_n = -B + n\Delta\beta$ for $n = 0, 1, \ldots, N$

Then w compute the Discrete Fourier Transform (DFT) via FFT:

$$\hat{K}[m] = \text{FFT}\left(\{K_{\text{PFVP}}(\beta_n)\}_{n=0}^{N-1}\right)$$

Define the discrete frequency grid $k_m = \frac{m}{2B}$ (with $m \in [-N/2, N/2 - 1]$).

Finally we evaluate the gradient by summing over the frequencies:

$$\frac{dK_{\text{PFVP}}}{d\beta}\bigg|_{\beta=0} \approx \sum_{m=-N/2}^{N/2-1} 2\pi i k_m \hat{K}[m]$$

Approach B: Harmonic / Circular Sampling (Phase-Nudge)

If we evaluate K_{PFVP} along a harmonic trajectory $\beta(\phi) = \beta_0 e^{i\phi}$ or sinusoidal oscillation $\beta(\theta) = \beta_0 \cos(\theta)$ around 0, $K_{\text{PFVP}}(\beta)$ becomes a periodic function of the phase angle ϕ. In this case, $\frac{dK_{\text{PFVP}}}{d\beta}\Big|_{\beta=0}$ reduces directly to the first Fourier coefficient c_1 of the periodic response:

$$\frac{dK_{\text{PFVP}}}{d\beta}\bigg|_{\beta=0} \approx \frac{1}{\pi\beta_0} \int_0^{2\pi} K_{\text{PFVP}}(\beta_0 \cos\phi) \cos(\phi)\, d\phi$$

Remark 3.1.2. *This approach yields a similar gradient rule to Holomorphic EP [13] without the constraint of Holomorphic function.*

Computational Cost: Instead of 1 evaluation (Single-Nudge) or 2 evaluations (Double-Nudge), a Fourier/spectral approach requires evaluating $K_{\text{PFVP}}(\beta)$ across N sample points per parameter update (thus N nudges). It is thus more computationally expensive than regular LEP or EP, but the number of sample points can be considered as a hyperparameter which in practice would be small, comparison of the dimension of the parameters or state (or also the time length of the trajectory for LEP), and it still offers, the possibility of considering large error signals. (here N is not the number of discrete time steps).

3.2 Stochastic EP

We now consider the framework of stochastic EP which was proposed by Scellier [14]. We start by a recall. For the illustration, without losing generality, we follow the framework proposed by Scellier [14]. The system dynamics are defined by the Stochastic Differential Equation (SDE) (an overdamped Langevin dynamics/diffusion):

$$dS_t^\beta = b(S_t^\beta, \beta)dt + \sigma(S_t^\beta, \beta)dB_t \tag{3.4}$$

Here we consider $b = -\nabla_s E - \beta \nabla_s C$

If the SDE is an overdamped Langevin diffusion with additive noise ($\sigma(x, \theta) = \sigma I$), the exact analytical form of the unique invariant measure $p_\beta^*(s)$ is known explicitly via the

Gibbs-Boltzmann distribution [14]:

$$p^*_\beta(s) = \frac{1}{Z(\beta)} \exp\left(-\frac{2}{\sigma^2}\Big(E(\theta, x, s) + \beta C(s, y)\Big)\right)$$

We have :

$$\frac{d}{d\beta}\bigg|_{\beta=0} \mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] = -\frac{2}{\sigma^2}\,\mathrm{Cov}_{s\sim p^*_0}\left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y)\right)$$

We use Scellier [14] notation's of temperature $T = \frac{\sigma^2}{2}$

3.2.1 Linking Litman [15], Massar and Mognetti [30], and Scellier [14]

Using the result :

$$\frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] = -\frac{2}{\sigma^2}\,\mathrm{Cov}_{s\sim p^*_\beta}\left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y)\right)$$

we can further apply the Fundamental Theorem of Calculus

$$\int_0^1 \frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] d\beta = \mathbb{E}_{s\sim p^*_{\beta=1}}\left[\frac{\partial E}{\partial \theta}\right] - \mathbb{E}_{s\sim p^*_{\beta=0}}\left[\frac{\partial E}{\partial \theta}\right] = \nabla_\theta J(\theta)$$

Which is the gradient of the objective function considered by Litman (2026), and thus we recover his fundamental result

$$\nabla_\theta J(\theta) = \int_0^1 -\frac{2}{\sigma^2}\,\mathrm{Cov}_{s\sim p^*_\beta}\left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y)\right) d\beta$$

In some sense Litman's gradient [15] represent the β integral of the gradient of the cost :

$$\nabla_\theta J(\theta) = \int_0^1 \frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] d\beta = \int_0^1 \frac{\partial}{\partial \theta}\mathbb{E}_{s\sim p^*_\beta}\left[C\right] d\beta$$

Massar and Mognetti [30] derived the formula

$$\frac{\partial}{\partial \theta}\mathbb{E}_{s\sim p^*_0}\left[C\right] = \frac{d}{d\beta}\bigg|_{\beta=0} \mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] = -\frac{2}{\sigma^2}\,\mathrm{Cov}_{s\sim p^*_0}\left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y)\right)$$

for Hopfield energy specifically. Below, we derive it for arbitrary energy functions, where the state distribution follows the Gibbs Boltzmann distribution.

3.2.2 Proof of the gradient formula with Gibbs Boltzman distribution

To keep the notation clean, we will write $\frac{\partial E}{\partial \theta}(\theta, x, s)$ simply as $f(s)$ and $C(s, y)$ as $C(s)$:

$$\mathbb{E}_{s\sim p^*_\beta}\left[f(s)\right] = \int f(s) p^*_\beta(s)\, ds$$

Where $p^*_\beta(s)$ is the Gibbs-Boltzmann invariant density function:

$$p^*_\beta(s) = \frac{1}{Z(\beta)} \exp\left(-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big)\right)$$

and the partition function $Z(\beta)$ is the normalizing constant:

$$Z(\beta) = \int \exp\left(-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big)\right) ds$$

Assuming the regularity conditions allow us to pass the derivative inside the integral (Leibniz integral rule), we differentiate with respect to β:

$$\frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}\left[f(s)\right] = \int f(s)\frac{\partial p^*_\beta(s)}{\partial \beta}\, ds$$

To find $\frac{\partial p^*_\beta(s)}{\partial \beta}$, we use the identical log-derivative property $\frac{\partial p}{\partial \beta} = p\frac{\partial \ln p}{\partial \beta}$:

$$\frac{\partial \ln p^*_\beta(s)}{\partial \beta} = \frac{\partial}{\partial \beta}\left[-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big) - \ln Z(\beta)\right]$$

$$\frac{\partial \ln p^*_\beta(s)}{\partial \beta} = -\frac{2}{\sigma^2}C(s) - \frac{1}{Z(\beta)}\frac{dZ(\beta)}{d\beta}$$

Now we need to evaluate the term involving the derivative of the partition function. Let's differentiate $Z(\beta)$ directly:

$$\frac{dZ(\beta)}{d\beta} = \frac{d}{d\beta}\int \exp\left(-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big)\right) ds$$

$$\frac{dZ(\beta)}{d\beta} = \int \left(-\frac{2}{\sigma^2}C(s)\right)\exp\left(-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big)\right) ds$$

If we pull the constant out and divide by $Z(\beta)$ on both sides, we notice that the remaining integral reconstructs the expectation of $C(s)$ under p^*_β:

$$\frac{1}{Z(\beta)}\frac{dZ(\beta)}{d\beta} = -\frac{2}{\sigma^2}\int C(s)\left[\frac{1}{Z(\beta)}\exp\left(-\frac{2}{\sigma^2}\big(E(s) + \beta C(s)\big)\right)\right] ds$$

$$\frac{1}{Z(\beta)}\frac{dZ(\beta)}{d\beta} = -\frac{2}{\sigma^2}\mathbb{E}_{s\sim p^*_\beta}\left[C(s)\right]$$

Substituting this result back into our expression for the derivative of the log-density:

$$\frac{\partial \ln p^*_\beta(s)}{\partial \beta} = -\frac{2}{\sigma^2}C(s) - \left(-\frac{2}{\sigma^2}\mathbb{E}_{s\sim p^*_\beta}\left[C(s)\right]\right)$$

$$\frac{\partial \ln p^*_\beta(s)}{\partial \beta} = -\frac{2}{\sigma^2}\Big(C(s) - \mathbb{E}_{s\sim p^*_\beta}\left[C(s)\right]\Big)$$

Since $\frac{\partial p^*_\beta}{\partial \beta} = p^*_\beta \frac{\partial \ln p^*_\beta}{\partial \beta}$, the absolute structural change of the probability density is:

$$\frac{\partial p^*_\beta(s)}{\partial \beta} = -\frac{2}{\sigma^2} p^*_\beta(s) \Big(C(s) - \mathbb{E}_{s\sim p^*_\beta}[C(s)] \Big)$$

Now, we substitute this back into our original integral :

$$\frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}[f(s)] = \int f(s) \left[-\frac{2}{\sigma^2} p^*_\beta(s) \Big(C(s) - \mathbb{E}_{s\sim p^*_\beta}[C(s)] \Big) \right] ds$$

Distribute the integral over the subtraction terms:

$$= -\frac{2}{\sigma^2} \left(\int f(s) C(s) p^*_\beta(s)\, ds - \mathbb{E}_{s\sim p^*_\beta}[C(s)] \int f(s) p^*_\beta(s)\, ds \right)$$

Convert these integrals back into expectations:

$$= -\frac{2}{\sigma^2} \Big(\mathbb{E}_{s\sim p^*_\beta}[f(s)C(s)] - \mathbb{E}_{s\sim p^*_\beta}[f(s)]\, \mathbb{E}_{s\sim p^*_\beta}[C(s)] \Big)$$

By definition, for any two random variables A and B, their statistical covariance is $\mathrm{Cov}(A, B) = \mathbb{E}[AB] - \mathbb{E}[A]\mathbb{E}[B]$. Thus:

$$\frac{d}{d\beta}\mathbb{E}_{s\sim p^*_\beta}[f(s)] = -\frac{2}{\sigma^2} \mathrm{Cov}_{s\sim p^*_\beta}\big(f(s),\, C(s)\big)$$

Finally, evaluating this exact continuous identity at the unnudged free phase ($\beta = 0$) gives the final expression:

$$\left.\frac{d}{d\beta}\right|_{\beta=0} \mathbb{E}_{s\sim p^*_\beta}\left[\frac{\partial E}{\partial \theta}\right] = -\frac{2}{\sigma^2} \mathrm{Cov}_{s\sim p^*_0}\left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y)\right)$$

3.3 Fisher EP for Gibbs Boltzmann distributions

We propose to replace the KL divergence based objective of Litman's EP rule [15]

$$J(\theta) = \mathbb{E}_{\rho_\beta(s,\theta)}(C(s)) + TKL(\rho_0(s,\theta)|\rho_\beta(s,\theta))$$

with a Fisher divergence based objective :

$$J(\theta) = \mathbb{E}_{\rho_\beta(s,\theta)}(C(s)) + T^2 FD(\rho_0(s,\theta)|\rho_\beta(s,\theta))$$

The Fisher Divergence (FD) between the perturbed density ρ_β and the unperturbed density ρ_0 is defined as:

$$FD(\rho_\beta \parallel \rho_0) = \frac{1}{2} \int_{\mathcal{S}} \rho_\beta(s) \left\| \nabla_s \log \rho_\beta(s) - \nabla_s \log \rho_0(s) \right\|^2 ds$$

Given the log-density:

$$\log \rho_\beta(s) = -\frac{1}{T} \Big(E(\theta, s) + \beta C(s, y) \Big) - \log \mathcal{Z}^\theta_\beta$$

Since $\mathcal{Z}_\beta^\theta$ is independent of s, we have $\nabla_s \log \mathcal{Z}_\beta^\theta = 0$. Thus:

$$\nabla_s \log \rho_\beta(s) = -\frac{1}{T}\nabla_s E(\theta, s) - \beta\frac{1}{T}\nabla_s C(s, y)$$

We also have:

$$\nabla_s \log \rho_\beta(s) - \nabla_s \log \rho_0(s) = -\frac{1}{T}\nabla_s E - \frac{1}{T}\beta\nabla_s C + \frac{1}{T}\nabla_s E = -\frac{1}{T}\beta\nabla_s C(s, y)$$

Fisher Divergence Expression

Substituting this score difference back into the integral:

$$\begin{aligned} FD(\rho_\beta \parallel \rho_0) &= \frac{1}{2}\int_{\mathcal{S}} \rho_\beta(s) \left\| \beta\frac{1}{T}\nabla_s C(s, y) \right\|^2 ds \\ &= \frac{1}{2}\mathbb{E}_{\rho_\beta(s)} \left[\left\| \beta\frac{1}{T}\nabla_s C(s, y) \right\|^2 \right] \\ &= \frac{\beta^2}{2T^2}\mathbb{E}_{\rho_\beta(s)} \left[\|\nabla_s C(s, y)\|^2 \right] \end{aligned}$$

Parameter Gradient of Expected Cost Norm

To compute the gradient with respect to θ:

$$\mathbb{E}_{\rho_\beta(s)} \left[\|\nabla_s C(s, y)\|^2 \right] = \int_{\mathcal{S}} \|\nabla_s C(s)\|^2 \frac{1}{\mathcal{Z}_\beta^\theta} e^{-(E+\beta C)}\, ds$$

Differentiating with respect to θ:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{\rho_\beta} \left[\|\nabla_s C\|^2 \right] &= -\int_{\mathcal{S}} \|\nabla_s C(s)\|^2 \frac{\nabla_\theta \mathcal{Z}_\beta^\theta}{(\mathcal{Z}_\beta^\theta)^2} e^{-(E+\beta C)}\, ds \\ &= -\int_{\mathcal{S}} \|\nabla_s C(s)\|^2 \frac{\nabla_\theta E}{\mathcal{Z}_\beta^\theta} e^{-(E+\beta C)}\, ds \end{aligned}$$

Using the identity:

$$\frac{\nabla_\theta \mathcal{Z}_\beta^\theta}{\mathcal{Z}_\beta^\theta} = \nabla_\theta \log \mathcal{Z}_\beta^\theta = -\mathbb{E}_{\rho_\beta} \left[\nabla_\theta E(\theta, s) \right]$$

We get:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{\rho_\beta} \left[\|\nabla_s C\|^2 \right] &= \mathbb{E}_{\rho_\beta} \left[\nabla_\theta E(\theta, s) \right] \mathbb{E}_{\rho_\beta} \left[\|\nabla_s C(s)\|^2 \right] - \mathbb{E}_{\rho_\beta} \left[\|\nabla_s C\|^2 \nabla_\theta E \right] \\ &= -\operatorname{Cov}_{\rho_\beta} \left(\nabla_\theta E(\theta, s), \|\nabla_s C\|^2 \right) \end{aligned}$$

Thus

$$\nabla_\theta T^2 FD(\rho_\beta \parallel \rho_0) = -\frac{\beta^2}{2} \operatorname{Cov}_{\rho_\beta} \left(\nabla_\theta E(\theta, s), \|\nabla_s C\|^2 \right)$$

We also have from above that :

$$\nabla_\theta \mathbb{E}_{\rho_\beta(s,\theta)} = -\frac{1}{T} \operatorname{Cov}_{s\sim\rho_\beta} \left(\frac{\partial E}{\partial \theta}(\theta, x, s),\, C(s, y) \right)$$

Thus for

$$\boxed{J(\theta) = \mathbb{E}_{\rho_\beta(s,\theta)}(C(s)) + T^2 FD(\rho_0(s,\theta)|\rho_\beta(s,\theta))}$$

we have

$$\boxed{\nabla_\theta J(\theta) = -\frac{1}{T}\,\mathrm{Cov}_{s\sim\rho_\beta}\left(\frac{\partial E}{\partial\theta}(\theta,x,s),\, C(s,y)\right) - \frac{\beta^2}{2}\,\mathrm{Cov}_{\rho_\beta}\left(\nabla_\theta E(\theta,s), \|\nabla_s C\|^2\right)}$$

The gradient formula, include covariances, only in the nudge distribution ρ_β, which makes this method single phase, unlike KL based methods.

Note about the squared temperature T^2

To ensure a non divergent objective for the systems with low temperature, we need to include temperature in squared scale for the KL divergence, indeed :

The Fisher divergence diverges as $T \to 0$ like the KL divergence, but not with the same temperature scaling as KL. It is more singular.

For the Gibbs measures

$$\rho_0 \propto e^{-E/T}, \qquad \rho_\beta \propto e^{-(E+\beta C)/T},$$

the score functions with respect to s are

$$\nabla_s \log \rho_0 = -\frac{1}{T}\nabla_s E(s),$$

and

$$\nabla_s \log \rho_\beta = -\frac{1}{T}\nabla_s\big(E(s) + \beta C(s)\big).$$

Therefore the score difference is

$$\nabla_s \log \rho_0 - \nabla_s \log \rho_\beta = \frac{\beta}{T}\nabla_s C(s).$$

Hence the Fisher divergence scales as

$$D_F(\rho_0\|\rho_\beta) = \mathbb{E}_{\rho_0}\,\|\nabla_s \log \rho_0 - \nabla_s \log \rho_\beta\|^2 = \frac{\beta^2}{T^2}\mathbb{E}_{\rho_0}\|\nabla_s C(s)\|^2.$$

Thus, in the deterministic limit,

$$\boxed{\lim_{T\to 0} T^2 D_F(\rho_0\|\rho_\beta) = \beta^2\|\nabla_s C(s_0^*)\|^2}$$

where

$$s_0^* = \arg\min_s E(\theta, s).$$

Like the KL divergence, the Fisher divergence must be rescaled to have a finite deterministic limit, but the required scaling is T^2, not T. Unscaled, it diverges as $1/T^2$, faster than KL, which diverges as $1/T$.

3.4 Tsallis EP: Generalizing Litman's approach

We extend Litman's framework [15], from Gibbs Boltzmann distributions to Tsallis distributions. Tsallis Statistics (Power Laws) are one of the most famous generalizations in physics based on Tsallis non-extensive statistics. If a system exhibits long-range correlations or fractal structures, Shannon entropy is no longer suitable. Tsallis distribution represents a generalization of Gibbs Boltzmann distribution [37].

By maximizing Tsallis entropy, the state distribution becomes a q-exponential:

$$p_\theta(s) = \frac{1}{Z_q(\theta)} \left[1 - (1-q)\alpha E_\theta(s)\right]_+^{\frac{1}{1-q}}$$

Where $[a]_+ = \max(a, 0)$ and q is a deformation parameter.

If $q \to 1$: We recover exactly the classic Boltzmann law ($e^{-E(s)}$).

If $q > 1$: The distribution develops heavy tails (power laws). High-energy states remain much more probable than in a classic EBMs. This makes the model much more robust to outliers.

In this formula, α represents the inverse temperature of the system. This concept is inherited directly from statistical physics, where it is defined by the relation:

$$\alpha = \frac{1}{k_B T}$$

(where T is the absolute temperature of the system and k_B is the Boltzmann constant). In Machine Learning, we generally simplify this by setting $k_B = 1$, which gives simply $\alpha = \frac{1}{T}$.

3.4.1 Extension to Tsallis distributions for EP

We consider Litman's objective [15] as the following definition

$$J(\theta) := \mathbb{E}_{\rho_\beta}[C(s)] + T\,\mathrm{KL}(\rho_\beta \parallel \rho_0)$$

and derive the gradient formula for a Tsallis distribution.

$$\rho_{\beta,q}(s,\theta) = \frac{1}{\mathcal{Z}_{\beta,q}} \psi_{\beta,q,0}^{\frac{1}{1-q}}(s) \quad \text{where} \quad \psi(s) = \left(1 - (1-q)\frac{\beta}{T} F_\theta(s)\right)_+$$

1. Density Gradient Derivation

The parameter gradient of the distribution density is given by:

$$\nabla_\theta \rho_{\beta,q}(s,\theta;q) = -\frac{\nabla_\theta \mathcal{Z}}{\mathcal{Z}^2} \psi^{\frac{1}{1-q}} + \frac{1}{\mathcal{Z}} \nabla_\theta \psi^{\frac{1}{1-q}}$$

Using the chain rule:

$$\nabla_\theta \psi^{\frac{1}{1-q}} = \frac{1}{1-q} \nabla_\theta \psi \, \psi^{\frac{1}{1-q}-1}$$

$$\nabla_\theta \psi = \nabla_\theta \left(1 - (1-q)\frac{\beta}{T} F\right)_+ = -\frac{(q-1)}{T} \beta \nabla_\theta E \quad \text{if } \psi > 0$$

For the partition function $\mathcal{Z}$:

$$\nabla_\theta \mathcal{Z} = \nabla_\theta \int_{\mathcal{S}} \psi^{\frac{1}{1-q}} \, ds = \int_{\mathcal{S}} \nabla_\theta \psi^{\frac{1}{1-q}} \, ds \implies \frac{\nabla_\theta \mathcal{Z}}{\mathcal{Z}} = \frac{1}{1-q} \mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right)$$

Substituting back into $\nabla_\theta \rho_\beta$:

$$\begin{aligned}
\nabla_\theta \rho_\beta &= -\frac{1}{\mathcal{Z}} \psi^{\frac{1}{1-q}} \frac{1}{1-q} \mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right) + \frac{1}{\mathcal{Z}} \frac{1}{1-q} \nabla_\theta \psi \, \psi^{\frac{1}{1-q}-1} \\
&= \frac{1}{q-1} \frac{1}{\mathcal{Z}} \psi^{\frac{1}{1-q}} \left(\nabla_\theta \psi \, \psi^{-1} - \mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right)\right) \\
&= \frac{1}{q-1} \rho_\beta \left(\nabla_\theta \psi \, \psi^{-1} - \mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right)\right)
\end{aligned}$$

2. Objective Gradient Evaluation

Considering the general expression of the objective's gradient:

$$\nabla_\theta J(\theta) = \int_{\mathcal{S}} C(s) \nabla_\theta \rho_\beta \, ds + T \int_{\mathcal{S}} \left[\nabla_\theta \rho_\beta \log\left(\frac{\rho_\beta}{\rho_0}\right) + \nabla_\theta \rho_\beta - \frac{\rho_\beta}{\rho_0} \nabla_\theta \rho_0\right] ds$$

Similarly to the Gibbs-Boltzmann case:

$$\int_{\mathcal{S}} \nabla_\theta \rho_\beta \, ds = \frac{1}{q-1} \left[\mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right) - \mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi \, \psi^{-1}\right)\right] = 0$$

and evaluating the log-density ratio term:

$$\begin{aligned}
-T \int_{\mathcal{S}} \frac{\rho_\beta}{\rho_0} \nabla_\theta \rho_0 \, ds &= -T \int_{\mathcal{S}} \frac{\rho_\beta}{\rho_0} \frac{1}{q-1} \rho_0 \left(\nabla_\theta \psi_0 \, \psi_0^{-1} - \mathbb{E}_{\rho_0} \left(\nabla_\theta \psi_0 \, \psi_0^{-1}\right)\right) ds \\
&= \frac{T}{1-q} \left(\mathbb{E}_{\rho_\beta} \left(\nabla_\theta \psi_0 \, \psi_0^{-1}\right) - \mathbb{E}_{\rho_0} \left(\nabla_\theta \psi_0 \, \psi_0^{-1}\right)\right)
\end{aligned}$$

Covariance Formulations and Simplified Expressions

$$\int_{\mathcal{S}} C(s) \nabla_\theta \rho_\beta \, ds = \frac{1}{q-1} \operatorname{Cov}_{\rho_\beta} \left(C(s), \, \nabla_\theta \psi_\beta \psi_\beta^{-1}\right)$$

and

$$T \int_{\mathcal{S}} \nabla_\theta \rho_\beta \log\left(\frac{\rho_\beta}{\rho_0}\right) ds = \frac{T}{q-1} \operatorname{Cov}_{\rho_\beta} \left(\log\left(\frac{\rho_\beta}{\rho_0}\right), \, \nabla_\theta \psi_\beta \psi_\beta^{-1}\right)$$

or since:

$$\log\left(\frac{\rho_\beta}{\rho_0}\right) = \log\left(\frac{\mathcal{Z}_0}{\mathcal{Z}_\beta}\right) + \frac{1}{1-q} \log\left(\frac{\psi_\beta}{\psi_0}\right)$$

and $\log\left(\frac{\mathcal{Z}_0}{\mathcal{Z}_\beta}\right)$ is a deterministic variable, we have $\operatorname{Cov}_{\rho_\beta}\left(\log\left(\frac{\mathcal{Z}_0}{\mathcal{Z}_\beta}\right), X\right) = 0$.
This yields the final Objective Gradient Result

$$\nabla_\theta J(\theta) = \frac{1}{q-1}\,\mathrm{Cov}_{\rho_\beta}\left(C(s),\,\nabla_\theta\psi_\beta\psi_\beta^{-1}\right) + \frac{T}{1-q}\,\mathrm{Cov}_{\rho_\beta}\left(\log\left(\frac{\psi_\beta}{\psi_0}\right),\,\nabla_\theta\psi_\beta\psi_\beta^{-1}\right) + \frac{T}{1-q}\left(\mathbb{E}_{\rho_\beta}\left[\nabla_\theta\psi_0\psi_0^{-1}\right] - \mathbb{E}_{\rho_0}\left[\nabla_\theta\psi_0\psi_0^{-1}\right]\right)$$

Using the gradient identities:

$$\nabla_\theta\psi_\beta = \frac{q-1}{T}\nabla_\theta E \quad \text{if } \psi_\beta > 0$$

$$\nabla_\theta\psi_0 = \frac{q-1}{T}\nabla_\theta E \quad \text{if } \psi_0 > 0$$

Thus:

$$\nabla_\theta J(\theta) = \frac{1}{T}\,\mathrm{Cov}_{\rho_\beta}\left(C(s),\,\nabla_\theta E\left(1-\frac{(1-q)}{T}F\right)^{-1}\mathbb{I}_{\psi_\beta>0}\right) + (1-q)\,\mathrm{Cov}_{\rho_\beta}\left(\log\frac{\psi_\beta}{\psi_0},\,\nabla_\theta E\,\psi_\beta^{-1}\mathbb{I}_{\psi_\beta>0}\right) + \mathbb{E}_{\rho_0}\left[\nabla_\theta E\,\psi_0^{-1}\mathbb{I}_{\psi_0>0}\right] - \mathbb{E}_{\rho_\beta}\left[\nabla_\theta E\,\psi_0^{-1}\mathbb{I}_{\psi_0>0}\right]$$

3.4.2 Advantage of Tsallis distributions

In the Langevin equation, if the noise is multiplicative (i.e., if σ depends on the state s itself) or if the noise amplitude undergoes fluctuations (a framework known in physics as superstatistics). Tsallis distributions (q-exponentials) emerge naturally as the invariant measures of SDEs where the noise scales linearly with the state variable. For example, if we consider a simplified one-dimensional version where the noise is state-dependent:

$$dS_t = -\frac{\partial E}{\partial s}dt + \sqrt{\sigma_0^2 + \gamma E(s)}\,dB_t$$

If we solve the Fokker-Planck equation for this multiplicative system, the stationary distribution is no longer an exponential. Instead, it becomes a Tsallis q-exponential.

For example, we consider the following generalized SDE (comapred to the dynamics assocated to the Gibbs Boltzmann dsitribution) :

Let

$$D(s) = \sigma_0^2 + \gamma E(s).$$

For the Itô SDE

$$dS_t = -E'(S_t)dt + \sqrt{D(S_t)}\,dB_t,$$

the stationary Fokker–Planck equation gives

$$E'(s)p(s) + \frac{1}{2}\frac{d}{ds}\big(D(s)p(s)\big) = 0.$$

Solving,

$$p^*(s) \propto \frac{1}{D(s)} \exp\left(-2 \int \frac{E'(s)}{D(s)}\, ds\right).$$

Since

$$\int \frac{E'(s)}{\sigma_0^2 + \gamma E(s)}\, ds = \frac{1}{\gamma} \ln(\sigma_0^2 + \gamma E(s)),$$

one gets

$$p^*(s) \propto \left(\sigma_0^2 + \gamma E(s)\right)^{-1-2/\gamma}.$$

So the stationary distribution is

$$p^*(s) \propto \left(1 + \frac{\gamma}{\sigma_0^2} E(s)\right)^{-1-2/\gamma},$$

It can be written as a Tsallis q-exponential, but with $1 - q = -\frac{\gamma}{\gamma+2}$

3.4.3 Extending Massar and Mognetti [30] with Tsallis distributions

As derived above, for the Tsallis distribution :

$$\rho_{\beta,q}(s,\theta) = \frac{1}{\mathcal{Z}_{\beta,q}} \psi_{\beta,q,0}^{\frac{1}{1-q}}(s) \quad \text{where} \quad \psi(s) = \left(1 - (1-q)\frac{\beta}{T} F_\theta(s)\right)_+$$

we have :

$$\int_{\mathcal{S}} C(s) \nabla_\theta \rho_\beta\, ds = \frac{1}{q-1} \operatorname{Cov}_{\rho_\beta}\left(C(s),\, \nabla_\theta \psi_\beta \psi_\beta^{-1}\right)$$

Thus

$$\boxed{\nabla_\theta \mathbb{E}_{\rho_0}[C(s)] = \frac{1}{q-1} \operatorname{Cov}_{\rho_0}\left(C(s),\, \nabla_\theta \psi_0 \psi_0^{-1}\right)}$$

Chapter 4

Stochastic LEP and HEP for Dynamic Data

In this chapter, we start first by showing, that the Schwarz formula, which is the central tool used for EP, can be considered for the commutation of partial derivatives (∂_θ and ∂_β) instead of total derivatives (d_θ and d_β) in the stochastic EP framework [14].

Then we show an alternative approach for deriving the EP learning rule in the stochastic case, when considering states that follow the Gibbs Boltzmann distribution. The alternative consists in considering the expected energy as the objective to which we apply the Schwarz formula, instead of the Helmholtz energy which was originally considered by Scellier [14].

Building on theses remarks and results, we propose a framework, to enable EP to handle temporal (dynamical) noisy data

4.1 About the Schwarz formula

4.1.1 Deterministic EP and LEP

Schwarz formula can be written for commutation of either total derivatives or partial derivatives, namely:

$$d_\theta d_\beta = d_\beta d_\theta$$

and

$$\partial_\theta \partial_\beta = \partial_\beta \partial_\theta \quad \text{or more rigorously} \quad \nabla_\theta \partial_\beta = \partial_\beta \nabla_\theta$$

because β is a scalar and θ is a vector.

For Scellier's approach [14], only the commutation of total derivatives is relevant in the deterministic framework and yields the expression involving the gradient of the cost function; The application of Schwarz formula for the commutation of partial derivatives is useless, indeed in the deterministic static EP framework, we have:

$$E_\beta(\theta, \beta, s) = E(\theta, s) + \beta C(s, y)$$

$$\frac{\partial}{\partial \theta} E_\beta(\theta, \beta, s) = \frac{\partial}{\partial \theta} E(\theta, s) \implies \frac{\partial}{\partial \beta}\frac{\partial}{\partial \theta} E_\beta(\theta, \beta, s) = \frac{\partial}{\partial \beta}\frac{\partial}{\partial \theta} E(\theta, s) = 0$$

and

$$\frac{\partial}{\partial\beta}E_\beta(\theta,\beta,s) = C(s,y) \implies \frac{\partial}{\partial\theta}\frac{\partial}{\partial\beta}E_\beta(\theta,\beta,s) = \frac{\partial C(s,y)}{\partial\theta} = 0$$

Thus:

$$\frac{\partial}{\partial\theta}\frac{\partial}{\partial\beta}E_\beta(\theta,\beta,s) = \frac{\partial}{\partial\beta}\frac{\partial}{\partial\theta}E_\beta(\theta,\beta,s) \implies 0 = 0$$

On the other hand,

$$\frac{d}{d\theta}\frac{d}{d\beta}E_\beta(\theta,\beta,s) = \frac{d}{d\beta}\frac{d}{d\theta}E_\beta(\theta,\beta,s)$$

yields:

$$\frac{d}{d\theta}C(s,y) = \frac{d}{d\beta}\frac{\partial}{\partial\theta}E(\theta,\beta,s)\Big|_{\beta=0}$$

Similarly, for the deterministic dynamical LEP framework, it is mandatory to consider Schwarz's commutation formula for total derivatives, which in this case yields costly residual terms for the CIVP formulation. The CBPVP formulation enables the suppression of these residual terms, but this approach is incompatible with streaming and thus doesn't offer computational complexity of the level of PFVP / RHEL frameworks (which also have a restrictive application due to the requirement of time reversal symmetry).

4.1.2 Stochastic EP and analogy for LEP

Similarly to the deterministic case, the static stochastic EP framework considered in Scellier's Approach is based on the Helmholtz free energy of the system $\ln \mathcal{Z}_\theta^\beta$ to which the Schwarz formula for the commutation of total derivatives was originally considered, which yields the gradient formula for the expected loss [14]

$$\mathcal{L}_{\text{sto}}(\theta,x,y) = \mathbb{E}_{s\sim\rho_\theta^0(s)}\big[C(s,y)\big]$$

$$\frac{\partial\mathcal{L}_{\text{sto}}}{\partial\theta}(\theta,x,y) = \frac{d}{d\beta}\Big|_{\beta=0}\mathbb{E}_{s\sim\rho_\theta^\beta(s)}\left[\frac{\partial E}{\partial\theta}(\theta,x,s)\right]$$

Scellier's relation using total derivatives:

$$\frac{d}{d\theta}\frac{d}{d\beta}\ln\mathcal{Z}_\theta^\beta = \frac{d}{d\beta}\frac{d}{d\theta}\ln\mathcal{Z}_\theta^\beta$$

Question: Why not use partial derivatives instead?

$$\frac{\partial}{\partial\theta}\frac{\partial}{\partial\beta}\ln\mathcal{Z}_\theta^\beta = \frac{\partial}{\partial\beta}\frac{\partial}{\partial\theta}\ln\mathcal{Z}_\theta^\beta$$

(Note: β is a scalar and θ is a vector, so $\frac{\partial}{\partial\beta}\nabla_\theta = \nabla_\theta\frac{\partial}{\partial\beta}$)

The Schwarz commutation formula of total derivatives is the same as the commutation of partial derivatives in the stochastic case for the Helmoltz energy

$$\frac{d}{d\theta}\frac{d}{d\beta}\ln\mathcal{Z}_\theta^\beta = \frac{\partial}{\partial\theta}\frac{\partial}{\partial\beta}\ln\mathcal{Z}_\theta^\beta = \frac{d}{d\beta}\frac{d}{d\theta}\ln\mathcal{Z}_\theta^\beta = \frac{\partial}{\partial\beta}\frac{\partial}{\partial\theta}\ln\mathcal{Z}_\theta^\beta$$

Indeed the quantity $\ln\mathcal{Z}_\theta^\beta = \ln\int_{\mathcal{S}} e^{-F(\theta,\beta,s)}\,ds$ is independent of the state s :

$$\partial_s \ln \mathcal{Z}_\theta^\beta = \frac{1}{\ln \mathcal{Z}_\theta^\beta} \partial_s \mathcal{Z}_\theta^\beta = \frac{1}{\ln \mathcal{Z}_\theta^\beta} \partial_s \int_{\mathcal{S}} e^{-F(\theta,\beta,s)}\, ds = 0$$

because $\partial_s \int_{\mathcal{S}} e^{-F(\theta,\beta,s)}\, ds = 0$: The integral is independent of s because it was integrated over the whole domain of s, it's an definite integral (not an indefinite integral), thus it's derivative w.r.t s is 0

when computing $\frac{d}{d\beta} \ln \mathcal{Z}_\theta^\beta$

we have

$$\frac{d}{d\beta} \ln \mathcal{Z}_\theta^\beta = \frac{1}{\mathcal{Z}_\theta^\beta} \frac{d}{d\beta} \mathcal{Z}_\theta^\beta = \frac{1}{\mathcal{Z}_\theta^\beta} (\frac{\partial}{\partial \beta} \mathcal{Z}_\theta^\beta + \frac{\partial}{\partial s} \mathcal{Z}_\theta^\beta \frac{\partial}{\partial \beta} s) = \frac{1}{\mathcal{Z}_\theta^\beta} \frac{\partial}{\partial \beta} \mathcal{Z}_\theta^\beta = \frac{\partial}{\partial \beta} \ln \mathcal{Z}_\theta^\beta$$

Remark

Here $\partial_s \mathcal{Z}_\theta^\beta = 0$ trivially because $\mathcal{Z}_\theta^\beta$ is independent of s, we don't need assumptions of optimization.

And similarly, we have

$$\frac{d}{d\theta} \ln \mathcal{Z}_\theta^\beta = \frac{\partial}{\partial \theta} \ln \mathcal{Z}_\theta^\beta$$

$$\frac{d}{d\theta} \frac{\partial}{\partial \beta} \ln \mathcal{Z}_\theta^\beta = \frac{\partial}{\partial \theta} \frac{\partial}{\partial \beta} \ln \mathcal{Z}_\theta^\beta$$

$$\frac{d}{d\beta} \frac{\partial}{\partial \theta} \ln \mathcal{Z}_\theta^\beta = \frac{\partial}{\partial \beta} \frac{\partial}{\partial \theta} \ln \mathcal{Z}_\theta^\beta$$

Thus stochastic EP's original gradient formula can be written as :

$$\frac{\partial \mathcal{L}_{\text{sto}}}{\partial \theta}(\theta, x, y) = \frac{d}{d\beta}\bigg|_{\beta=0} \mathbb{E}_{s\sim\rho_\theta^\beta(s)} \left[\frac{\partial E}{\partial \theta}(\theta, x, s)\right] = \frac{\partial}{\partial_\beta}\bigg|_{\beta=0} \mathbb{E}_{s\sim\rho_\theta^\beta(s)} \left[\frac{\partial E}{\partial \theta}(\theta, x, s)\right]$$

The main conclusion of this discussion, is that in the stochastic case, we can apply Schwarz commutation formula for partial derivatives (∂_θ and ∂_β) which offers a significant advantage in the dynamical data framework (LEP), which is that the costly boundary residuals wouldn't appear at all and thus, in some sense the CIVP and CBPVP frameworks merge, which would in principle enable computational efficiency of the same order of LEP PFVP/RHEL, while having the approach applicable for any Lagrangian or Hamiltonian (dropping the requirement of time reversal symmetry), and being compatible with streaming.

4.2 Alternative Derivation of the EP Learning Rule without the Helmholtz free energy

Instead of considering the Helmholtz free energy for the Gibbs-Boltzmann distribution as considered by Scellier [14], we consider the expected energy, as the main object

$$\mathbb{E}_{\rho_\beta(\theta)}(F)$$

Then following the same argument as above, we apply the Schwarz commutation formula for partial derivatives to derive the stochastic EP gradient formula, and recover the results of Scellier [14], thus we consider

$$\nabla_\theta \nabla_\beta \mathbb{E}_{\rho_\beta(\theta)}(F) = \nabla_\beta \nabla_\theta \mathbb{E}_{\rho_\beta(\theta)}(F) \quad \text{with } F = E + \beta C \quad (\text{we have } \nabla^2_{\theta\beta} F = 0) \tag{4.1}$$

Using the identity: (see Chapter 3)

$$\nabla_\beta \mathbb{E}(F) = \mathbb{E}(\partial_\beta F) - \frac{1}{T}\operatorname{Cov}(F, \partial_\beta F) \tag{4.2}$$

and similarly:

$$\nabla_\theta \mathbb{E}(F) = \mathbb{E}(\nabla_\theta F) - \frac{1}{T}\operatorname{Cov}(F, \nabla_\theta F) \tag{4.3}$$

Then:

$$\partial_\beta \mathbb{E}(\nabla_\theta F) = \partial_\beta \mathbb{E}(\nabla_\theta F) + \frac{1}{T}\left(\underset{S_\beta(\theta)}{\partial_\beta} \operatorname{Cov}(F, \nabla_\theta F) - \underset{S_\beta(\theta)}{\nabla_\theta} \operatorname{Cov}(F, \partial_\beta F) \right) \tag{4.4}$$

Thus to show the EP gradient rule :

$$\underset{\rho_\beta(\theta)}{\nabla_\theta}\, \mathbb{E}(C) = \underset{\rho_\beta(\theta)}{\partial_\beta}\, \mathbb{E}(\nabla_\theta E) \tag{4.5}$$

We need to show

$$\boxed{\underset{\rho_\beta(\theta)}{\nabla_\theta} \operatorname{Cov}(F, \partial_\beta F) - \underset{\rho_\beta(\theta)}{\partial_\beta} \operatorname{Cov}(F, \nabla_\theta F) = 0} \tag{4.6}$$

Which is an easy result after development (see Appendix A).

4.3 Approach for dynamical stochastic EP with discrete time

4.3.1 Modified Approach for stochastic damped Hamiltonian IVP EP

We start by considering a framework of stochastic damped Hamiltonian IVP (Initial value problem). We consider an IVP framework to enable streaming.

1. Non-degenerate nudge dynamics with noise in both q and p

We consider the following dynamics, with position q, momenta p, Hamiltonian $H(q, p, \theta)$, cost $C(q, p)$ and nudge parameter β, and damp/dissipation coefficient γ

$$\boxed{\begin{aligned} dq &= \nabla_p H(q,p,\theta)\, dt + \beta \nabla_p C(q,p)\, dt + \sqrt{2\varepsilon_q}\, dW_t^q, \\ dp &= -\nabla_q H(q,p,\theta)\, dt - \beta \nabla_q C(q,p)\, dt - \gamma \nabla_p H(q,p,\theta)\, dt - \beta\gamma \nabla_p C(q,p)\, dt + \sqrt{2\varepsilon_p}\, dW_t^p, \end{aligned}}$$

where

W_t^q, W_t^p are independent standard Wiener processes, $\varepsilon_q, \varepsilon_p > 0$ are noise intensities for position and momentum.

2. Euler–Maruyama discretization

Then, we consider a simple framework, where time is considered to be discrete. Let $x_i = (q_i, p_i)$. The Euler–Maruyama update is

$$q_{i+1} = q_i + \nabla_p H(q_i, p_i, \theta)\,\Delta t + \beta \nabla_p C(q_i, p_i)\,\Delta t + \sqrt{2\varepsilon_q \Delta t}\,\eta_i,$$

$$p_{i+1} = p_i - \nabla_q H(q_i, p_i, \theta)\,\Delta t - \beta \nabla_q C(q_i, p_i)\,\Delta t - \gamma \nabla_p H(q_i, p_i, \theta)\,\Delta t - \beta\gamma \nabla_p C(q_i, p_i)\,\Delta t + \sqrt{2\varepsilon_p \Delta t}\,\xi_i,$$

where

$$\eta_i, \xi_i \sim \mathcal{N}(0, I_d) \quad \text{independent.}$$

Define the deterministic parametrized mean vector

$$\mu_i^\beta(q_i, p_i, \theta) = \begin{pmatrix} q_i + \nabla_p H(q_i, p_i, \theta)\,\Delta t + \beta \nabla_p C(q_i, p_i)\,\Delta t \\ p_i - \nabla_q H(q_i, p_i, \theta)\,\Delta t - \beta \nabla_q C(q_i, p_i)\,\Delta t - \gamma \nabla_p H(q_i, p_i, \theta)\,\Delta t - \beta\gamma \nabla_p C(q_i, p_i)\,\Delta t \end{pmatrix},$$

and the covariance matrix (independent of θ and β)

$$\Sigma = \begin{pmatrix} 2\varepsilon_q \Delta t\, I_d & 0 \\ 0 & 2\varepsilon_p \Delta t\, I_d \end{pmatrix} \in \mathbb{R}^{2d \times 2d}.$$

The transition density is then a multivariate Gaussian:

$$\boxed{p_\theta^\beta(x_{i+1} \mid x_i) = \mathcal{N}\big(x_{i+1};\, \mu_i^\beta(x_i, \theta),\, \Sigma\big)}$$

with

$$\Sigma \succ 0$$

because both $\varepsilon_q, \varepsilon_p > 0$, the path measure is absolutely continuous with respect to Lebesgue measure on the phase-space path.

3. Path likelihood

The joint density of a trajectory $x_{0:N}$ conditioned on x_0 is the product of transition densities:

$$p_\theta^\beta(x_{1:N} \mid x_0) = \prod_{i=0}^{N-1} \mathcal{N}\big(x_{i+1};\, \mu_i^\beta(x_i, \theta),\, \Sigma\big).$$

The log-likelihood is therefore

$$\log p_\theta^\beta(x_{1:N} \mid x_0) = -\frac{1}{2}\sum_{i=0}^{N-1}\big(x_{i+1} - \mu_i^\beta(x_i,\theta)\big)^\top \Sigma^{-1}\big(x_{i+1} - \mu_i^\beta(x_i,\theta)\big) + \text{const},$$

where "const" does not depend on θ nor β (because Σ doesn't depend on them).

4. Score function estimator

For a cost functional $\Phi(x_{0:N})$, the expectation under the path measure is

$$\mathbb{E}_{p_\theta^\beta}[\Phi] = \int \Phi(x_{0:N})\, p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N}.$$

The score function estimator gives

$$\nabla_\theta \mathbb{E}_{p_\theta^\beta}[\Phi] = \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\, \nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)\right].$$

Because the log-likelihood is a sum over transitions,

$$\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0) = \sum_{i=0}^{N-1} \nabla_\theta \log p_\theta^\beta(x_{i+1} \mid x_i).$$

For the Gaussian transition,

$$\log p_\theta^\beta(x_{i+1} \mid x_i) = -\frac{1}{2}(x_{i+1} - \mu_i^\beta)^\top \Sigma^{-1}(x_{i+1} - \mu_i^\beta) + \text{const}.$$

Thus

$$\boxed{\nabla_\theta \log p_\theta^\beta(x_{i+1} \mid x_i) = \big(\nabla_\theta \mu_i^\beta(x_i,\theta)\big)^\top \Sigma^{-1}\big(x_{i+1} - \mu_i^\beta(x_i,\theta)\big).}$$

Remark

Here $\nabla_\theta \mu_i(x_i,\theta)$ is the partial derivative of the mean with respect to θ with the previous state x_i held fixed. We do not differentiate x_i with respect to θ, and we do not backpropagate through the whole trajectory. This is the main advantage of the score function estimator: it avoids differentiating the simulated states.

We also have the same for the β derivative

$$\frac{\partial}{\partial_\beta} \log p_\theta^\beta(x_{i+1} \mid x_i) = \big(\frac{\partial}{\partial_\beta}\mu_i^\beta(x_i,\theta)\big)^\top \Sigma^{-1}\big(x_{i+1} - \mu_i^\beta(x_i,\theta)\big)$$

In this part, we consider the log derivative trick, to generalize the result of Massar and Mognetti [30] in this dynamical noisy data framework. We have

$$\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N} C(q_k,p_k)\right] = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k,p_k)\, \nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)\right]$$

Application. In practice, we would consider a forward phase given by the dynamics :

$$q_{i+1} = q_i + \nabla_p H(q_i, p_i, \theta)\,\Delta t + \sqrt{2\varepsilon_q \Delta t}\,\eta_i,$$

$$p_{i+1} = p_i - \nabla_q H(q_i, p_i, \theta)\,\Delta t - \gamma \nabla_p H(q_i, p_i, \theta)\,\Delta t + \sqrt{2\varepsilon_p \Delta t}\,\xi_i,$$

At each time step, we compute the term :

$$C(q_k, p_k)\,\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)$$

to obtain at the final time, the sum :

$$\sum_{k=0}^{N} C(q_k, p_k)\,\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)$$

This process would be repeated over multiple random trajectories, and we would then consider the average terms to approximate the expected gradient estimator :

$$= \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\,\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)\right]$$

5. Estimator for the expected path cost (EP like rule)

In this part, we consider the log derivative trick, combined with an EP like construction, inspired by the previous remarks for stochastic EP, to promote EP such that it can be used for this dynamical noisy data framework.

Suppose we want

$$\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N} C(q_k, p_k)\right].$$

We start by considering the following total quantity :

$$A_\theta^\beta = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k)\right]$$

Schwarz formula give

$$\frac{\partial}{\partial_\beta} \nabla_\theta A_\theta^\beta = \nabla_\theta \frac{\partial}{\partial_\beta} A_\theta^\beta$$

Again using :

$$\nabla_\theta \sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k) = \sum_{k=0}^{N} \nabla_\theta H(q_k, p_k, \theta)$$

and

$$\frac{\partial}{\partial_\beta} \sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k) = \sum_{k=0}^{N} C(q_k, p_k, \theta)$$

Furthermore, we consider the previous formula of score function estimation for the parameters θ

$$\begin{aligned}\int \Phi(x_{0:N})\,\nabla_\theta p_\theta^\beta(x_{1:N}\mid x_0)\,dx_{1:N} &= \mathbb{E}_{p_\theta^\beta}\Big[\Phi(x_{0:N})\,\nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\Big]\\ &= \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\left(\sum_{i=0}^{N-1}\nabla_\theta \log p_\theta^\beta(x_{i+1}\mid x_i)\right)\right]\\ &= \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\sum_{i=0}^{N-1}\Big(\nabla_\theta \mu_i^\beta(x_i,\theta)\Big)^\top \Sigma^{-1}\Big(x_{i+1}-\mu_i^\beta(x_i,\theta)\Big)\right]\end{aligned} \tag{4.7}$$

As well as the nudge parameter β

$$\begin{aligned}\int \Phi(x_{0:N})\,\frac{\partial}{\partial\beta} p_\theta^\beta(x_{1:N}\mid x_0)\,dx_{1:N} &= \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\,\frac{\partial}{\partial\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)\right]\\ &= \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\left(\sum_{i=0}^{N-1}\frac{\partial}{\partial\beta}\log p_\theta^\beta(x_{i+1}\mid x_i)\right)\right]\\ &= \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\sum_{i=0}^{N-1}\Big(\frac{\partial}{\partial\beta}\mu_i^\beta(x_i,\theta)\Big)^\top \Sigma^{-1}\Big(x_{i+1}-\mu_i^\beta(x_i,\theta)\Big)\right]\end{aligned} \tag{4.8}$$

Thus by developing the Schwarz formula

$$\frac{\partial}{\partial_\beta}\nabla_\theta A_\theta^\beta = \nabla_\theta \frac{\partial}{\partial_\beta} A_\theta^\beta$$

We obtain (see Appendix A for details)

$$\begin{aligned}\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N} C(q_k,p_k)\right] = \frac{\partial}{\partial\beta}\Big|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)\right]\\ + \frac{\partial}{\partial\beta}\Big|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)+\beta C(q_k,p_k)\right)\cdot\nabla_\theta\log p_\theta^\beta(x_{1:N}\mid x_0)\right]\\ - \mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^{N} H(q_k,p_k,\theta)\right)\cdot\frac{\partial}{\partial\beta}\Big|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta\log p_\theta^0(x_{1:N}\mid x_0)\right]\end{aligned} \tag{4.9}$$

Remark

$$\frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{i+1}\mid x_i) = \big(\frac{\partial}{\partial_\beta}\mu_i^\beta(x_i,\theta)\big)^\top\Sigma^{-1}\big(x_{i+1}-\mu_i^\beta(x_i,\theta)\big)$$

Where

$$\frac{\partial}{\partial_\beta}\mu_i^\beta(q_i,p_i) = \begin{pmatrix}\nabla_p C(q_i,p_i)\,\Delta t\\ -\nabla_q C(q_i,p_i)\,\Delta t-\gamma\nabla_p C(q_i,p_i)\,\Delta t\end{pmatrix},$$

because

$$\mu_i^\beta(q_i,p_i) = \begin{pmatrix}q_i+\nabla_p H(q_i,p_i)\,\Delta t+\beta\nabla_p C(q_i,p_i)\,\Delta t\\ p_i-\nabla_q H(q_i,p_i)\,\Delta t-\beta\nabla_q C(q_i,p_i)\,\Delta t-\gamma\nabla_p H(q_i,p_i)\,\Delta t-\beta\gamma\nabla_p C(q_i,p_i)\,\Delta t\end{pmatrix},$$

Or we have

$$\log p_\theta^\beta(x_{1:N}\mid x_0) = \sum_{i=0}^{N-1}\log p_\theta^\beta(x_{i+1}\mid x_i)$$

Thus

$$\begin{aligned}\frac{\partial}{\partial\beta}\log p_\theta^\beta(x_{1:N}\mid x_0) &= \sum_{i=0}^{N-1}\frac{\partial}{\partial\beta}\log p_\theta^\beta(x_{i+1}\mid x_i)\\ &= \sum_{i=0}^{N-1}\left(\frac{\partial}{\partial\beta}\mu_i^\beta(x_i,\theta)\right)^\top\Sigma^{-1}\left(x_{i+1}-\mu_i^\beta(x_i,\theta)\right)\end{aligned}\tag{4.10}$$

Furthermore, we have

$$\nabla_\theta\log p_\theta^\beta(x_{1:N}\mid x_0) = \sum_{i=0}^{N-1}\nabla_\theta\log p_\theta^\beta(x_{i+1}\mid x_i)$$

$$=\sum_{i=0}^{N-1}\nabla_\theta\log p_\theta^\beta(x_{i+1}\mid x_i) = \big(\nabla_\theta\mu_i^\beta(x_i,\theta)\big)^\top\Sigma^{-1}\big(x_{i+1}-\mu_i^\beta(x_i,\theta)\big)$$

And

$$\nabla_\theta\mu_i^\beta(q_i,p_i) = \begin{pmatrix}\nabla_\theta\nabla_p H(q_i,p_i,\theta)\,\Delta t\\ -\nabla_\theta\nabla_q H(q_i,p_i,\theta)\,\Delta t-\gamma\nabla_\theta\nabla_p H(q_i,p_i,\theta)\,\Delta t\end{pmatrix},$$

Remark

Unlike in the stochastic EP framework considered with Gibbs Boltzmann by Scellier [14], the EP gradient formula does not yield a clean result of the from :

$$\nabla_\theta\mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N}C(q_k,p_k)\right] = \partial_\beta|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)\right]$$

Indeed, we showed that there are additional terms :

$$+\partial_\beta|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)+\beta C(q_k,p_k)\right)\cdot\nabla_\theta\log p_\theta^\beta(x_{1:N}\mid x_0)\right]$$

and

$$-\mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^{N}H(q_k,p_k,\theta)\right)\cdot\frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta\log p_\theta^0(x_{1:N}\mid x_0)\right]$$

Nevertheless, these terms are easy to compute, and are irrelevant to the applicability of this EP framework to handle dynamical noisy data (unlike the costly boundary residuals terms of LEP CIVP for example).

Application. In practice, we would consider a forward free phase ($\beta = 0$), and a forward nudge phase ($\beta \neq 0$). Both are given by the dynamics :

$$q_{i+1} = q_i + \nabla_p H(q_i,p_i,\theta)\,\Delta t + \beta\nabla_p C(q_i,p_i)\,\Delta t + \sqrt{2\varepsilon_q\Delta t}\,\eta_i,$$

$$p_{i+1} = p_i - \nabla_q H(q_i,p_i,\theta)\,\Delta t - \beta\nabla_q C(q_i,p_i)\,\Delta t - \gamma\nabla_p H(q_i,p_i,\theta)\,\Delta t - \beta\gamma\nabla_p C(q_i,p_i)\,\Delta t + \sqrt{2\varepsilon_p\Delta t}\,\xi_i,$$

We would then consider multiple free stochastic trajectories that follow the law p_θ^0, and nudge stochastic trajectories that follow the law p_θ^β, to estimate the expected cost gradient considering the average values.

4.4 Limitless HEP

We aim to consider a framework similar to Litman's proposal [15], for dynamical noisy data. Indeed this original framework is suited only for static noisy data, and it enables a limitless EP architecture. We consider the following objective function :

$$J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] + \lambda \,\mathrm{KL}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0))$$

$$= \int \sum_{k=0}^{N} C(q_k, p_k) p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N} + \lambda \int p_\theta^\beta(x_{1:N} \mid x_0) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right) dx_{1:N}$$

where λ is a regularization hyperparameter which depends on the noise/variance.
It is easy to show that (see Appendix A for detailed derivation)

$$\nabla_\theta \,\mathrm{KL}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0)) = \mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right)\right]$$

$$+\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) p_\theta^0(x_{1:N} \mid x_0) - \nabla_\theta p_\theta^0(x_{1:N} \mid x_0) p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)^2}\right]$$

Or again using the log derivative trick, we derive

$$\nabla_\theta \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] = \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^{N} C(q_k, p_k)\right) \nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right)\right]$$

Which gives the final expression of the gradient of the objective function

$$J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] + \lambda \,\mathrm{KL}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0))$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^{N} C(q_k, p_k)\right) \nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right)\right]$$

$$+\mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right)\right]$$

$$+\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) p_\theta^0(x_{1:N} \mid x_0) - \nabla_\theta p_\theta^0(x_{1:N} \mid x_0) p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)^2}\right]$$

This yields a Limitless dynamical stochastic EP framework which generalizes Litman's [15] framework to dynamical temporal data.

4.5 Fisher HEP: Limitless single phase

We extend our proposal of Fisher EP (see Chapter 3), which was based on Litman's limitless EP architecture [15], where we replaced the KL divergence term in the objective function with the Fisher divergence term. This lead to an EP architecture, which was based on a single nudge phase. We consider the following objective function :

$$J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] + \lambda^2 \,\mathrm{FD}\left(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0)\right) \tag{4.11}$$

$$= \int \sum_{k=0}^{N} C(q_k, p_k)\, p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N} + \frac{\lambda^2}{2} \int p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N} \tag{4.12}$$

where λ is a regularization hyperparameter which depends on the noise/variance.

For $J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] + \lambda^2 \,\mathrm{FD}\left(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0)\right)$, we have (see Appendix A for derivation):

$$\nabla_\theta J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2\right] + 2\mathbb{E}_{p_\theta^\beta}\left[\left(\nabla_\theta \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_\theta \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0)\right)^\top \cdot \left(\nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0)\right)\right] \tag{4.13}$$

This offers a framework for a single phase limitless dynamical EP.

Remark 4.5.1. *One of the reasons this single phase approach works in this configuration is that the normalization constant in this framework is independent of the parameters θ and the nudge β because the path density is a Gaussian density :*

The joint density of a trajectory $x_{0:N}$ conditioned on x_0 is the product of transition densities:

$$p_\theta^\beta(x_{1:N} \mid x_0) = \prod_{i=0}^{N-1} \mathcal{N}\big(x_{i+1};\, \mu_i^\beta(x_i, \theta),\, \Sigma\big)$$

The log-likelihood is therefore

$$\log p_\theta^\beta(x_{1:N} \mid x_0) = -\frac{1}{2} \sum_{i=0}^{N-1} \big(x_{i+1} - \mu_i^\beta(x_i, \theta)\big)^\top \Sigma^{-1} \big(x_{i+1} - \mu_i^\beta(x_i, \theta)\big) + \mathit{const},$$

where "const" does not depend on θ nor β (because Σ doesn't depend on them).

4.6 Framework for stochastic first order dynamical systems

EP has been considered only for Lagrangian or Hamiltonian dynamical systems (2nd order dynamical systems) as proposed by Pourcel et al. [4] and Scellier [14], but not all dynamical systems are Lagrangian or Hamiltonian, in fact when considering a general dynamical system, the framework considered is of 1st order dynamical systems.

4.6.1 Approach for discrete SDEs

1. The Freidlin–Wentzell action

For

$$dX_t = b(X_t, \theta)\, dt + \sqrt{\varepsilon}\, \Sigma(X_t)\, dW_t, \qquad D = \Sigma\Sigma^\top,$$

The Freidlin–Wentzell action which represents the large-deviation rate function is

$$I[\phi] = \frac{1}{4} \int_0^T (\dot{\phi}_t - b(\phi_t, \theta))^\top D(\phi_t)^{-1} (\dot{\phi}_t - b(\phi_t, \theta))\, dt$$

We cannot get the stochastic trajectory s_t by solving the deterministic Euler–Lagrange equations of the Freidlin–Wentzell action.

But we can use the Lagrangian to define a discrete-time path measure whose transitions are exactly the SDE's Euler–Maruyama updates. The Brownian noise dW_t then appears as the random fluctuation around the discrete minimizer.

Discrete action

We discretize the time interval $[0, T]$ into N steps of size $\Delta t = T/N$. Let x_i denote the state at time $t_i = i\Delta t : x_i \approx s_{t_i}$.

A natural discrete version of the Freidlin–Wentzell action is

$$S_d(x_{0:N}) = \frac{1}{4\varepsilon} \sum_{i=0}^{N-1} \left(\frac{x_{i+1} - x_i}{\Delta t} - b(x_i, \theta) \right)^\top D(x_i)^{-1} \left(\frac{x_{i+1} - x_i}{\Delta t} - b(x_i, \theta) \right) \Delta t.$$

This can be written as a sum of a discrete Lagrangian L_d:

$$S_d = \sum_{i=0}^{N-1} L_d(x_i, x_{i+1}, \theta) \Delta t,$$

where

$$L_d(x_i, x_{i+1}, \theta) = \frac{1}{4\varepsilon} \left(\frac{x_{i+1} - x_i}{\Delta t} - b(x_i, \theta) \right)^\top D(x_i)^{-1} \left(\frac{x_{i+1} - x_i}{\Delta t} - b(x_i, \theta) \right).$$

The path measure and stochastic transitions

The crucial step is to interpret the discrete action as the exponent of a probability weight on the set of discrete paths $\{x_0, x_1, \ldots, x_N\}$:

$$\mathbb{P}(x_{0:N}) \;\propto\; \exp\left(-\frac{1}{\varepsilon} S_d(x_{0:N}, \theta)\right).$$

This is the finite-dimensional analogue of the Freidlin–Wentzell path density. Because S_d is a sum over steps, the joint distribution factorizes into a Markov chain:

$$\mathbb{P}_\theta(x_{0:N}) = p(x_0) \prod_{i=0}^{N-1} p_\theta(x_{i+1} \mid x_i),$$

where the transition probability is

$$p_\theta(x_{i+1} \mid x_i) \;\propto\; \exp\left(-\frac{1}{\varepsilon} L_d(x_i, x_{i+1}, \theta)\Delta t\right).$$

Inserting the explicit form of L_d,

$$p_\theta(x_{i+1} \mid x_i) \;\propto\; \exp\left(-\frac{1}{4\varepsilon\Delta t}(x_{i+1} - x_i - b(x_i,\theta)\Delta t)^\top D(x_i)^{-1}(x_{i+1} - x_i - b(x_i,\theta)\Delta t)\right).$$

This is a Gaussian density with

$$\mathbb{E}[x_{i+1} \mid x_i] = x_i + b(x_i,\theta)\Delta t, \qquad \mathrm{Cov}[x_{i+1} \mid x_i] = 2\varepsilon D(x_i)\Delta t.$$

Therefore, sampling a trajectory from this path measure is equivalent to the recurrence

$$x_{i+1} = x_i + b(x_i,\theta)\Delta t + \sqrt{2\varepsilon}\,\Sigma(x_i)\sqrt{\Delta t}\,\xi_i, \qquad \xi_i \sim \mathcal{N}(0, I),$$

where $\Sigma(x_i)\Sigma(x_i)^\top = D(x_i)$
which is exactly the Euler–Maruyama discretization of the SDE

$$ds_t = b(s_t,\theta)\,dt + \sqrt{2\varepsilon}\,\Sigma(s_t)\,dW_t.$$

So dW_t appears naturally as the random fluctuation with covariance $2\varepsilon D\Delta t$, coming from the quadratic form in the discrete action.

4.6.2 Stochastic EP for 1st order dynamical noisy systems

Following the same logic developed for the stochastic damped Hamiltonian IVP EP,

where the discrete time path density was : a gaussian with mean :

$$\mu_i^\beta(q_i, p_i, \theta) = \begin{pmatrix} q_i + \nabla_p H(q_i, p_i, \theta)\,\Delta t + \beta \nabla_p C(q_i, p_i)\,\Delta t \\ p_i - \nabla_q H(q_i, p_i, \theta)\,\Delta t - \beta \nabla_q C(q_i, p_i)\,\Delta t - \gamma \nabla_p H(q_i, p_i, \theta)\,\Delta t - \beta\gamma \nabla_p C(q_i, p_i)\,\Delta t \end{pmatrix},$$

and the covariance matrix (independent of θ and β)

$$\Sigma = \begin{pmatrix} 2\varepsilon_q \Delta t \, I_d & 0 \\ 0 & 2\varepsilon_p \Delta t \, I_d \end{pmatrix} \in \mathbb{R}^{2d \times 2d}.$$

we can follow the same steps, to derive the analog of the previous derivations : stochastic EP for noisy dynamical Hamitlonian systems, the framework ofMassar and Mognetti [30], the framework of Litman [15], and Fisher EP. Indeed we have also a Gaussian multivariate path density with mean : $b(x, \theta)$ and covariance matrix $D(x)$ (independent of θ and β). Furthermore, to get a more trivial application of EP's framework, we consider that the deterministic part of the dynamics is governed by a gradient

$$b(x, \theta) = \nabla_x E(x, \theta)$$

, thus the nudge dynamics would correspond to

$$b(x, \theta, \beta) = \nabla_x E(x, \theta) + \beta \nabla_x E(x, \theta)$$

where we augmented the scalar potential : $E \to E + \beta C$

4.6.3 Dynamical stochastic EP with continuous time

We now aim to derive an EP framework for dynamical noisy data in continuous time. We start by considering an architecture for 1st order SDEs. This is a novel contribution, because all proposed EP for dynamical data were suited only for 2nd order dynamical data in the deterministic framework ([4],[20],[14]). We then consider an EP architecture for dynamical noisy data for 2nd order SDEs.

Girsanov Theorem and Score Function Gradient for Parameterized SDEs

We consider the following 1st order SDE (in Ito form)

$$ds_t = f(s_t, t)\, dt + \sigma \, dB_t$$

W denotes the law of: $ds_t = \sigma \, dB_t$
Q denotes the law of: $ds_t = f(s_t, t)\, dt + \sigma \, dB_t$

Radon-Nikodym Derivative: Then under suitable regularity conditions via Girsanov theorem :

$$\frac{dQ}{dW}(s) = \exp\left(\frac{1}{\sigma^2} \int_0^T f(s_t, t) \cdot ds_t - \frac{1}{2\sigma^2} \int_0^T \|f(s_t, t)\|^2 \, dt \right) \tag{4.14}$$

and $\mathbb{E}_W \left(\frac{dQ}{dW} \right) = 1$, because:

$$\int_{\text{paths}} \frac{dQ(s)}{dW} \, dW(s) = \int dQ(s) = 1 \tag{4.15}$$

Score Function Estimator via Log-Likelihood Ratio (REINFORCE Trick)

Gradient Computation for Parameterized Law Q_θ: Now we consider a parameterized law Q_θ:

$$ds_t = f(s_t, x_t; \theta)\, dt + \sigma\, dB_t \tag{4.16}$$

$$\mathbb{E}_{Q_\theta}\left[\int_0^T C(s_t)\, dt\right] = \int_{\text{paths}} \int_0^T C(s_t)\, dt\, dQ_\theta(s) \tag{4.17}$$

As $dQ_\theta(s) = \frac{dQ_\theta(s)}{dW}\, dW(s)$, so:

$$\begin{aligned} \int_{\text{paths}} \int_0^T C(s_t)\, dt\, dQ_\theta(s) &= \int_{\text{paths}} \int_0^T C(s_t)\, dt\, \frac{dQ_\theta(s)}{dW}\, dW(s) \\ &= \mathbb{E}_W\left[\left(\int_0^T C(s_t)\, dt\right) \frac{dQ_\theta(s)}{dW}\right] \end{aligned} \tag{4.18}$$

Then:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{Q_\theta}\left[\int_0^T C(s_t)\, dt\right] &= \nabla_\theta\left(\mathbb{E}_W\left[\int_0^T C(s_t)\, dt\, \frac{dQ_\theta(s)}{dW}\right]\right) \\ &= \mathbb{E}_W\left[\int_0^T C(s_t)\, dt\, \frac{dQ_\theta(s)}{dW} \nabla_\theta \log \frac{dQ_\theta(s)}{dW}\right] \end{aligned} \tag{4.19}$$

where:

$$\log \frac{dQ_\theta(s)}{dW} = \frac{1}{\sigma^2}\int_0^T f(s_t, x_t; \theta) \cdot ds_t - \frac{1}{2\sigma^2}\int_0^T \|f(s_t, x_t; \theta)\|^2\, dt \tag{4.20}$$

Thus:

$$\nabla_\theta \log \frac{dQ_\theta(s)}{dW} = \frac{1}{\sigma^2}\int_0^T \nabla_\theta f(s_t, x_t; \theta) \cdot ds_t - \frac{1}{\sigma^2}\int_0^T \nabla_\theta f(s_t, x_t; \theta) f(s_t, x_t; \theta)\, dt \tag{4.21}$$

This generalizes the result of [30] for noisy 1st order dynamical systems in continuous time.

Application. In practice, we would consider a forward phase that follows the SDE :

$$ds_t = f(s_t, x_t, \theta)\, dt + \sigma\, dB_t$$

We would then consider multiple stochastic trajectories given by this SDE, to estimate the gradient estimator, by averaging the values of :

$$\int_0^T C(s_t)\, dt\, \frac{dQ_\theta(s)}{dW} \nabla_\theta \log \frac{dQ_\theta(s)}{dW}$$

4.6.4 EP-like Learning Rule for Continuous-Time SDEs

We derive now, a true EP like learning rule, which would be based on a free phase, and a nudge phase.

We consider the following law of the stochastic trajectory, which follows a 1st order SDE :

$$Q_{\theta(\beta)} : \text{The law of : } ds_t = \nabla_s E(s_t, x_t, \theta)\, dt + \beta\ \nabla_s c(s_t)\, dt + \sigma\, dB_t \tag{4.22}$$

Here we consider a scalar potential $E(s_t, x_t, \theta)$ from which we get the gradient $\nabla_s E(s_t, x_t, \theta)$ that governs the SDE.

We consider the expression:

$$\mathbb{E}_{Q_{\theta(\beta)}}\left[\int_0^T E(s_t, x_t, \theta)\, dt + \beta \int_0^T C(s_t)\, dt\right] \triangleq A_\theta(\beta) \tag{4.23}$$

and we are looking to derive an EP-like rule with:

$$\nabla_\theta \partial_\beta A_\theta(\beta) = \partial_\beta \nabla_\theta A_\theta(\beta) \tag{4.24}$$

which should give:

$$\nabla_\theta \mathbb{E}_{Q_{\theta(\beta)}}\left[\int_0^T C(s_t)\, dt\right] = \partial_\beta \mathbb{E}_{Q_{\theta(\beta)}}\left[\nabla_\theta \int_0^T E(s_t, x_t, \theta)\, dt\right] + \text{ terms} \tag{4.25}$$

Objective Expectation:

$$\begin{aligned} A_\theta^\beta &= \mathbb{E}_{Q_\theta^\beta}\left(\int_0^T E(s_t, x_t, \theta)\, dt + \beta \int_0^T c(s_t)\, dt\right) \\ &= \int_{\text{paths}} \left(\int_0^T E\, dt + \beta \int_0^T c\, dt\right) dQ_\theta^\beta(s) \end{aligned} \tag{4.26}$$

Radon-Nikodym Derivative: as $dQ_\theta^\beta(s) = \frac{dQ_\theta^\beta(s)}{dW}\, dW(s)$

and:

$$\frac{dQ_\theta^\beta}{dW}(s) = \exp\left(\frac{1}{\sigma^2}\int_0^T (\nabla_s E + \beta \nabla_s c) \cdot ds_t - \frac{1}{2\sigma^2}\int_0^T \|\nabla_s E + \beta \nabla_s c\|^2\, dt\right) \tag{4.27}$$

Expectation under Base Measure W: Thus:

$$A_\theta^\beta = \mathbb{E}_W\left(\int_0^T (E + \beta c)\, dt\ \frac{dQ_\theta^\beta(s)}{dW}\right) \tag{4.28}$$

Derivation of Second-Order Mixed Gradients $\nabla_\theta \nabla_\beta A_\theta^\beta$

Gradient with respect to θ:

$$\begin{aligned} \nabla_\theta A_\theta^\beta &= \nabla_\theta \mathbb{E}_W\left(\int (E + \beta c)\, dt\ \frac{dQ_\theta^\beta(s)}{dW}\right) \\ &= \mathbb{E}_W\left(\int \nabla_\theta E\, dt\ \frac{dQ}{dW} + \int (E + \beta c)\, dt\ \nabla_\theta \log\left(\frac{dQ}{dW}\right)\frac{dQ}{dW}\right) \end{aligned} \tag{4.29}$$

$$\Rightarrow \nabla_\beta \nabla_\theta A_\theta^\beta = \nabla_\beta \mathbb{E}_W \left(\int \nabla_\theta E \, dt \, \frac{dQ}{dW} \right) + \nabla_\beta \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \nabla_\theta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \tag{4.30}$$

Derivative with respect to β: And similarly:

$$\partial_\beta A_\theta^\beta = \mathbb{E}_W \left(\int c \, dt \, \frac{dQ}{dW} \right) + \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \nabla_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \tag{4.31}$$

$$\Rightarrow \nabla_\theta \partial_\beta A_\theta^\beta = \nabla_\theta \mathbb{E}_W \left(\int c \, dt \, \frac{dQ}{dW} \right) + \nabla_\theta \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \tag{4.32}$$

Equality of Mixed Partials Result: Thus:

$$\begin{aligned} \nabla_\theta \mathbb{E}_{Q_\theta^\beta} \left(\int c \, dt \right) &= \nabla_\theta \mathbb{E}_W \left(\int c \, dt \, \frac{dQ}{dW} \right) \\ &= \partial_\beta \mathbb{E}_W \left(\int \nabla_\theta E \, dt \, \frac{dQ}{dW} \right) + \partial_\beta \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \nabla_\theta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \\ &\quad - \nabla_\theta \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \end{aligned} \tag{4.33}$$

Remark 4.6.1. *The term*

$$\nabla_\theta \mathbb{E}_W \left(\int (E + \beta c) \, dt \, \partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right)$$

would be developed by

$$\begin{aligned} \nabla_\theta \mathbb{E}_W \left[\int (E + \beta c) \, dt \cdot \partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right] &= \mathbb{E}_W \left[\int \nabla_\theta E \, dt \cdot \partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right] \\ &\quad + \mathbb{E}_W \left[\int (E + \beta c) \, dt \cdot \nabla_\theta \left(\partial_\beta \log \left(\frac{dQ}{dW} \right) \frac{dQ}{dW} \right) \right] \end{aligned} \tag{4.34}$$

Application. In practice, we would consider a forward free phase ($\beta = 0$) and a nudge phase ($\beta \neq 0$). Both follows the SDE :

$$ds_t = \nabla_s E(s_t, x_t, \theta) \, dt + \beta \, \nabla_s c(s_t) \, dt + \sigma \, dB_t$$

We would then consider multiple stochastic trajectories given by this SDE, to estimate the gradient estimator, by averaging the values of the terms to estimate the gradient estimator.

4.7 2nd order stochastic dynamical systems

We extend the previous architecture developed for dynamical noisy data which followed a 1st order SDEs, to 2nd order SDEs.

4.7.1 Score function estimator for Stochastic 2nd order dynamical systems in continuous time

We consider the following 2nd order SDE, written in terms of a system of 1st order SDEs

$$\boxed{\begin{aligned} dq &= \nabla_p H_\theta \, dt + \sigma_q \, dW_t^q, \\ dp &= -\nabla_q H_\theta \, dt - \gamma \, \nabla_p H_\theta \, dt + \sigma_p \, dW_t^p, \end{aligned}}$$

with W^q and W^p independent standard Brownian motions. The diffusion matrix is now

$$\Sigma = \begin{pmatrix} \sigma_q & 0 \\ 0 & \sigma_p \end{pmatrix},$$

which has full rank as long as $\sigma_q > 0$ and $\sigma_p > 0$.

We consider

Q_θ the law of (q, p) and W the lax of $(\sigma_q \, dW_t^q, \sigma_p \, dW_t^p)$

Q_θ and the driftless reference W have the same diffusion matrix.

Girsanov density for the joint process

Girsanov applies, and the Radon–Nikodym derivative is well-defined.

Choose the reference W as the law of

$$dq = \sigma_q \, dW_t^q, \qquad dp = \sigma_p \, dW_t^p.$$

Then

$$\frac{dQ_\theta}{dW}(q,p) = \exp\left(\int_0^T \begin{pmatrix} \nabla_p H_\theta / \sigma_q^2 \\ f_\theta / \sigma_p^2 \end{pmatrix} \cdot \begin{pmatrix} dq \\ dp \end{pmatrix} - \frac{1}{2} \int_0^T \left(\frac{\|\nabla_p H_\theta\|^2}{\sigma_q^2} + \frac{\|f_\theta\|^2}{\sigma_p^2} \right) dt \right),$$

with $f_\theta = -\nabla_q H_\theta - \gamma \, \nabla_p H_\theta$.

We consider the quantity :

$$A_\theta = \mathbb{E}_{Q_\theta}\left[\int_{t_0}^{t_1} C(q(t)) dt \right]$$

Then we are looking for :

$$\nabla_\theta A_\theta$$

Computation

We have :

$$\mathbb{E}_{Q_\theta}\left[\int_0^T C(q_t,p_t)\,dt\right] = \int_{\text{paths}}\int_0^T C(q_t,p_t)\,dt\,dQ_\theta(q,p) \tag{4.35}$$

As $dQ_\theta(q,p) = \frac{dQ_\theta(q,p)}{dW}\,dW(q,p)$, so:

$$\begin{aligned}\int_{\text{paths}}\int_0^T C(q_t,p_t)\,dt\,dQ_\theta(q,p) &= \int_{\text{paths}}\int_0^T C(q_t,p_t)\,dt\,\frac{dQ_\theta(s)}{dW}\,dW(q,p)\\ &= \mathbb{E}_W\left[\left(\int_0^T C(q_t,p_t)\,dt\right)\frac{dQ_\theta(q,p)}{dW}\right]\end{aligned} \tag{4.36}$$

Then:

$$\begin{aligned}\nabla_\theta\mathbb{E}_{Q_\theta}\left[\int_0^T C(q_t,p_t)\,dt\right] &= \nabla_\theta\left(\mathbb{E}_W\left[\int_0^T C(q_t,p_t)\,dt\,\frac{dQ_\theta(q,p)}{dW}\right]\right)\\ &= \mathbb{E}_W\left[\int_0^T C(q_t,p_t)\,dt\,\frac{dQ_\theta(q,p)}{dW}\nabla_\theta\log\frac{dQ_\theta(q,p)}{dW}\right]\end{aligned} \tag{4.37}$$

where:

$$\log\frac{dQ_\theta(q,p)}{dW} = \int_0^T\begin{pmatrix}\nabla_p H_\theta/\sigma_q^2\\ f_\theta/\sigma_p^2\end{pmatrix}\cdot\begin{pmatrix}dq\\ dp\end{pmatrix} - \frac{1}{2}\int_0^T\left(\frac{\|\nabla_p H_\theta\|^2}{\sigma_q^2}+\frac{\|f_\theta\|^2}{\sigma_p^2}\right)dt \tag{4.38}$$

Because :

$$\frac{dQ_\theta}{dW}(q,p) = \exp\left(\int_0^T\begin{pmatrix}\nabla_p H_\theta/\sigma_q^2\\ f_\theta/\sigma_p^2\end{pmatrix}\cdot\begin{pmatrix}dq\\ dp\end{pmatrix} - \frac{1}{2}\int_0^T\left(\frac{\|\nabla_p H_\theta\|^2}{\sigma_q^2}+\frac{\|f_\theta\|^2}{\sigma_p^2}\right)dt\right),$$

with $f_\theta = -\nabla_q H_\theta - \gamma\,\nabla_p H_\theta$.
Thus:

$$\nabla_\theta\log\frac{dQ_\theta(q,p)}{dW} = \int_0^T\begin{pmatrix}\nabla_\theta\nabla_p H_\theta/\sigma_q^2\\ \nabla_\theta f_\theta/\sigma_p^2\end{pmatrix}\cdot\begin{pmatrix}dq\\ dp\end{pmatrix} - \frac{1}{2}\int_0^T\left(\frac{2\nabla_p H_\theta^\top\nabla_\theta\nabla_p H_\theta}{\sigma_q^2}+\frac{2f_\theta^\top\nabla_\theta f_\theta}{\sigma_p^2}\right)dt \tag{4.39}$$

This generalizes the result of [30] for noisy dissipative 2nd order dynamical systems in continuous time.

Remark

The integral $\int_0^T\begin{pmatrix}\nabla_p H_\theta/\sigma_q^2\\ f_\theta/\sigma_p^2\end{pmatrix}\cdot\begin{pmatrix}dq\\ dp\end{pmatrix}$ is an Itô stochastic integral, because dq and dp are Itô differentials.

4.7.2 EP for stochastic 2nd order dynamical systems in continuous time

We consider the previous 2nd order SDE, written in terms of a system of 1st order SDEs

$$\boxed{\begin{aligned} dq &= \nabla_p H(\theta)dt + \beta\,\nabla_p C\,dt + \sigma_q\,dW_t^q, \\ dp &= -\nabla_q H_\theta\,dt - \beta\,\nabla_q C\,dt - \gamma\,\nabla_p H_\theta\,dt - \gamma\,\beta\,\nabla_p C\,dt + \sigma_p\,dW_t^p, \end{aligned}}$$

with W^q and W^p independent standard Brownian motions. The diffusion matrix is now

$$\Sigma = \begin{pmatrix} \sigma_q & 0 \\ 0 & \sigma_p \end{pmatrix},$$

which has full rank as long as $\sigma_q > 0$ and $\sigma_p > 0$.

We consider

Q_θ^β the law of (q,p) and W the law of $(\sigma_q\,dW_t^q, \sigma_p\,dW_t^p)$

Q_θ^β and the driftless reference W have the same diffusion matrix.

Girsanov density for the joint process

Girsanov applies, and the Radon–Nikodym derivative is well-defined.

Choose the reference W as the law of

$$dq = \sigma_q\,dW_t^q, \qquad dp = \sigma_p\,dW_t^p.$$

Then

$$\frac{dQ_\theta^\beta}{dW}(q,p) = \exp\left(\int_0^T \begin{pmatrix} (\nabla_p H_\theta + \beta\nabla_p C)/\sigma_q^2 \\ f_\theta^\beta/\sigma_p^2 \end{pmatrix} \cdot \begin{pmatrix} dq \\ dp \end{pmatrix} - \frac{1}{2}\int_0^T \left(\frac{\|\nabla_p H_\theta + \beta\nabla_p C\|^2}{\sigma_q^2} + \frac{\|f_\theta^\beta\|^2}{\sigma_p^2} \right) dt \right),$$

with $f_\theta^\beta = -\nabla_q H_\theta - \beta\nabla_q C - \gamma\,\nabla_p H_\theta - \beta\,\gamma\,\nabla_p C$

We consider the quantity :

$$A_\theta^\beta = \mathbb{E}_{Q_\theta^\beta}\left[\int_{t_0}^{t_1} H_\theta(q(t),p(t))dt + \beta \int_{t_0}^{t_1} C(q(t),p(t))dt \right]$$

Then we consider Schwarz formula :

$$\nabla_\theta \partial_\beta A_\theta^\beta = \partial_\beta \nabla_\theta A_\theta^\beta$$

Following the same steps of the 1st order SDE (see above) we would get the expression of $\nabla_\theta \mathbb{E}_{Q_\theta^\beta}\left[\int_{t_0}^{t_1} C(q(t),p(t))dt \right]$ from the Schwarz formula.

Application. The application would be similar to the SDE framework (see above).

Chapter 5

EP and LEP for reinforcement learning and control

5.1 Closed loop deterministic control with LEP PFVP

In classical (deterministic) optimal control [38], The dynamics is given by $\dot{x} = f(t, x, u)$ and cost by $\int_0^T l(t, x, u)\, dt + g(x_T)$

We make an assumption for the applicability of the LEP PFVP algorithm :

Instead of general dynamics $\dot{x} = f(t, x, u)$ we consider Lagrangian dynamics governed by a Lagrangian $L_0(x, \dot{x}, u)$. Furthermore we suppose that this Lagrangian satisfies time-reversal symmetry

$$L_0(x, \dot{x}, u) = L_0(x, -\dot{x}, u)$$

We also consider that the cost doesn't depend explicitly on the control u (from an RL perspective, we evaluate only the state, not the pair of state and action) : $\int_0^T l(t, x)\, dt + g(x_T)$

Furthermore, to allow the applicability of the LEP PFVP algorithm, we suppose that the cost doesn't evaluate velocity.

Remark 5.1.1. *RHEL can be used, when the cost depends on both position, and velocity, nevertheless, we consider here LEP PFVP, because it can handle dissipative systems, unlike RHEL.*

To apply the LEP PFVP algorithm, we naively start by considering a functional derivative w.r.t control u : δ_u,

Thus we consider the free controled action :

$$S_0(x, u) = \int_0^T L_0(t, x, u)\, dt$$

and the nudge controlled action :

$$S_\beta(x, u) = \int_0^T L_0(t, x, u) + \beta L(t, x)\ dt + \beta g(x_T)$$

Nevertheless, this would result in a requirement of BPTT memory (similar to LEP CIVP), which makes LEP PFVP inherently incompatible with such approach (against the philosophy

of EP of computational efficiency), because we would have to store all the functional derivatives at all time steps after the first free phase. Thus we consider instead a framework of closed loop control to avoid the requirement of functional-derivative (requires BPTT memory even with LEP PFVP).

5.1.1 Setup: substitute the policy into the Lagrangian

With $u(t) = \pi_\theta(x(t))$, we define the effective Lagrangian

$$\tilde{L}_\beta(x, \dot{x}, \theta) := L_\beta(x, \dot{x}, \pi_\theta(x))$$

This is now a Lagrangian parametrized by a genuine finite-dimensional θ — used identically at every instant, through the fixed map π_θ.

The controlled Euler–Lagrange equation becomes an ordinary (no-u) equation in x alone:

$$\frac{d}{dt}\partial_{\dot{x}} L_\beta(x, \dot{x}, \pi_\theta(x)) - \partial_x L_\beta(x, \dot{x}, \pi_\theta(x)) - \partial_u L_\beta(x, \dot{x}, \pi_\theta(x)) \cdot \partial_x \pi_\theta(x) = 0$$

(using $\partial_{\dot{x}} \tilde{L}_\beta = \partial_{\dot{x}} L_\beta$ since $\pi_\theta(x)$ carries no $\dot{x}$-dependence, and $\partial_x \tilde{L}_\beta = \partial_x L_\beta + \partial_u L_\beta \cdot \partial_x \pi_\theta(x)$ by the chain rule).

5.1.2 Closed loop LEP PFVP control

Since θ is finite-dimensional, we can apply the PFVP formula verbatim, with $\tilde{L}_\beta$ in place of L_β:

We consider a PFVP LEP control, where we consider a natural dynamics governed by a reversible Lagrangian :

$$L_\beta(x, \dot{x}, \pi_\theta(x)) = L_\beta(x, -\dot{x}, \pi_\theta(x))$$

where

$$u(t) = \pi_\theta(x(t))$$

We consider again the effective Lagrangian

$$\tilde{L}_\beta(x, \dot{x}, \theta) := L_\beta(x, \dot{x}, \pi_\theta(x))$$

The augmented Action is defined as :

$$\int_0^T L_\beta(x, \dot{x}, \pi_\theta(x)) + \beta\big(\int_0^T l(t, x(t))\, dt + g(x_T)\big)$$

We directly use the LEP PFVP gradient formula for parametrized initial position $x(0) = \alpha_0$ and velocity $\dot{x}(0) = \gamma_0$

Thus we have

$$d_\theta C[\theta] = \lim_{\beta \to 0} \frac{1}{\beta}\Big[\int_0^T \big(\partial_\theta \tilde{L}_\beta(x_\beta, \dot{x}_\beta, \theta) - \partial_\theta \tilde{L}_0(x_0, \dot{x}_0, \theta)\big) dt + \partial_\theta \alpha_0^\top \big(\partial_{\dot{x}} \tilde{L}_\beta(0) - \partial_{\dot{x}} \tilde{L}_0(0)\big) - \big(d_\theta \partial_{\dot{x}} \tilde{L}_0(\alpha_0(\theta), \gamma_0(\theta))\big)^\top (x_\beta(0) - \alpha_0(\theta)\Big]$$

Which is

$$
\begin{aligned}
d_\theta C[\theta] = \lim_{\beta \to 0} \frac{1}{\beta} \Big[\int_0^T & \big(\partial_u L_\beta(x_\beta(t), \dot{x}_\beta(t), \pi_\theta(x_\beta(t))) \cdot \partial_\theta \pi_\theta(x_\beta(t))) - \\
& \partial_u L_0(x_0(t), \dot{x}_0(t), \pi_\theta(x_0(t))) \cdot \partial_\theta \pi_\theta(x_0(t))\big) dt \\
& + \partial_\theta \alpha_0(\theta)^\top \big(\partial_{\dot{x}} L_\beta(x_\beta(0), \dot{x}_\beta(0), \pi_\theta(x_\beta(0)) - \partial_{\dot{x}} L_0(\alpha_0(\theta), \gamma_0(\theta), \pi_\theta(\alpha_0(\theta)))\big) \\
& - d_\theta \partial_{\dot{x}} L_0(\alpha_0(\theta), \gamma_0(\theta), \pi_\theta(\alpha_0(\theta)))^\top (x_\beta(0) - \alpha_0(\theta)) \Big]
\end{aligned}
$$

5.2 Energy-Based EP actor(stochastic)-critic RL

We consider an actor critic RL framework which is fully implementable with EP, unlike the proposal of Kubo, Chalmers, and Luczak [17] which relied on backpropagation to train the critic network. Furthermore,we propose the application of Litman's framework [15] to train the actor stochastic policy with finite error signals.

Policy Definition

The policy is defined via the Boltzmann distribution of the energy function combined with the Q-value:

$$\pi_{\beta,\theta}(A|S) \propto \exp\left(-\frac{1}{T}(E(S,A,\theta) - \beta Q_w(S,A))\right) \tag{5.1}$$

and

$$\pi_{0,\theta}(A|S) \propto \exp -\frac{1}{T}(E(S,A,\theta)) \tag{5.2}$$

Objective Function $J(\theta)$

The objective $J(\theta)$ can be taken to maximize the expected Q-value:

$$J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\pi_\theta(.|S)}[Q_w(S,A)] \quad \max J(\theta) \text{ (Expected Q-maximization)} \tag{5.3}$$

Nevertheless, we aim to consider a framework compatible Litman's proposal [15], thus we consider :

$$J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\pi^0_\theta(.|S)}[Q_w(S,A)] + TKL(\pi^1_\theta(.|S)||\pi^0_\theta(.|S))$$

5.2.1 Critic Optimization

To optimize the critic Q_w, we minimize the Temporal Difference (TD) error:

$$\text{Min}_w \frac{1}{2} \left(Q_w(S,A) - (R + \gamma Q_w(S',A'))\right)^2 \tag{5.4}$$

Critic network "deterministic EP"

The actor chooses the best action given a state, so it needs the critic to evaluate a pair of (state,action). The critics evaluate a pair of (state,action) by estimating the reward. The actor learns a stochastic policy, and thus learns using the stochastic EP algorithm, while the critic uses the classical deterministic EP algorithm.

At the start we have π_θ and Q_w (θ,w initialized)

For a given transition (S, A, S', R):

we have

$$S \rightarrow^{\pi_\theta} A \rightarrow S^{'}, R \rightarrow^{\pi_\theta} A^{'} \tag{5.5}$$

1. Current Free Phase:

$$(S, A)_{input} \rightarrow G(S, A, w, V)_{\text{EBM of critic}} \rightarrow V_0^*{}_{\text{Prediction for (S,A)}} \tag{5.6}$$

2. Nudged Phase: Define the energy landscape for the critic as:

$$H(S, A, w, \alpha, V) = G(S, A, w, V) + \frac{\alpha}{2}\left(V - (R + \gamma\ argmin_V(G(S^{'}, A^{'}, w, V))\right)^2 \tag{5.7}$$

Then Perform Equilibrium Propagation to update the parameters w.

Here the input signal of the EBM (represented by x in classical EP "$E(x, \theta, s)$") is the pair of state action (S, A) of the RL point of view (the "state" of RL is different from the state of the EBM).

The state of the EBM (represented by s in classical EP "$E(x, \theta, s)$") here is represented by V in $G(S, A, w, V)$ and it represents the evaluation of the critic of the given pair of (state,action) using the parameter of the EBM w.

The nudge parameter is α, and the cost function which is aimed to be minimized in classical EP (represented by C) is there replaced by the objective to be minimized by the critic to correctly assign the prediction of the value of the pair of state action using the reward R : $\frac{1}{2}\left(V - (R + \gamma\ argmin_V(G(S^{'}, A^{'}, w, V))\right)^2$

Remark 5.2.1. *To make this approach work, we need to ignore the dependency of the reward on the learning parameters, which is also done on classical RL.*

We would also have to ignore the dependency on the parameters of $\arg\min_V(G(S^{'}, A^{'}, w, V)$.

5.2.2 Actor network "stochastic EP"

We extend this to RL by defining the policy and critic using the EBM framework.

Policy Definition

The policy is derived from the Boltzmann distribution of the Q-function:

The free phase is :

$$\pi_{\theta,0}(A|S) \propto \exp(-\frac{1}{T}(E(S,A,\theta)) \tag{5.8}$$

The nudge phase is :

$$\pi_{\theta,\beta}(A|S) \propto \exp\left(-\frac{1}{T}(E(S,A,\theta) - \beta Q_w(S,A))\right) \tag{5.9}$$

Here the state of the stochastic EBM represents the action A of the RL agent. The input of the EBM is the state S of the RL agent, and the cost of the agent is evaluated given the input "state" by the critic $Q_w(S,A)$. The goal would be to maximize (thus minimize -$Q_w(S,A)$ the critics evaluation.

Using Litman's proposal [15], we can update the parameters of the actors using the laerning rule :

$$\nabla_\theta J(\theta) = (\mathbb{E}_{s\sim\pi_1}[\nabla_\theta E] - \mathbb{E}_{s\sim\pi_0}[\nabla_\theta E]) \tag{5.10}$$

5.3 Distributional RL: Matching Return Distributions

In standard RL, the goal is to estimate the expected cumulative return $Q(s,a)$. In Distributional RL [39], we instead parameterize the full probability distribution of the returns, $Z_\theta(s,a)$.

The Parameterized Distribution: A model $Z_\theta(s,a)$ predicting the probabilities of different return values.

The Goal Distribution: The target Bellman distribution $\mathcal{T}Z_{target}(s,a) \overset{D}{=} R(s,a) + \gamma Z_{target}(S',A')$.

In standard RL, the Bellman equation for a policy π focuses on the expected return $Q^\pi(s,a) = \mathbb{E}[Z^\pi(s,a)]$. The standard Bellman operator applies to these expected values. In Distributional RL, we drop the expectation and work directly with the random variable of the return, $Z^\pi(s,a)$.

The Distributional Bellman Equation

The foundation is the distributional Bellman equation, which states that the distribution of returns at the current state-action pair is recursively related to the distribution of returns at the next state-action pair:

$$Z^\pi(s,a) \overset{D}{=} R(s,a) + \gamma Z^\pi(S',A')$$

Where:

$Z^\pi(s,a)$ is the random variable representing the cumulative discounted return.

$\overset{D}{=}$ denotes equality in distribution (the two random variables have the same probability distribution).

$R(s,a)$ is the random variable of the immediate reward.$S' \sim P(\cdot|s,a)$ is the next state sampled from the transition dynamics.

$A' \sim \pi(\cdot|S')$ is the next action sampled from the policy.

The Distributional Bellman Operator($\mathcal{T}^\pi$)

Let $\mathcal{Z}$ be the space of state-action value distributions (mappings from state-action pairs to probability distributions over $\mathbb{R}$).The distributional Bellman operator $\mathcal{T}^\pi : \mathcal{Z} \to \mathcal{Z}$ is

$$\mathcal{T}^\pi Z(s,a)(x) = \mathbb{E}_{s' \sim P, a' \sim \pi}\left[f_{R+\gamma Z(s',a')}(x)\right]$$
$$\mathcal{T}^\pi Z(s,a)(x) = \sum_{s',a'} P(s'|s,a)\pi(a'|s') f_{R+\gamma Z(s',a')}(x)$$

To prove that an RL algorithm converges, we rely on Banach's Fixed-Point Theorem, which requires the Bellman operator to be a contraction mapping. (Meaning applying the operator brings any two distributions closer together). In standard RL, the Bellman operator is a γ-contraction in the supremum norm. However, in Distributional RL, the operator $\mathcal{T}^\pi$ is not a contraction in Kullback-Leibler (KL) divergence or Total Variation distance.

The Cramér Distance (Energy Distance):

Defined as the L_2 distance between the Cumulative Distribution Functions (CDFs) of two distributions [40]:

$$l_2^2(P,Q) = \int_{-\infty}^{\infty} (F_P(x) - F_Q(x))^2 dx$$

The Bellman operator is a $\sqrt{\gamma}$-contraction in the Cramér distance [41]. This is highly useful in practice because, unlike the Wasserstein metric, the Cramér distance has unbiased sample gradients, making it easier to optimize with neural networks.

EP approach for distributional RL with Cramer distance The Cramér distance can be rewritten equivalently using the Energy Distance formulation [40]:

$$l_2^2(P,Q) = \mathbb{E}[|X-Y|] - \frac{1}{2}\mathbb{E}[|X-X'|] - \frac{1}{2}\mathbb{E}[|Y-Y'|]$$

1. Definition of the Variables

Let: X and X' be two independent and identically distributed (i.i.d.) random variables drawn from distribution P.

Y and Y' be two i.i.d. random variables drawn from distribution Q.

X and Y are independent of each other.

2. Component-by-Component Breakdown

The formula consists of three distinct expectation terms:

$$\underbrace{l_2^2(P,Q)}_{\text{Cramér Distance}} = \underbrace{\mathbb{E}[|X-Y|]}_{\text{Cross-Distance}} - \underbrace{\frac{1}{2}\mathbb{E}[|X-X'|]}_{\text{Internal Spread of } P} - \underbrace{\frac{1}{2}\mathbb{E}[|Y-Y'|]}_{\text{Internal Spread of } Q}$$

Mathematically, Parseval's theorem and related integral identities prove that the squared difference between two CDFs integrated over the entire real line:

$$l_2^2(P,Q) = \int_{-\infty}^{\infty} (F_P(x) - F_Q(x))^2 \, dx$$

is identically equal to the expected absolute distance formulation.

Because X and X' are defined as independent and identically distributed (i.i.d.) random variables drawn from the same distribution P, their joint distribution is simply the product of

their individual marginal distributions.If the distribution P has a probability density function $p(x)$, the joint probability density of (X, X'), denoted as $p_{X,X'}(x, x')$, factorizes completely:

$$p_{X,X'}(x, x') = p(x)p(x')$$

Expanding the Expectations

Using Joint Distributions To compute or express the expectations in the energy distance formula using probability distributions, we integrate the absolute difference function against these joint densities.Here is how each of the three expectation terms expands into double integrals:

1. The Internal Spread of P: $\mathbb{E}[|X - X'|]$

Since X and X' are independent draws from P, their joint distribution is $p(x)p(x')$. The expectation becomes a 2D integral over the real plane:

$$\mathbb{E}[|X - X'|] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} |x - x'|\, p(x)\, p(x')\, dx\, dx'$$

2. The Internal Spread of Q: $\mathbb{E}[|Y - Y'|]$

Similarly, Y and Y' are i.i.d. draws from the target distribution Q with density $q(y)$. Their joint distribution is $q(y)q(y')$:

$$\mathbb{E}[|Y - Y'|] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} |y - y'|\, q(y)\, q(y')\, dy\, dy'$$

3. The Cross-Distance Term: $\mathbb{E}[|X - Y|]$

Here, X is drawn from P and Y is drawn from Q. By definition of the energy distance framework, X and Y are mutually independent of each other. Therefore, their joint distribution is the product of their independent margins, $p(x)q(y)$:

$$\mathbb{E}[|X - Y|] = \int_{\mathbb{R}^2} |x - y|\, p(x)\, q(y)\, dx\, dy$$

Putting It All Together : Substituting these integral expansions back into the original formula yields the full continuous distribution form of the energy (Cramér) distance:

$$l_2^2(P, Q) = \int_{\mathbb{R}^2} |x-y|\, p(x)q(y)\, dx\, dy - \frac{1}{2} \int_{\mathbb{R}^2} |x-x'|\, p(x)p(x')\, dx\, dx' - \frac{1}{2} \int_{\mathbb{R}^2} |y-y'|\, q(y)q(y')\, dy\, dy'$$

5.3.1 Gradient derivation

1. Differentiating the Cross-Term ($\mathbb{E}[|X - Y|]$)

Since only X depends on θ (the target distribution Q is fixed), the gradient applies directly inside the integral:

$$\nabla_\theta \mathbb{E}[|X - Y|] = \int_{\mathbb{R}^2} |x - y| \nabla_\theta p_\theta(x) q(y) dx dy$$

Applying the log-derivative trick ($\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$):

$$\nabla_\theta \mathbb{E}[|X - Y|] = \int_{\mathbb{R}^2} |x - y| p_\theta(x) \nabla_\theta \log p_\theta(x) q(y) dx dy$$

$$\nabla_\theta \mathbb{E}[|X - Y|] = \mathbb{E}_{X\sim P, Y\sim Q}\left[|X - Y| \nabla_\theta \log p_\theta(X)\ \right]$$

2. Differentiating the Internal Spread Term ($\mathbb{E}[|X - X'|]$)

Because both X and X' are drawn independently from p_θ, their joint distribution is $p_\theta(x)p_\theta(x')$. Applying the product rule for derivatives:

$$\nabla_\theta \mathbb{E}[|X - X'|] = \int_{\mathbb{R}^2} |x - x'| \left[\nabla_\theta p_\theta(x) p_\theta(x') + p_\theta(x) \nabla_\theta p_\theta(x')\right] dx dx'$$

Substituting the score function into both terms:

$$\nabla_\theta \mathbb{E}[|X - X'|] = \int_{\mathbb{R}^2} |x - x'| p_\theta(x) p_\theta(x') \left[\nabla_\theta \log p_\theta(x) + \nabla_\theta \log p_\theta(x')\right] dx dx'$$

Because X and X' are identically distributed, these two integral terms are completely symmetric and equal to each other:

$$\nabla_\theta \mathbb{E}[|X - X'|] = 2\, \mathbb{E}_{X,X'\sim P}\left[|X - X'| \nabla_\theta \log p_\theta(X)\right]$$

3. Putting It Together for the Full Energy Distance Gradient

The squared Cramér (energy) distance is:

$$l_2^2(P, Q) = \mathbb{E}[|X - Y|] - \frac{1}{2}\mathbb{E}[|X - X'|] - \frac{1}{2}\mathbb{E}[|Y - Y'|]$$

Taking the gradient with respect to θ (noting that the Y term has no θ, so its derivative is zero):

$$\nabla_\theta l_2^2(P, Q) = \nabla_\theta \mathbb{E}[|X - Y|] - \frac{1}{2}\nabla_\theta \mathbb{E}[|X - X'|]$$

Substitute the expanded expectation formulas:

$$\nabla_\theta l_2^2(P, Q) = \mathbb{E}\left[|X - Y| \nabla_\theta \log p_\theta(X)\right] - \frac{1}{2}\left(2\, \mathbb{E}\left[|X - X'| \nabla_\theta \log p_\theta(X)\right]\right)$$

The 2 and the $\frac{1}{2}$ cleanly cancel out, leaving a compact final expression:

$$\boxed{\nabla_\theta l_2^2(P, Q) = \mathbb{E}\left[\left(|X - Y| - |X - X'|\right) \nabla_\theta \log p_\theta(X)\ \right]}$$

EP like approach

For this approach we use a philosophy similar to the proposal of a single phase EP, which is based on optimizing directly the Fisher divergence. In this case our objective is the Cramer distance, between our parametrized probability distribution which would play the analog of the free distribution in classical EP, and the the analog of the nudge distribution would be the the target distribution.

We would use the derived gradient to update the parameters, without relying on back-propagation.

Remark 5.3.1. *The target distribution which represent the critic network would also need to be trained by an extra network which could be based on an EP like learning rule or other like*

in the proposals of RL EP [17], [16].

Remark 5.3.2. *This approach is similar to our proposal of single phase Fisher, which itself is similar to the Litman proposal, mainly being Limitless and not based on the implicit function theorem used for EP. But in this case, the target is not a simple additive distribution of the free distribution, which makes this proposal slightly more different than EP.*

Remark 5.3.3 (About the EP/EBM Philosophy)**.** *As stated, this proposal can be implemented in general where the distributions P and Q don't share similar functional forms and are not related to Energy functions, but we could still consider a philosophy close to Energy functions; where the pair of state actions (s, a) are obtained by minimizing the RL distributions P and Q and because we are considering an approach similar to Limitless EP like Litman's and our proposal of single phase Fisher EP. The fact that the 2 distributions don't share similar functional form is irrelevant (we don't need an equality as the nudge term β goes to 0); thus with this proposal we may consider an EP/EBM like philosophy which would be suited for distributional RL.*

5.4 Stochastic deterministic control: Discrete time

We aim to derive an EP-like rule for stochastic control in open loop framework. We consider an approach similar to Chapter 4, where we will optimize over the control variable u_t here by considering the functional derivative $\frac{\delta}{\delta_t}$, instead of the finite dimensional parameter θ (considered previously). We will consider a single phase approach, thus avoiding the requirement of BPTT memory (like in deterministic open loop control with two phases). We consider the following stochastic control dynamics where the joint distribution factorizes into a Markov chain:

$$\mathbb{P}(x_{0:N} \mid u_{0:N}) = p(x_0) \prod_{i=0}^{N-1} p(x_{i+1} \mid x_i),$$

where the transition probability is

$$p(x_{i+1} \mid x_i, u_i) \;\propto\; \exp\left(-\frac{1}{\varepsilon} L_d(x_i, u_i, x_{i+1})\Delta t\right).$$

Inserting the explicit form of L_d,

$$p(x_{i+1} \mid x_i, u_i) \;\propto\; \exp\left(-\frac{1}{4\varepsilon\Delta t}(x_{i+1} - x_i - b(x_i, u_i)\Delta t)^\top D(x_i)^{-1}(x_{i+1} - x_i - b(x_i, u_i)\Delta t)\right).$$

This is a Gaussian density with

$$\mathbb{E}[x_{i+1} \mid x_i, u_i] = x_i + b(x_i, u_i)\Delta t, \qquad \mathrm{Cov}[x_{i+1} \mid x_i, u_i] = 2\varepsilon D(x_i, u_i)\Delta t.$$

Therefore, sampling a trajectory from this path measure is equivalent to the recurrence

$$x_{i+1} = x_i + b(x_i, u_i)\Delta t + \sqrt{2\varepsilon}\, D(x_i, u_i)\sqrt{\Delta t}\, \xi_i, \qquad \xi_i \sim \mathcal{N}(0, I),$$

which is exactly the Euler–Maruyama discretization of the SDE

$$ds_t = b(s_t, u_t)\,dt + \sqrt{2\varepsilon}\,D(s_t, u_t)\,dW_t.$$

Stochastic objective

$$J(u_{0:N}) = \mathbb{E}_{\mathbb{P}(x_{0:N}|u_{0:N})}\left[\sum_{k=0}^{N} L(x_k, u_k) \; + g(x_N)\right]$$

Considering a dynamical discrete stochastic trajectory $x(t)$ and a dynamical discrete parameter trajectory $u(t)$, we optimize the objective with respect to $u(t)$. Expressing $J(u)$ as an explicit sum/integral over trajectory space:

$$J(u_{0:N}) = \int\Big(\sum_{k=0}^{N} L(x_k, u_k) \; + g(x_N)\Big) p_\theta^0(x_{1:N} \mid x_0, u_{0:N})\,dx_{1:N}$$

Discrete functional derivative

For a finite discrete sequence $u_{0:N}$, the functional derivative is just an ordinary partial derivative:

$$\frac{\delta}{\delta u_k} = \frac{\partial}{\partial u_k},$$

or ∇_{u_k} if u_k is a vector. So there is no true functional derivative in the discrete proposal.

Let

$$C(x, u) = \sum_{j=0}^{N} L(x_j, u_j) + g(x_N).$$

Then

$$J(u_{0:N}) = \int C(x, u)\, p_\theta^0(x_{1:N} \mid x_0, u_{0:N})\,dx_{1:N},$$

and the discrete derivative is

$$\frac{\partial J}{\partial u_k} = \mathbb{E}_p\left[\frac{\partial C(x,u)}{\partial u_k}\right] + \mathbb{E}_p\left[C(x,u)\,\frac{\partial}{\partial u_k}\log p(x_{1:N} \mid x_0, u_{0:N})\right].$$

Since u_k appears in the cost only through $L(x_k, u_k)$,

$$\frac{\partial C(x,u)}{\partial u_k} = \nabla_{u_k} L(x_k, u_k).$$

Now the score term. Define

$$r_k = x_{k+1} - x_k - b(x_k, u_k)\Delta t, \qquad B_k = \frac{\partial b(x_k, u_k)}{\partial u_k}.$$

Assuming $D(x_k)$ does not depend on u_k, the log-density derivative is

$$\boxed{\frac{\partial}{\partial u_k}\log p(x_{1:N} \mid x_0, u_{0:N}) = \frac{1}{2\varepsilon}\,B_k^\top D(x_k)^{-1} r_k}$$

or equivalently

$$\frac{1}{2\varepsilon\Delta t}\left(\frac{\partial(b(x_k,u_k)\Delta t)}{\partial u_k}\right)^\top D(x_k)^{-1}r_k.$$

So the final discrete gradient is

$$\boxed{\frac{\delta J}{\delta u_k}=\mathbb{E}_p\left[\nabla_{u_k}L(x_k,u_k)+C(x,u)\,\frac{1}{2\varepsilon}\left(\frac{\partial b(x_k,u_k)}{\partial u_k}\right)^\top D(x_k)^{-1}\big(x_{k+1}-x_k-b(x_k,u_k)\Delta t\big)\right].}$$

5.5 Stochastic deterministic control: Continuous time

We extend the results of the previous section, where we considered discrete time, to continuous time, following the pattern of Chapter 4. We follow the same logic of the previous section but by considering now, continuous time.

5.5.1 Functional derivative

we consider the following framework

W denotes the law of: $ds_t=\sigma\,dB_t$

Q_u denotes the law of the controlled SDE : $ds_t=f(s_t,,u_t,t)\,dt+\sigma\,dB_t$

Radon-Nikodym Derivative: The density (under suitable regularity conditions for Girsanov theorem))is

$$\frac{dQ_u}{dW}(s)=\exp\left(\frac{1}{\sigma^2}\int_0^T f(s_t,u_t,t)\cdot ds_t-\frac{1}{2\sigma^2}\int_0^T\|f(s_t,u_t,t)\|^2\,dt\right),$$

we are interested in $\frac{\delta}{\delta u(t)}\log\frac{dQ_u}{dW}(s)$

Let

$$L=\frac{dQ}{dW}(s).$$

For the usual functional derivative with respect to $u(t)$, holding the path s fixed, one has

$$\frac{\delta}{\delta u(t)}\log L=\frac{1}{\sigma^2}\big(\nabla_u f(s_t,u_t,t)\big)^\top ds_t-\frac{1}{\sigma^2}\big(\nabla_u f(s_t,u_t,t)\big)^\top f(s_t,u_t,t)\,dt.$$

Therefore

$$\boxed{\frac{\delta}{\delta u(t)}\log\frac{dQ_u}{dW}(s)=\frac{1}{\sigma^2}\big(\nabla_u f(s_t,u_t,t)\big)^\top\Big(ds_t-f(s_t,u_t,t)\,dt\Big)}$$

or, in discrete notation,

$$\frac{\partial}{\partial u_t}\log L=\frac{1}{\sigma^2}\big(\nabla_u f_t\big)^\top\big(\Delta s_t-f_t\Delta t\big).$$

Gradient Computation of the objective function: Now we consider the controlled law Q_u:

$$ds_t=f(s_t,u_t,t)\,dt+\sigma\,dB_t \tag{5.11}$$

We have

$$J(u) = \mathbb{E}_{Q_u\left[\int_0^T C(s_t)\,dt\right]=\int_{\text{paths}}\int_0^T C(s_t)\,dt\,dQ_u(s)}(5.12)$$

As $dQ_u(s) = \frac{dQ_u(s)}{dW}\,dW(s)$, so:

$$\begin{aligned}\int_{\text{paths}}\int_0^T C(s_t)\,dt\,dQ_u(s) &= \int_{\text{paths}}\int_0^T C(s_t)\,dt\,\frac{dQ_u(s)}{dW}\,dW(s)\\ &= \mathbb{E}_W\left[\left(\int_0^T C(s_t)\,dt\right)\frac{dQ_u(s)}{dW}\right]\end{aligned} \tag{5.13}$$

Then:

$$\begin{aligned}\frac{\partial}{\partial u_t}\mathbb{E}_{Q_u}\left[\int_0^T C(s_t)\,dt\right] &= \frac{\partial}{\partial u_t}\left(\mathbb{E}_W\left[\int_0^T C(s_t)\,dt\,\frac{dQ_u(s)}{dW}\right]\right)\\ &= \mathbb{E}_W\left[\int_0^T C(s_t)\,dt\,\frac{dQ_u(s)}{dW}\frac{\partial}{\partial u_t}\log\frac{dQ_u(s)}{dW}\right]\end{aligned} \tag{5.14}$$

Or we have :

$$\boxed{\frac{\delta}{\delta u(t)}\log\frac{dQ_u}{dW}(s) = \frac{1}{\sigma^2}\big(\nabla_u f(s_t,u_t,t)\big)^\top\Big(ds_t - f(s_t,u_t,t)\,dt\Big)}$$

This generalizes the result of [30] for noisy controlled dynamical systems in continuous time.

Chapter 6

Single phase EP and novel DL architecture

6.1 Limitless, single phase, EP based on Fisher divergence

6.1.1 Fisher Divergence and Score Matching

To adopt a learning rule based on a single phase, we consider the Fisher divergence as an optimization criteria instead of the KL divergence.

Instead of comparing densities directly, Fisher divergence compares the gradients of log-densities (the score functions):

$$D_F(p \parallel q) = \frac{1}{2} \int p(s) \left\| \nabla_s \log p(s) - \nabla_s \log q_\theta(s) \right\|^2 ds$$

Because taking the derivative with respect to the state s eliminates the parameter-dependent constant $\log Z_E(\theta, x)$:

$$\nabla_s \log \mathcal{L}_E(s \mid x, \theta) = \nabla_s \left(-\frac{1}{T} E(x, s; \theta) - \log Z_E(\theta, x) \right) = -\frac{1}{T} \nabla_s E(x, s; \theta)$$

Score Matching allows to train energy-based models and update parameters without ever sampling from the free phase E or evaluating $Z_E(\theta, x)$, because the normalizer vanishes under the state gradient ∇_s. Thus by considering the Fisher divergence, we eliminate the intractable partition function $Z_E(\theta, x)$ entirely, and we only need states sampled from a single phase (the F-nudged phase).

6.1.2 The Formal Derivation

The Fisher divergence between two distributions $p(s)$ and $q_\theta(s)$ is defined as the expected squared Euclidean distance between their score functions (gradients of log-densities with respect to state s):

$$D_F(p \parallel q_\theta) = \frac{1}{2} \mathbb{E}_{s \sim p} \left[\left\| \nabla_s \log p(s) - \nabla_s \log q_\theta(s) \right\|^2 \right]$$

Now we substitute the distributions:

Target distribution: $p(s) = \mathcal{L}_F^{old}(s \mid x, y) \propto \exp\left(-\frac{1}{T}E(x, s; \theta_{old}) - \beta\frac{1}{T}C(s, y)\right)$

Model distribution: $q_\theta(s) = \mathcal{L}_E^{\theta}(s \mid x) \propto \exp\left(-\frac{1}{T}E(x, s; \theta)\right)$

Computing the Score Functions Model score:

$$\nabla_s \log \mathcal{L}_E(s \mid x, \theta) = \nabla_s \left(-\frac{1}{T}E(x, s; \theta) - \log Z_E(\theta, x)\right) = -\frac{1}{T}\nabla_s E(x, s; \theta)$$

(The partition function $Z_E(\theta, x)$ drops out because it does not depend on s.)

Target score:

$$\nabla_s \log \mathcal{L}_F^{old}(s \mid x, y) = \nabla_s \left(-\frac{1}{T}E(x, s; \theta_{old}) - \frac{1}{T}\beta C(s, y) - \log Z_F\right) = -\frac{1}{T}\nabla_s E(x, s; \theta_{old}) - \beta\frac{1}{T}\nabla_s C(s, y)$$

The Explicit Loss Function

Plugging both score functions into $D_F\left(\mathcal{L}_F^{old} \parallel \mathcal{L}_E^{\theta}\right)$:

$$D_F\left(\mathcal{L}_F^{old} \parallel \mathcal{L}_E^{\theta}\right) = \frac{1}{2T^2}\mathbb{E}_{s\sim\mathcal{L}_F^{old}}\left[\left\| \left(-\nabla_s E(x, s; \theta_{old}) - \beta\nabla_s C(s, y)\right) - \left(-\nabla_s E(x, s; \theta)\right)\right\|^2\right]$$

Simplifying signs inside the norm:

$$\mathcal{J}(\theta) = \frac{1}{2}\mathbb{E}_{s\sim\mathcal{L}_F^{old}}\left[\left\|\nabla_s E(x, s; \theta) - \left(\nabla_s E(x, s; \theta_{old}) + \beta\nabla_s C(s, y)\right)\right\|^2\right]$$

where The objective is taken as (similarly to Fisher EP in Chapter 3) :

$$\mathcal{J}(\theta) = T^2 D_F\left(\mathcal{L}_F^{old} \parallel \mathcal{L}_E^{\theta}\right)$$

6.1.3 Single-Phase EP

True Single-Phase Sampling:

The expectation $\mathbb{E}_{s\sim\mathcal{L}_F^{old}}$ is taken exclusively over F-relaxed states. We relax states under $F = E + \beta C$, collect s_t, and we never need to sample or relax un-nudged E-states during training.

Physical Meaning:

At a fully relaxed nudged state s_F^*, the total nudged force is zero ($\nabla_s E(x, s_F^*; \theta_{old}) + \beta\nabla_s C(s_F^*, y) = 0$). Minimizing $\mathcal{J}(\theta)$ forces $\nabla_s E(x, s_F^*; \theta) \to 0$, which turns the target-nudged state s_F^* directly into a stable stationary point of the un-nudged free energy E.

Practical Implementation

To update θ, we take the gradient of $\mathcal{J}(\theta)$ with respect to θ:

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{s\sim\mathcal{L}_F^{old}}\left[\left(\nabla_s E(x, s; \theta) - \nabla_s F(x, s, y; \theta_{old})\right)^\top \cdot \nabla_\theta \nabla_s E(x, s; \theta)\right]$$

A Note on Convergence

Under mild regularity conditions (smoothness and shared support), minimizing the Fisher divergence forces $q(s)$ to approach $p(s)$, and $D_F(p \parallel q) = 0$ guarantees that $p(s) = q(s)$ almost

everywhere.

Driving the free distributions $\mathcal{L}_E^\theta$ close to the nudge distribution $\mathcal{L}_F^{old}$ by minimizing the Fisher divergence is equivalent to driving the cost C to 0. Thus this approach is still compatible with the whole philosophy of classical EP.

Indeed

$$\mathcal{L}_E^\theta(s \mid x) = \frac{e^{-\frac{1}{T}E(x,s;\theta)}}{Z_E(\theta)}$$

$$\mathcal{L}_F^\theta(s \mid x, y) = \frac{e^{-\frac{1}{T}E(x,s;\theta)-\beta\frac{1}{T}C(s,y)}}{Z_F(\theta)}$$

The ratio between the two distributions is governed explicitly by the cost function $C(s, y)$:

$$\frac{\mathcal{L}_F^\theta(s \mid x, y)}{\mathcal{L}_E^\theta(s \mid x)} = \frac{Z_E(\theta)}{Z_F(\theta)} e^{-\frac{1}{T}\beta C(s,y)}$$

Notice that $\mathcal{L}_F^\theta$ and $\mathcal{L}_E^\theta$ can only be identical distributions if $\beta C(s, y)$ is constant with respect to s—which requires $C(s, y) = 0$ (zero prediction error).

Remark 6.1.1 (Single phase predictive EP)**.** *Contrary to the proposal of Luczak, McNaughton, and Kubo [8] which is based on a single phase EP like rule, but requires a predictive phase, the Fisher based single phase EP doesn't require a predictive step, or training a predictor, it's a true single phase EP like approach.*

Remark about considering the distributions divergence as the objective function

Considering an objective function as the KL or Fisher divergence between the augmented distribution and the free distribution, would make a learning rule based on gradient descent of the KL or Fisher divergence, thus updating the parameters such that the two distributions becomes similar; which is equivalent of driving the cost to 0 ; because the augmented distribution differs from the free distribution only by the term βC; thus making them equal requires $C = 0$

This remark is valid only when the parameters in the nudge are fixed/frozen, thus the nudge distribution is considered as a target distribution similarly in RL. If the parameters are not fixed in the nudge: the expected cost in the nudge distribution has to be added in the objective function like proposed by Litman [15].

6.1.4 Deterministic single phase static EP

The single phase Fisher EP learning rule yields the following gradient rule :

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{s\sim\mathcal{L}_F^{old}}\left[\Big(\nabla_s E(x, s; \theta) - \nabla_s F(x, s, y; \theta_{old})\Big)^\top \cdot \nabla_\theta \nabla_s E(x, s; \theta)\right]$$

Important remark

This gradient rule is the only single phase rule which is based only on an expectation, and not covariances (unlike single nudge phase Fisher with nudge parametrized (see Chapter 3), and single free phase considering only the expected cost (Massar and Mognetti [30]))

Because this gradient rule is based solely on an expectation, we consider a deterministic limit where the variation of the random variance is taken to 0 (or in this particular case, we consider T going to 0), thus we consider a gradient update for static deterministic EP, using only the relaxed nudge state s^*_β :

$$\nabla_\theta \mathcal{J}(\theta) = \Big(\nabla_s E(x, s^*_\beta; \theta) - \nabla_s F(x, s^*_\beta, y; \theta_{old})\Big)^\top \cdot \nabla_\theta \nabla_s E(x, s^*_\beta; \theta)$$

Thus at each parameter update, only the nudge phase is relaxed, and the relaxed nudge state is used to compute the gradient of the cost. For now this proposal is still a heuristic, and we consider the conjecture that the optimization of the nudge state may be related to the optimization of the free state. Furthermore an approach to consider a dynamical nudge parameter which goes to 0 as learning is made can be explored.

6.1.5 Deterministic single phase LEP with Large error signal and no boundary residuals for any Lagrangian

We consider an extension of the gradient formula for the dynamical states, by considering only the relaxed nudge trajectory $s^*_\beta = (s^*_{t,\beta})t$

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) = \Bigg(&\int_0^T \nabla_s L(s^*_{t,\beta}, \dot{s}^*_{t,\beta}, \theta)\, dt - \int_0^T \nabla_s L(s^*_{t,\beta}, \dot{s}^*_{t,\beta}, \theta_{\text{old}})\, dt \\ &- \beta \nabla_s \int_0^T c(s^*_{t,\beta})\, dt\Bigg)^\top \cdot \int_0^T \nabla_\theta \nabla_s L(s^*_{t,\beta}, \dot{s}^*_{t,\beta}, \theta)\, dt \end{aligned} \tag{6.1}$$

Among the Limitations of the LEP PFVP mentioned in the original paper of Pourcel et al. [4] are :

1) The requirements of an additional echo phase (which makes the algorithm offline)

2) The requirement of a reversible Lagrangian (limiting the scope of application)

This proposal would aim to solve this 2 limitations simultaneously, by avoiding boundary residuals (thus the requirement of reversible Lagrangian), and performing only one phase (making it on-line). Furthermore, this approach is limitless, like Litman's EP framework [15].

Nevertheless, this proposal is still a heuristic method, and further work to make the approach more rigorous is required.

Impact for some 1st order dynamical systems This method would be suited even for some 1st order dynamical systems, by considering the Lagrangian

$$L(x, \dot{x}, u) = A(x)^\top \dot{x} + V(x, u)$$

which would lead to the 1st order dynamical systems (by applying the Euler Lagrange equation)

$$\dot{x} = f(x, u)$$

The requirement for this approach is to have

$$f(x,u) = J(x)^{-1}\nabla_x V(x,u)$$

with

$$J(x) = \frac{\partial A}{\partial x} - \left(\frac{\partial A}{\partial x}\right)^{\top}$$

6.1.6 Numerical experimentation

We performed a simple numerical experiment with a simulated example by comparing BPTT, LEP CIVP with the cheap bulk term only (avoiding the expensive boundary residuals which are against EP's philosophy)), and also our proposal of heuristic.

When considering a short horizon of the bulk LEP CIVP algorithm stays close to BPTT, and the heuristic method looks to converge faster to a higher cost (still close to the value of BPTT) as illustrated in Figure 6.1, but when considering long horizon, the LEP CIVP method becomes inefficient to reduce cost due to the negligence of the expensive boundary residuals, yet the heuristic method still performs close to BPTT, while offering computational efficiency close to cheap LEP (faster than cheap bulk LEP CIVP). In Figure 6.1, we considered 10 random initializations for the algorithms, and plotted the mean loss curve. We considered a small supervised regression problem, with the following Lagrangian :

$$L(s,\dot{s},\theta) = w(t)\left[\tfrac{1}{2}\|\dot{s}\|^2 - V(x,s;\theta)\right]$$

where the potential is given by :

$$V(x,s;\theta) = \tfrac{1}{2}\|h\|^2 + \tfrac{1}{2}\|y\|^2 \;-\; h\cdot(W_1 x + b_1) \;-\; y\cdot\big(\tanh(h)W_2^{\top} + b_2\big)$$

and we aim to optimize the cost :

$$C(s, y_{\text{target}}) = \tfrac{1}{2}\|y - y_{\text{target}}\|^2$$

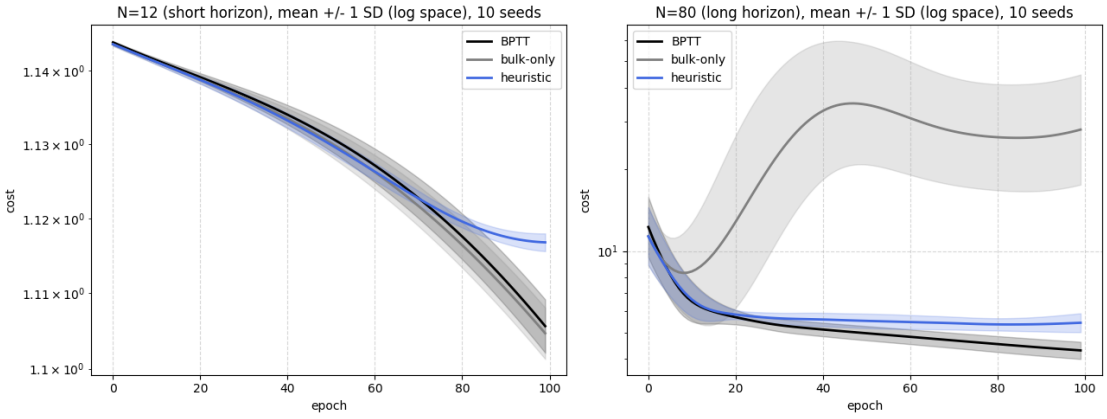


Figure 6.1: Temporal cost evolution comparaison of heuristic, BPTT and bulk CIVP .

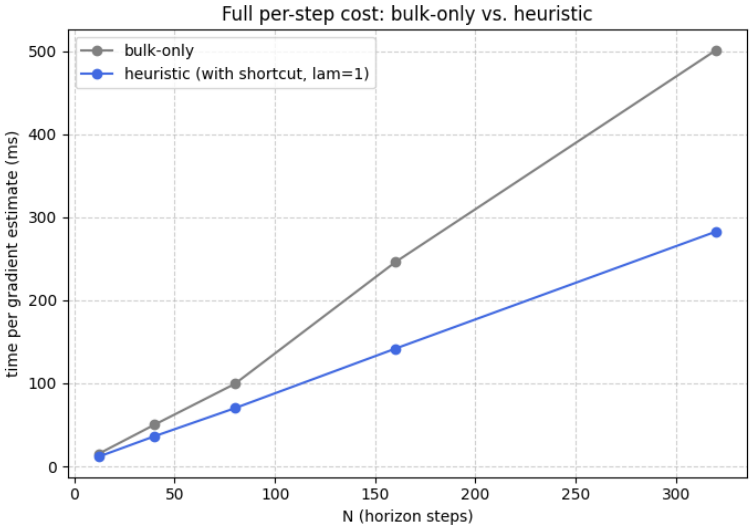


Figure 6.2: Time complexity comparison of heuristic and bulk CIVP .

As shown in Figure 6.2 the heuristic proposal is about 1.6-1.9 faster in gradient computation when compared to bulk CIVP. This is due to the single phase consideration, and also the fact that the heuristic method computes a "cheap" hessian like term : $\nabla^2_{\theta,s} L$, instead of a simple "cheaper" gradient like term of bulk CIVP : $\nabla_\theta L$. Nevertheless both terms, are cheap, thus the heuristic method is faster.

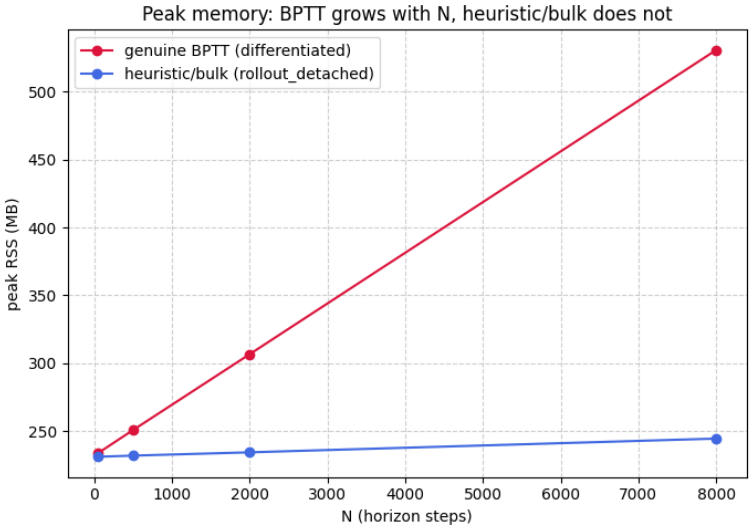


Figure 6.3: Memory comparaison of heuristic, BPTT and bulk CIVP .

The heuristic method, also avoids BPTT memory, and stays roughly constant in memory requirement as illustrated in Figure 6.3

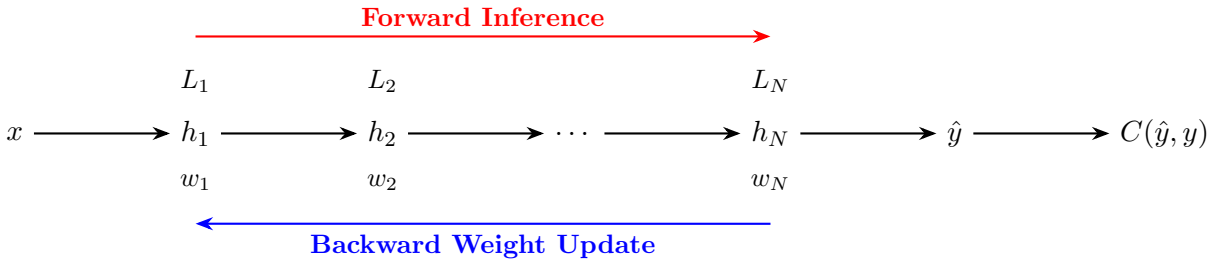
The notebook code (google colab) is available at the following link :

https://colab.research.google.com/drive/1vi7x0jP8H1DmJDwSAkeR2NQAI3A5bYxW?usp=sharing

6.2 DL Architecture: Classical Paradigm and Forward Only Proposal

Classical MLP: A standard Multi-Layer Perceptron (MLP) utilizes a *Forward Propagation* phase to generate the network prediction (output) from a given input. Subsequently, it relies on a *Backward Propagation* phase to compute the gradient of the objective cost function with respect to the trainable parameters (weights and biases) of each individual layer. Each layer possesses its own distinct set of parameters/weights.

Classical Architecture Scheme

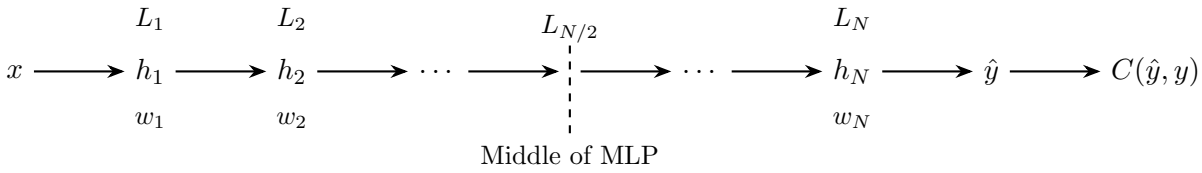


6.2.1 Proposed Approach

Proposal for Forward-Only, Single-Phase, Simultaneous Inference and Learning

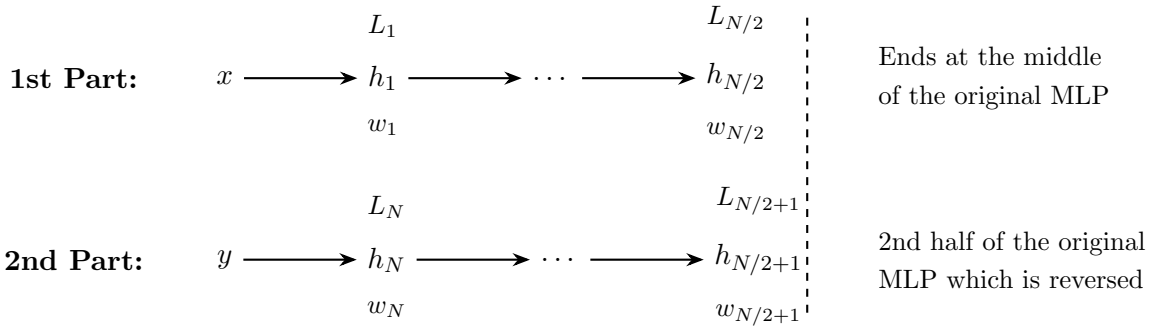
In this proposed framework, we structurally divide the standard MLP architecture by two (2). Crucially, we mathematically and structurally invert the second half of the network.

Original MLP Layout



Divided Architecture Formulation

By decomposing the system, we establish two parallel forward operations occurring concurrently:



We thus consider exactly two Forward Propagations using the two halves of the original MLP. For the 2nd part, we invert it, and we consider the target output 'y' as a pseudo-"input".

Layer Pairing and Weight Sharing

In this architecture, we consider paired layers. They correspond to the layers of the "sons" MLPs which have the same position in the propagation chain. Thus, the pairs are explicitly defined as:

(L_1, L_N) ; (L_2, L_{N-1}) ; (L_3, L_{N-2}) ...

until the last pair at the middle of the original MLP: $(L_{N/2}, L_{N/2+1})$

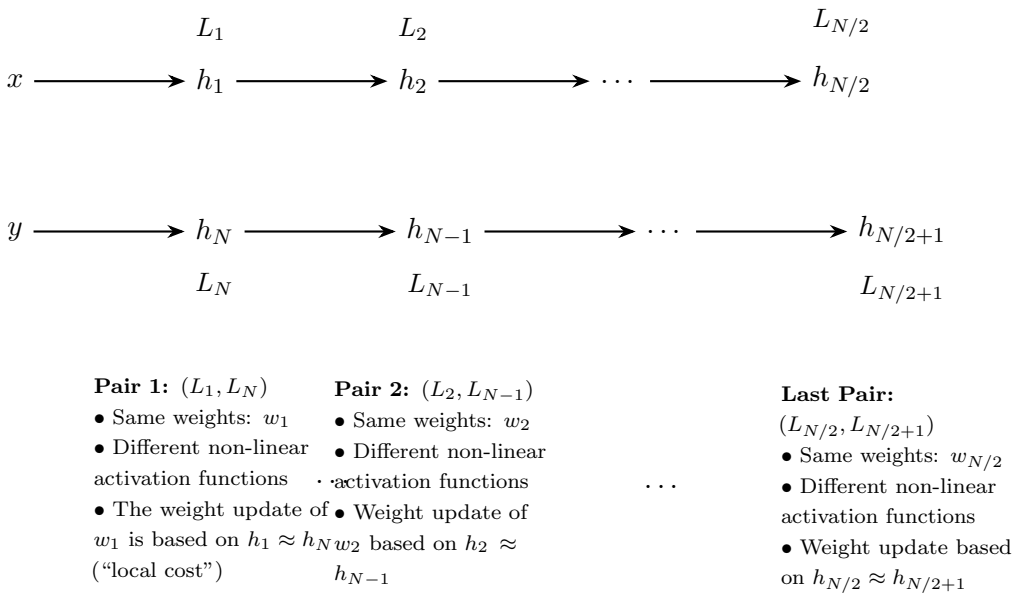
The paired layers share the same linear learnable weights. Thus, we enforce:

$$w_1 = w_N \; ; \; w_2 = w_{N-1} \; ; \; w_3 = w_{N-2} \; \ldots \; w_{N/2} = w_{N/2+1} \tag{6.2}$$

Crucial Requirement: Although the paired layers share identical linear weights, they must possess different non-linear fixed activation functions.

Architectural Philosophy and State Correlation

The core philosophy of this approach is to construct a sequence of hidden states for each "sub" MLP such that corresponding hidden states become increasingly correlated as the forward propagation is performed.



As a consequence of this progressive alignment mechanism along the layer hierarchy, the structural similarity satisfies the following correlation inequality sequence:

$$\text{corr}(h_1, h_N) < \text{corr}(h_2, h_{N-1}) < \cdots < \text{corr}(h_{N/2}, h_{N/2+1}) \tag{6.3}$$

Dynamic Optimization Formulation

A potential implementation approach would be to consider a dynamic cost function. This objective function would be designed to be structurally "relaxed" during the initial stages of the forward propagation phase, and would eventually converge to equal the true target cost function at the terminal end boundary—precisely when the final two inner hidden states ($h_{N/2}$ and $h_{N/2+1}$) have achieved their highest degree of correlation.

Local Learning Rules and Simultaneous Weight Updates

The core objective of this architecture is to enable simultaneous inference and weight optimization ("learning") via a strictly local learning rule. Under this paradigm:

- The first weight set w_1 is updated by evaluating a localized (and potentially relaxed) cost function:

$$C_1(h_1, h_N) \tag{6.4}$$

- The second weight set w_2 is updated concurrently using:

$$C_2(h_2, h_{N-1}) \tag{6.5}$$

- This pattern proceeds layer by layer until the median boundary is reached, where $w_{N/2}$ is updated using the true objective cost function:

$$C_{N/2}\left(h_{N/2}, h_{N/2+1}\right) = C\left(h_{N/2}, h_{N/2+1}\right) \tag{6.6}$$

Network Reconstruction and Bijectivity

Once the learning phase is finalized (or during standard inference/evaluation), the complete, unrolled Multi-Layer Perceptron can be fully reconstructed. This is achieved by structurally inverting the second "son"-MLP—which originally functioned as the inverted half of the network—and seamlessly merging it with the first "son"-MLP.

To mathematically guarantee that the second sub-network can be inverted back to a standard forward flow, a strict bijectivity condition must be satisfied by its layer transformations.

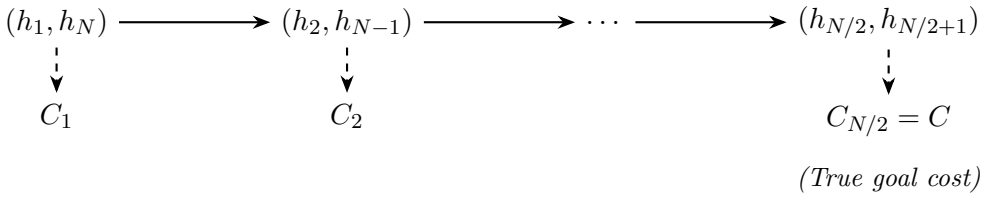
Relaxation of Weight Sharing Constraints

Crucially, the explicit condition requiring identical weight parameters between paired layers ($w_i = w_{N-i+1}$) can be relaxed. The architecture remains viable if all layers retain entirely independent parameters. For example, any given paired layer set (L_1, L_N) can be successfully optimized even when:

$$w_1 \neq w_N \tag{6.7}$$

6.2.2 The Cheat Cost Concept

The fundamental requirement of this optimization framework relies on the introduction of the **Cheat Cost Concept**. The purpose of this formulation is to gradually and more accurately evaluate the hidden state pairs generated across the dual sub-networks. This progression can be mapped as follows:



Within this cheat cost framework, we utilize distinct local cost functions engineered with specific mathematical landscapes. These intermediate objectives are designed to "cheat" by allowing lower penalizations, accelerating convergence to 0, or being structurally easier to minimize compared to the global goal.

As forward propagation deepens toward the center of the network, this relaxed optimization property systematically weakens until arriving precisely at the terminal true objective function:

$$C_{N/2} = C \tag{6.8}$$

Thus, for any layer indices such that $i < j$, the cost relationship satisfies:

$$C_i \prec C_j \tag{6.9}$$

meaning that $C_i \to 0$ faster than C_j, or that C_i presents an optimization landscape that is inherently easier to minimize than C_j.

Regarding the bijectivity constraint

An application of this approach using KANs (see Liu et al. [42])instead of NNs could potentially allow better numerical stability (inverting bijective functions instead of matrix inversion requirement).

The concept of bijectivity for NNs has also been explored (e.g **Conditional Invertible Neural Networks (cINNs)** see Ardizzone et al. [43])

Approaches to bijectivity : "Bijectivization" via Latent Augmentation (Lifting)

If $f \circ g$ loses information, we can transform the problem by adding an auxiliary (latent) variable z that captures the lost information. Instead of trying to invert $x \mapsto y$, we construct a map:

$$T : x \mapsto (y, z)$$

Where: $y = f(g(x))$ is the observed measurement. $z = h(x)$ is a complementary transform designed so that T is bijective. Once T is bijective, an exact inverse $T^{-1}(y, z) = x$ exists.

Modern Application: This is the core principle behind Conditional Invertible Neural Networks (cINNs) and Normalizing Flows ([44], [45]). If z is modeled as a standard Gaussian distribution, we can sample different values of z for a fixed y to systematically generate all plausible inputs x that could have produced y.

6.2.3 Approach without bijectivity constraint

We divide the MLP into multiple single layer perceptrons:

$$\begin{aligned} x &\to h_1 \\ h_1 &\to h_2 \\ &\vdots \\ h_{N-1} &\to h_N \\ h_N &\to y \end{aligned}$$

At the start we consider the 1st paired layers:

$$\begin{aligned} x &\to h_1 \\ h_N &\to y \end{aligned}$$

In this approach we have the usual parameters of a layer which are used for inference; for example: W_1 is used to compute h_1 from x; and we consider a novel concept of pseudo parameters, which will enable us to delete the bijectivity requirement.

So for the first paired layers: we would be able to compute h_1 from x; but we can't compute h_N from y because of the absence of bijectivity; Thus we use the pseudo parameters to compute h'_N from h_1; and we would compute an estimation of y from this pseudo h'_N; and then with this estimation of y, we would make a local gradient update rule to update both of the parameters and the pseudo parameters, thus:

We have:

$$x \xrightarrow{W_1} h_1 \xrightarrow{W'_{1,N}} h'_N \approx h_N \xrightarrow{W_N} y' \approx y$$

and we use $C_1(y', y)$ to update W_1, W_N and $W'_{1,N}$.

Then we continue with the next paired layers:

$$h_1 \xrightarrow{W_2} h_2 \xrightarrow{W'_{2,N-1}} h'_{N-1} \approx h_{N-1} \xrightarrow{W_{N-1}} h_N$$

and we keep the concept of cheat cost

$$C_1 < C_2 < \cdots < C_{N/2}$$

Remark about the pseudo parameters

The number of pseudo parameters (weights and bias) represents half the number of parameters, because each pseudo parameter is linked to 2 single layers of the network.

They could seem to an objection to biological plausibility because they appear similar to considering a secondary network, but they have a direct impact of information link between the hidden states of the network, thus despite the fact that won't be use for direct inference and are required only for learning, they enable a different information flow, which could enhance training and may not be biologically implausible.

Computational improvement

An approach to delete some of the intermediate hidden layers, and using directly the pseudo parameters, could be considered. This should be implemented after the network achieved a first phase of learning where all the parameters converged to their optimal values with a first big MLP architecture. Then the Big MLP architecture would be transformed into a more efficient smaller MLP. This may allow first learning than starting directly with a small MLP.

Unification of Inference and Learning via Pseudo Predictions

For the approach without the bijectivity constraint, we could use the pseudo generation in inference for further unification of inference and learning.

Multilevel Predictions

$$x \xrightarrow{W_1} h_1 \xrightarrow{W'_{1,N}} h'_N \xrightarrow{W_N} \hat{y}_1 \quad : \text{1st fast prediction}$$

$$x \xrightarrow{W_1} h_1 \xrightarrow{W_2} h_2 \xrightarrow{W'_{2,N}} h'_{N-1} \xrightarrow{W_{N-1}} h''_N \xrightarrow{W_N} \hat{y}_2 \quad : \text{2nd prediction}$$

$$\vdots$$

Until the true prediction (requires the whole length of the network):

$$x \xrightarrow{W_1} h_1 \xrightarrow{W_2} h_2 \longrightarrow \cdots \longrightarrow h_{N-1} \xrightarrow{W_{N-1}} h_N \xrightarrow{W_N} \hat{y}$$

So we would get multilevel predictions:

$$\hat{y}_1 \,;\, \hat{y}_2 \,;\, \ldots \,;\, \hat{y}_{N-1} \,;\, \hat{y}$$

such that we have an order relation of precision (relates to amortized concept):

$$\underset{\text{“less accurate prediction”}}{\hat{y}_1} < \hat{y}_2 < \ldots < \hat{y}_{N-1} < \underset{\text{“most accurate prediction”}}{\hat{y}}$$

Utility of Intermediate Predictions $(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_{N-1})$

We could consider extra learnable pseudo learnable parameters that could be used to obtain more accurate predictions; this would result in a more elaborate network, where we add multiple single layers:

$$\begin{aligned} \hat{y}_1 &\xrightarrow{W_1''} \hat{y}_2 \\ \hat{y}_2 &\xrightarrow{W_2''} \hat{y}_3 \\ &\vdots \\ \hat{y}_{N-1} &\xrightarrow{W_{N-1}''} \hat{y} \end{aligned}$$

Core Philosophy

The main philosophy is to unify inference and learning, thus they should have the same parameters that play a significant role in both.

Chapter 7

Discussions, Limitations, and Future Works

In this report we have achieved contributions to EP algorithm for ML applications.
The topics considered where :
1) Application of LEP to GNNs, GCNs, and also GATs
2) Considering large error signals for EP and LEP
3) Contributions to stochastic EP and LEP to handle noisy data.
4) application of EP and LEP in RL and control.
5) single phase EP
We describe some limitations, identified, and potential directions for future work.

Numerical Experiments of the proposed methods

Numerical experiments are a priority for future work, to confirm the advantage of the proposed novel EP architectures, for example Tsallis EP, shall enable the applicability of EP for some noisy data settings where the standard Gibbs Boltzmann based stochastic EP would perform badly, and also Fisher EP, shall enable a faster learning due to the use of only a single phase, and as a last example, Fourier EP, should enable large error signals which would enhance the performance of EP, and enable a scalable application in ML.

Implementation of LEP PFVP for GNN-based approaches for time series analysis

The implementation we considered for GNNs were based on architecture where dynamical Graph structures, represented an extension of Neural ODEs [24] to graph data. Thus the dynamics of the graph represents the hidden layer of a Neural Network, and ultimately the architectures considered where developed only for static input and output data. There are other proposals of Graph Neural Nets which are designed for time series data (for example the proposal of Jin et al. [46]), nevertheless theses proposals where based on 1st order differential systems, which are not Lagrangian systems, and thus incompatible with LEP PFVP. Designing GNNs for time series data based on 2nd order Lagrangian systems, would enable the applicability of LEP PFVP, and achieve cheaper computation and learning for theses tasks.

Comparison between the proposal for RL actor critic with the proposal of Kubo, Chalmers, and Luczak [17] In the EP proposal for actor critic RL by Kubo, Chalmers, and Luczak [17], they mentioned that they tried an all EP method to train both actor and critic, nevertheless, the performance of the approach was not good, thus because our proposed architecture (we consider a Q-learning approach to evaluate both state and action, and also Litman's limitless EP framework) is different, it is important to test the performance and see the comparison with the results of Kubo, Chalmers, and Luczak [17].

Theoretical developments of the heuristic proposals

For the heuristic Fisher EP single phase proposal (different from the stochastic EP), a theoretical investigation as well as further experimental investigation is required. This method has a potential to offer an improved framework for LEP (limitless, single phase, any Lagrangian), thus despite the lack of theoretical rigor, it is worth pursuing.

Extension of LEP PFVP for cost that evaluates velocity (for RL closed loop applications)

For applications of LEP PFVP to closed loop control, it would be advantageous to extend the applicability of LEP PFVP to cost functions which evaluate both position and velocity (similarly to RHEL which is suited for cost functions that evaluate both position and momentum), thus it would be possible to consider cost functions related to control that evaluates both position and velocity, when the free-control dynamics is a 2nd order dynamics. From an RL perspective, we would be able to evaluate the pair of states and actions (Q-learning style) instead of states only.

Rigorous demonstration of equivalence between LEP PFVP and HEB

For the application of dissipative LEP PFVP to Graph CON, we used directly the fact that LEP PFVP can be used to supervised learning setting where the cost functions only evaluates the last state, because of the equivalence of LEP PFVP with RHEL, and the fact that RHEL can be applied in this setting, and the method gave correct performance close to BPTT. Nevertheless, it would be more rigorous to show this capacity theoretically.

Theoretical development of the novel heuristic DL architecture

The novel heuristic DL architecture requires theoretical development and comparison of the DL algorithms which are based on froward only philosophy such as the framework of Bengio [9] and Lee et al. [10]. Furthermore, numerical experiments is required.

Theoretical comparison between stochastic score gradient and EP

As shown by Massar and Mognetti [30], EP is related to the score gradient estimator in stochastic framework, which is obtained by the log derivative trick. Further investigation on comparing EP with this method to highlight advantages of one method over the other is required.

Consideration of Litman's framework in continuous temporal framework

A potential future work, would be to extend Litman's framework in dynamical case, with continuous time (instead of discrete time as we considered in Chapter 3). We could consider an approach with Path-Space KL Divergence (see Liptser and Shiryaev [47]).

Appendix A

Some proofs of Chapter 4

A.1 Alternative Derivation of the EP Learning Rule (without the Helmholtz free energy for the Gibbs-Boltzmann distribution) (see Chapter 4)

A.1.1 Proof of the EP formula derivation from Chapter 4

Instead of considering the Helmholtz free energy for the Gibbs-Boltzmann distribution as considered by Scellier [14], we consider the expected energy, as the main object

$$\mathbb{E}_{\rho_\beta(\theta)}(F)$$

Then following the same argument as above, we apply the Schwarz commutation formula for partial derivatives to derive the stochastic EP gradient formula, and recover the results of Scellier [14], thus we consider

$$\nabla_\theta \nabla_\beta \mathbb{E}_{\rho_\beta(\theta)}(F) = \nabla_\beta \nabla_\theta \mathbb{E}_{\rho_\beta(\theta)}(F) \quad \text{with } F = E + \beta C \quad (\text{we have } \nabla^2_{\theta\beta} F = 0) \tag{A.1}$$

Using the identity: (see Chapter 3)

$$\nabla_\beta \mathbb{E}(F) = \mathbb{E}(\partial_\beta F) - \frac{1}{T}\operatorname{Cov}(F, \partial_\beta F) \tag{A.2}$$

and similarly:

$$\nabla_\theta \mathbb{E}(F) = \mathbb{E}(\nabla_\theta F) - \frac{1}{T}\operatorname{Cov}(F, \nabla_\theta F) \tag{A.3}$$

Then:

$$\partial_\beta \mathbb{E}(\nabla_\theta F) = \partial_\beta \mathbb{E}(\nabla_\theta F) + \frac{1}{T}\left(\underset{\rho_\beta(\theta)}{\partial_\beta} \operatorname{Cov}(F, \nabla_\theta F) - \underset{\rho_\beta(\theta)}{\nabla_\theta} \operatorname{Cov}(F, \partial_\beta F) \right) \tag{A.4}$$

Thus to show the EP gradient rule :

$$\underset{\rho_\beta(\theta)}{\nabla_\theta}\, \mathbb{E}(C) = \underset{\rho_\beta(\theta)}{\partial_\beta}\, \mathbb{E}(\nabla_\theta E) \tag{A.5}$$

We need to show:

$$\underset{\rho_\beta(\theta)}{\nabla_\theta} \operatorname{Cov}(F, \partial_\beta F) - \underset{\rho_\beta(\theta)}{\partial_\beta} \operatorname{Cov}(F, \nabla_\theta F) = 0 \tag{A.6}$$

Expansion of terms:

$$\begin{aligned}
\nabla_\theta \operatorname{Cov}(F, \partial_\beta F) &= \nabla_\theta[\mathbb{E}(F\partial_\beta F) - \mathbb{E}(F)\mathbb{E}(\partial_\beta F)] \\
&= \nabla_\theta \mathbb{E}(F\partial_\beta F) - \nabla_\theta(\mathbb{E}(F))\mathbb{E}(\partial_\beta F) - \mathbb{E}(F)\nabla_\theta \mathbb{E}(\partial_\beta F) \\
&= \left[\mathbb{E}(\nabla_\theta F \partial_\beta F) - \frac{1}{T}\operatorname{Cov}(\nabla_\theta F, F\partial_\beta F)\right] \\
&\quad - \left[\mathbb{E}(\nabla_\theta F) - \frac{1}{T}\operatorname{Cov}(F, \nabla_\theta F)\right]\mathbb{E}(\partial_\beta F) \\
&\quad - \mathbb{E}(F)\left(-\frac{1}{T}\operatorname{Cov}(\nabla_\theta F, \partial_\beta F)\right)
\end{aligned} \tag{A.7}$$

Similarly:

$$\begin{aligned}
\partial_\beta \operatorname{Cov}(F, \nabla_\theta F) &= \left[\mathbb{E}(\partial_\beta F \nabla_\theta F) - \frac{1}{T}\operatorname{Cov}(\partial_\beta F, F\nabla_\theta F)\right] \\
&\quad - \left[\mathbb{E}(\partial_\beta F) - \frac{1}{T}\operatorname{Cov}(F, \partial_\beta F)\right]\mathbb{E}(\nabla_\theta F) \\
&\quad - \mathbb{E}(F)\left(-\frac{1}{T}\operatorname{Cov}(\partial_\beta F, \nabla_\theta F)\right)
\end{aligned} \tag{A.8}$$

All the symmetric terms in (θ, β) vanish in the difference.
The terms that remain are:

$$\begin{aligned}
&- \operatorname{Cov}(\nabla_\theta F, F\nabla_\beta F) + \mathbb{E}(\nabla_\beta F)\operatorname{Cov}(F, \nabla_\theta F) \\
&+ \operatorname{Cov}(\nabla_\beta F, F\nabla_\theta F) - \mathbb{E}(\nabla_\theta F)\operatorname{Cov}(F, \nabla_\beta F) \\
&= -\mathbb{E}(\nabla_\theta F\, F\nabla_\beta F) + \mathbb{E}(\nabla_\theta F)\mathbb{E}(F\nabla_\beta F) \\
&\quad + \mathbb{E}(\nabla_\beta F)\left(\mathbb{E}(F\nabla_\theta F) - \mathbb{E}(F)\mathbb{E}(\nabla_\theta F)\right) \\
&\quad + \left(\mathbb{E}(\nabla_\beta F\, F\nabla_\theta F) - \mathbb{E}(\nabla_\beta F)\mathbb{E}(F\nabla_\theta F)\right) \\
&\quad - \mathbb{E}(\nabla_\theta F)\left(\mathbb{E}(F\nabla_\beta F) - \mathbb{E}(F)\mathbb{E}(\nabla_\beta F)\right) = 0
\end{aligned} \tag{A.9}$$

Which shows the cancellation of Covariance terms.

A.2 Approach for dynamical stochastic EP with discrete time (see Chapter 4)

A.2.1 Modified Approach for stochastic damped Hamiltonian IVP EP

A.2.2 Proof of the EP formula

Estimator for the expected path cost

In this part, we consider the log derivative trick, combined with an EP like construction, inspired by the previous remarks for stochastic EP, to promote EP such that it can be used for this dynamical noisy data framework.

Suppose we want

$$\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N} C(q_k, p_k)\right].$$

We start by considering the following total quantity :

$$A_\theta^\beta = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k)\right]$$

Schwarz formula give

$$\frac{\partial}{\partial_\beta}\nabla_\theta A_\theta^\beta = \nabla_\theta \frac{\partial}{\partial_\beta} A_\theta^\beta$$

Again using :

$$\nabla_\theta \sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k) = \sum_{k=0}^{N} \nabla_\theta H(q_k, p_k, \theta)$$

and

$$\frac{\partial}{\partial_\beta} \sum_{k=0}^{N} H(q_k, p_k, \theta) + \beta \sum_{k=0}^{N} C(q_k, p_k) = \sum_{k=0}^{N} C(q_k, p_k, \theta)$$

Furthermore, we consider the previous formula of score function estimation for the parameters θ

$\int \Phi(x_{0:N})\, \nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N} = \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\, \nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0)\right] =$
$\mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\left(\sum_{i=0}^{N-1} \nabla_\theta \log p_\theta(x_{i+1} \mid x_i)\right)\right]$
$= \mathbb{E}_{p_\theta}\left[\Phi(x_{0:N}) \sum_{i=0}^{N-1} \left(\nabla_\theta \mu_i^\beta(x_i, \theta)\right)^\top \Sigma^{-1}\left(x_{i+1} - \mu_i^\beta(x_i, \theta)\right)\right]$
As well as the nudge parameter β
$\int \Phi(x_{0:N})\, \frac{\partial}{\partial_\beta} p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N} = \mathbb{E}_{p_\theta^\beta}\left[\Phi(x_{0:N})\, \frac{\partial}{\partial_\beta} \log p_\theta^\beta(x_{1:N} \mid x_0)\right] =$
$\mathbb{E}_{p_\theta}\left[\Phi(x_{0:N})\left(\sum_{i=0}^{N-1} \frac{\partial}{\partial_\beta} \log p_\theta(x_{i+1} \mid x_i)\right)\right]$
$= \mathbb{E}_{p_\theta}\left[\Phi(x_{0:N}) \sum_{i=0}^{N-1} \left(\frac{\partial}{\partial_\beta} \mu_i^\beta(x_i, \theta)\right)^\top \Sigma^{-1}\left(x_{i+1} - \mu_i^\beta(x_i, \theta)\right)\right]$
Thus by developing the Schwarz formula
$\frac{\partial}{\partial_\beta}\nabla_\theta A_\theta^\beta = \nabla_\theta \frac{\partial}{\partial_\beta} A_\theta^\beta$

we obtain
$\nabla_\theta \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N C(q_k,p_k)\right] = \partial_\beta \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N H(q_k,p_k,\theta)\right] +$
$\partial_\beta \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$
$-\nabla_\theta \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)\right]$
Or

$\nabla_\theta \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)\right] =$
$\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0) + \sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot\nabla_\theta \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)$

$+\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$

Thus we obtain the final result of the EP like gradient estimator :
$\nabla_\theta \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N C(q_k,p_k)\right] = \partial_\beta \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N H(q_k,p_k,\theta)\right]$
$+\partial_\beta \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$
$-\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0) + \sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot\nabla_\theta \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)$

$-\mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$

Because the objective is $\mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^N C(q_k,p_k)\right]$
this becomes
$\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^N C(q_k,p_k)\right] = \partial_\beta|_{\beta=0} \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right]$
$+\partial_\beta|_{\beta=0} \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$
$-\mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0) + \sum_{k=0}^N H(q_k,p_k,\theta)\right)\cdot\nabla_\theta \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0)$

$-\mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta \log p_\theta^0(x_{1:N}\mid x_0)\right]$

Or $\nabla_\theta \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0) = 0$ thus
$\mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0) + \sum_{k=0}^N H(q_k,p_k,\theta)\right)\cdot\nabla_\theta \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0) = 0$
Thus
$\nabla_\theta \mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^N C(q_k,p_k)\right] = \partial_\beta|_{\beta=0} \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta)\right]$
$+\partial_\beta|_{\beta=0} \mathbb{E}_{p_\theta^\beta}\left[\left(\sum_{k=0}^N \nabla_\theta H(q_k,p_k,\theta) + \beta C(q_k,p_k)\right)\cdot \nabla_\theta \log p_\theta^\beta(x_{1:N}\mid x_0)\right]$
$-\mathbb{E}_{p_\theta^0}\left[\left(\sum_{k=0}^N H(q_k,p_k,\theta)\right)\cdot \frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N}\mid x_0)\cdot\nabla_\theta \log p_\theta^0(x_{1:N}\mid x_0)\right]$

Remark

$$\frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{i+1}\mid x_i) = \left(\frac{\partial}{\partial_\beta}\mu_i^\beta(x_i,\theta)\right)^\top \Sigma^{-1}\left(x_{i+1} - \mu_i^\beta(x_i,\theta)\right)$$

Where

$$\frac{\partial}{\partial_\beta}\mu_i^\beta(q_i,p_i) = \begin{pmatrix} \nabla_p C(q_i,p_i)\,\Delta t \\ -\nabla_q C(q_i,p_i)\,\Delta t - \gamma\nabla_p C(q_i,p_i)\,\Delta t \end{pmatrix},$$

because

$$\mu_i^\beta(q_i,p_i) = \begin{pmatrix} q_i + \nabla_p H(q_i,p_i)\,\Delta t + \beta\nabla_p C(q_i,p_i)\,\Delta t \\ p_i - \nabla_q H(q_i,p_i)\,\Delta t - \beta\nabla_q C(q_i,p_i)\,\Delta t - \gamma\nabla_p H(q_i,p_i)\,\Delta t - \beta\gamma\nabla_p C(q_i,p_i)\,\Delta t \end{pmatrix},$$

Or we have

$$\log p_\theta^\beta(x_{1:N} \mid x_0) = \sum_{i=0}^{N-1} \log p_\theta^\beta(x_{i+1} \mid x_i)$$

Thus
$\frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{1:N} \mid x_0) = \sum_{i=0}^{N-1}\frac{\partial}{\partial_\beta}\log p_\theta^\beta(x_{i+1} \mid x_i) =$
$\sum_{i=0}^{N-1}\big(\frac{\partial}{\partial_\beta}\mu_i^\beta(x_i,\theta)\big)^\top\Sigma^{-1}\big(x_{i+1}-\mu_i^\beta(x_i,\theta)\big)$
Furthermore, we have

$$\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0) = \sum_{i=0}^{N-1} \nabla_\theta \log p_\theta^\beta(x_{i+1} \mid x_i)$$

$$= \sum_{i=0}^{N-1} \nabla_\theta \log p_\theta^\beta(x_{i+1} \mid x_i) = \big(\nabla_\theta\mu_i^\beta(x_i,\theta)\big)^\top\Sigma^{-1}\big(x_{i+1}-\mu_i^\beta(x_i,\theta)\big)$$

And

$$\nabla_\theta\mu_i^\beta(q_i,p_i) = \begin{pmatrix} \nabla_\theta\nabla_p H(q_i,p_i,\theta)\,\Delta t \\ -\nabla_\theta\nabla_q H(q_i,p_i,\theta)\,\Delta t - \gamma\nabla_\theta\nabla_p H(q_i,p_i,\theta)\,\Delta t \end{pmatrix},$$

Remark

Unlike in the stochastic EP framework considered with Gibbs Boltzmann by Scellier [14], the EP gradient fromula does'nt yoelds a clean result of the from :

$$\nabla_\theta\mathbb{E}_{p_\theta^0}\left[\sum_{k=0}^{N} C(q_k,p_k)\right] = \partial_\beta|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)\right]$$

Indeed we showed, that there are additional terms :
$+\partial_\beta|_{\beta=0}\mathbb{E}_{p_\theta^\beta}\left[\Big(\sum_{k=0}^{N}\nabla_\theta H(q_k,p_k,\theta)+\beta C(q_k,p_k)\Big)\cdot\nabla_\theta\log p_\theta^\beta(x_{1:N} \mid x_0)\right]$
and
$-\mathbb{E}_{p_\theta^0}\left[\Big(\sum_{k=0}^{N} H(q_k,p_k,\theta)\Big)\cdot\frac{\partial}{\partial_\beta}|_{\beta=0}\log p_\theta^\beta(x_{1:N} \mid x_0)\cdot\nabla_\theta\log p_\theta^0(x_{1:N} \mid x_0)\right]$
Nevertheless, theses terms are easy to compute, and are irrelevant to the applicability of this EP framework to handle dynamical noisy data (unlike the costly boundary residuals terms of LEP CIVP for example).

A.3 About Litman's objective (Limitless HEP) (see Chapter 4)

A.3.1 Proof of the EP-like formula

We consider the following objective function :

$$J(\theta) = \mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^{N} C(q_k, p_k)\right] + \lambda\, \mathrm{KL}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0))$$

$$= \int \sum_{k=0}^{N} C(q_k, p_k) p_\theta^\beta(x_{1:N} \mid x_0)\, dx_{1:N} + \lambda \int p_\theta^\beta(x_{1:N} \mid x_0) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right) dx_{1:N}$$

where λ is a regularization hyperparameter which depends on the noise/variance.

Computation of $\nabla_\theta \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right)$

$\nabla_\theta \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right) = \nabla_\theta\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right)\frac{p_\theta^0(x_{1:N}|x_0)}{p_\theta^\beta(x_{1:N}|x_0)} =$
$\left(\frac{\nabla_\theta p_\theta^\beta(x_{1:N}|x_0) p_\theta^0(x_{1:N}|x_0) - \nabla_\theta p_\theta^0(x_{1:N}|x_0) p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)^2}\right)\frac{p_\theta^0(x_{1:N}|x_0)}{p_\theta^\beta(x_{1:N}|x_0)}$
Thus
$\nabla_\theta \mathrm{KL}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0)) = \nabla_\theta \int p_\theta^\beta(x_{1:N} \mid x_0) \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right) dx_{1:N} =$
$\int \nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right) dx_{1:N} +$
$\int p_\theta^\beta(x_{1:N} \mid x_0) \nabla_\theta \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right) dx_{1:N}$

and separately, use the log derivative trick we have

$$\int \nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right) dx_{1:N} = \int \nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right) p_\theta^\beta(x_{1:N} \mid x_0) dx_1$$

$$= \mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta \log\left(p_\theta^\beta(x_{1:N} \mid x_0)\right) \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right)\right]$$

And we also have
$\int p_\theta^\beta(x_{1:N} \mid x_0) \nabla_\theta \log\left(\frac{p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)}\right) dx_{1:N} =$
$\int p_\theta^\beta(x_{1:N} \mid x_0) \left(\frac{\nabla_\theta p_\theta^\beta(x_{1:N}|x_0) p_\theta^0(x_{1:N}|x_0) - \nabla_\theta p_\theta^0(x_{1:N}|x_0) p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)^2}\right)\frac{p_\theta^0(x_{1:N}|x_0)}{p_\theta^\beta(x_{1:N}|x_0)} dx_{1:N} =$
$\int \left(\frac{\nabla_\theta p_\theta^\beta(x_{1:N}|x_0) p_\theta^0(x_{1:N}|x_0) - \nabla_\theta p_\theta^0(x_{1:N}|x_0) p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)^2}\right) p_\theta^0(x_{1:N} \mid x_0) dx_{1:N} =$
$\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N}|x_0) p_\theta^0(x_{1:N}|x_0) - \nabla_\theta p_\theta^0(x_{1:N}|x_0) p_\theta^\beta(x_{1:N}|x_0)}{p_\theta^0(x_{1:N}|x_0)^2}\right]$
Thus

$$\int p_\theta^\beta(x_{1:N} \mid x_0) \nabla_\theta \log\left(\frac{p_\theta^\beta(x_{1:N} \mid x_0)}{p_\theta^0(x_{1:N} \mid x_0)}\right) dx_{1:N} =$$

$$\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N}\mid x_0)p_\theta^0(x_{1:N}\mid x_0)-\nabla_\theta p_\theta^0(x_{1:N}\mid x_0)p_\theta^\beta(x_{1:N}\mid x_0)}{p_\theta^0(x_{1:N}\mid x_0)^2}\right]$$

Which yields

$$\nabla_\theta \mathrm{KL}(p_\theta^\beta(x_{1:N}\mid x_0)\parallel p_\theta^0(x_{1:N}\mid x_0))=\mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta\log\left(p_\theta^\beta(x_{1:N}\mid x_0)\right)\log\left(\frac{p_\theta^\beta(x_{1:N}\mid x_0)}{p_\theta^0(x_{1:N}\mid x_0)}\right)\right]$$

$$+\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N}\mid x_0)p_\theta^0(x_{1:N}\mid x_0)-\nabla_\theta p_\theta^0(x_{1:N}\mid x_0)p_\theta^\beta(x_{1:N}\mid x_0)}{p_\theta^0(x_{1:N}\mid x_0)^2}\right]$$

Or again using the log derivative trick, we derive
$\nabla_\theta\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N C(q_k,p_k)\right]=\mathbb{E}_{p_\theta^\beta}\left[(\sum_{k=0}^N C(q_k,p_k))\nabla_\theta\log\left(p_\theta^\beta(x_{1:N}\mid x_0)\right)\right]$
Which gives the final expression of the gradient of the objective function

$$J(\theta)=\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N C(q_k,p_k)\right]+\lambda\,\mathrm{KL}(p_\theta^\beta(x_{1:N}\mid x_0)\parallel p_\theta^0(x_{1:N}\mid x_0))$$

$$\nabla_\theta J(\theta)=\mathbb{E}_{p_\theta^\beta}\left[(\sum_{k=0}^N C(q_k,p_k))\nabla_\theta\log\left(p_\theta^\beta(x_{1:N}\mid x_0)\right)\right]$$

$$+\mathbb{E}_{p_\theta^\beta}\left[\nabla_\theta\log\left(p_\theta^\beta(x_{1:N}\mid x_0)\right)\log\left(\frac{p_\theta^\beta(x_{1:N}\mid x_0)}{p_\theta^0(x_{1:N}\mid x_0)}\right)\right]$$

$$+\mathbb{E}_{p_\theta^0}\left[\frac{\nabla_\theta p_\theta^\beta(x_{1:N}\mid x_0)p_\theta^0(x_{1:N}\mid x_0)-\nabla_\theta p_\theta^0(x_{1:N}\mid x_0)p_\theta^\beta(x_{1:N}\mid x_0)}{p_\theta^0(x_{1:N}\mid x_0)^2}\right]$$

This yields a Limitless dynamical stochastic EP framework.

A.4 Fisher HEP (limitless single phase) (see Chapter 4)

A.4.1 Proof of the EP-like formula

We consider the following objective function :

$$J(\theta)=\mathbb{E}_{p_\theta^\beta}\left[\sum_{k=0}^N C(q_k,p_k)\right]+\lambda^2\,\mathrm{FD}(p_\theta^\beta(x_{1:N}\mid x_0)\parallel p_\theta^0(x_{1:N}\mid x_0))$$

$$=\int\sum_{k=0}^N C(q_k,p_k)p_\theta^\beta(x_{1:N}\mid x_0)\,dx_{1:N}+$$

$$\frac{\lambda^2}{2}\int p_\theta^\beta(x_{1:N}\mid x_0)\left\|\nabla_{x_{1:N}}\log p_\theta^\beta(x_{1:N}\mid x_0)-\nabla_{x_{1:N}}\log p_\theta^0(x_{1:N}\mid x_0)\right\|^2 dx_{1:N}$$

where λ is a regularization hyperparameter which depends on the noise/variance.

Computation of $\nabla_\theta \mathrm{FD}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0))$

We have

$\nabla_\theta \mathrm{FD}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0)) =$

$\nabla_\theta \int p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N} =$

$\int \nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N} +$

$\int p_\theta^\beta(x_{1:N} \mid x_0) \nabla_\theta \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N}$

Using the log derivative trick, we obtain :

$\int \nabla_\theta p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N} =$

$\mathbb{E}_{p_\theta^\beta} \left[\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 \right]$

And

$\nabla_\theta \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 =$

$2\big(\nabla_\theta \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_\theta \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \nabla_{x_{1:N}}\big)^\top \big(\nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0)\big)$

Thus

$\int p_\theta^\beta(x_{1:N} \mid x_0) \nabla_\theta \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 dx_{1:N} =$

$\mathbb{E}_{p_\theta^\beta}[2\big(\nabla_\theta \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_\theta \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \nabla_{x_{1:N}}\big)^\top \big(\nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0)\big)]$

Thus for $J(\theta) = \mathbb{E}_{p_\theta^\beta} \left[\sum_{k=0}^N C(q_k, p_k) \right] + \lambda^2 \mathrm{FD}(p_\theta^\beta(x_{1:N} \mid x_0) \parallel p_\theta^0(x_{1:N} \mid x_0))$

we have

$\nabla_\theta J(\theta) = \mathbb{E}_{p_\theta^\beta} \left[\nabla_\theta \log p_\theta^\beta(x_{1:N} \mid x_0) \left\| \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \right\|^2 \right] +$

$\mathbb{E}_{p_\theta^\beta}[2\big(\nabla_\theta \nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_\theta \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0) \nabla_{x_{1:N}}\big)^\top \big(\nabla_{x_{1:N}} \log p_\theta^\beta(x_{1:N} \mid x_0) - \nabla_{x_{1:N}} \log p_\theta^0(x_{1:N} \mid x_0)\big)]$

This offers a framework for a single phase limitless dynamical EP.

References

[1] Miles Cranmer et al. "Lagrangian Neural Networks". In: *arXiv preprint arXiv:2003.04630* (2020). DOI: 10.48550/arXiv.2003.04630.

[2] Sam Greydanus, Misko Dzamba, and Jason Yosinski. "Hamiltonian Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. DOI: 10.48550/arXiv.1906.01563. arXiv: 1906.01563.

[3] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.

[4] Guillaume Pourcel et al. "Lagrangian-based Equilibrium Propagation: Generalisation to Arbitrary Boundary Conditions & Equivalence with Hamiltonian Echo Learning". In: *arXiv preprint arXiv:2506.06248* (June 2025). DOI: 10.48550/arXiv.2506.06248.

[5] Paul A. M. Dirac. *The Principles of Quantum Mechanics*. 4th ed. Oxford: Oxford University Press, 1958.

[6] Jacob Biamonte et al. "Quantum Machine Learning". In: *Nature* 549 (2017), pp. 195–202. DOI: 10.1038/nature23474.

[7] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. "Augmented Neural ODEs". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[8] Artur Luczak, Bruce L. McNaughton, and Yoshimasa Kubo. "Neurons Learn by Predicting Future Activity". In: *Nature Machine Intelligence* 4.1 (2022), pp. 62–72. DOI: 10.1038/s42256-021-00430-y.

[9] Yoshua Bengio. "How Auto-Encoders Could Provide Credit Assignment in Deep Networks via Target Propagation". In: *arXiv preprint arXiv:1407.7906* (2014). DOI: 10.48550/arXiv.1407.7906. arXiv: 1407.7906 [cs.LG].

[10] Dong-Hyun Lee et al. "Difference Target Propagation". In: *Machine Learning and Knowledge Discovery in Databases – European Conference, ECML PKDD 2015*. Vol. 9284. Lecture Notes in Computer Science. Springer, 2015, pp. 498–515. DOI: 10.1007/978-3-319-23528-8_31. arXiv: 1412.7525 [cs.LG].

[11] T. Konstantin Rusch et al. "Graph-Coupled Oscillator Networks". In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 18888–18909. URL: https://proceedings.mlr.press/v162/rusch22a.html.

[12] Ling Zhao et al. "T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction". In: *IEEE Transactions on Intelligent Transportation Systems* 21.9 (2020), pp. 3848–3858. DOI: 10.1109/TITS.2019.2935152.

[13] Axel Laborieux and Friedemann Zenke. "Holomorphic Equilibrium Propagation Computes Exact Gradients Through Finite Size Oscillations". In: *arXiv preprint arXiv:2209.00530* (2022).

[14] Benjamin Scellier. "A deep learning theory for neural networks grounded in physics". In: *arXiv preprint arXiv:2103.09985* (2021). DOI: 10.48550/arXiv.2103.09985.

[15] Elon Litman. "Equilibrium Propagation Without Limits". In: *arXiv preprint arXiv:2511.22024* (2025).

[16] Zhuangyu Han and Abhronil Sengupta. "Neuromorphic Reinforcement Learning for Quadruped Locomotion Control on Uneven Terrain". In: *arXiv preprint arXiv:2605.09595* (May 2026). DOI: 10.48550/arXiv.2605.09595. arXiv: 2605.09595 [cs.NE].

[17] Yoshimasa Kubo, Eric Chalmers, and Artur Luczak. "Combining Backpropagation with Equilibrium Propagation to Improve an Actor-Critic Reinforcement Learning Framework". In: *Frontiers in Computational Neuroscience* 16 (2022), p. 980613. DOI: 10.3389/fncom.2022.980613.

[18] Benjamin Scellier and Yoshua Bengio. "Equilibrium Propagation: Bridging the Gap between Energy-Based Models and Backpropagation". In: *Frontiers in Computational Neuroscience* 11 (2017), p. 24. DOI: 10.3389/fncom.2017.00024.

[19] Peter J. Olver. *The Calculus of Variations*. University of Minnesota. Lecture notes. Jan. 2022. URL: https://www-users.cse.umn.edu/~olver/ln_/cvc.pdf.

[20] Serge Massar. "Equilibrium Propagation for Learning in Lagrangian Dynamical Systems". In: *arXiv preprint arXiv:2505.07363* (May 2025). DOI: 10.48550/arXiv.2505.07363.

[21] Guillaume Pourcel and Maxence Ernoult. "Learning Long Range Dependencies Through Time Reversal Symmetry Breaking". In: *arXiv preprint arXiv:2506.05259* (June 2025). DOI: 10.48550/arXiv.2506.05259.

[22] Victor Lopez-Pastor and Florian Marquardt. "Self-learning Machines based on Hamiltonian Echo Backpropagation". In: *Physical Review X* 13.3 (2023), p. 031020. DOI: 10.1103/PhysRevX.13.031020. arXiv: 2103.04992 [cs.LG].

[23] Lev Semenovich Pontryagin et al. *Mathematical Theory of Optimal Processes*. Routledge, 1985.

[24] Ricky T. Q. Chen et al. "Neural Ordinary Differential Equations". In: *arXiv preprint arXiv:1806.07366* (2018). DOI: 10.48550/arXiv.1806.07366.

[25] Louis-Pascal A. C. Xhonneux, Meng Qu, and Jian Tang. "Continuous Graph Neural Networks". In: *arXiv preprint arXiv:1912.00967* (2020). DOI: 10.48550/arXiv.1912.00967.

[26] Matthew Thorpe et al. “GRAND++: Graph Neural Diffusion with a Source Term”. In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=EMxu-dzvJk.

[27] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *International Conference on Learning Representations*. 2017. arXiv: 1609.02907.

[28] Kai Zhao et al. “Adversarial Robustness in Graph Neural Networks: A Hamiltonian Approach”. In: *Advances in Neural Information Processing Systems*. Vol. 36. Curran Associates, Inc., 2023, pp. 3338–3361. DOI: 10.52202/075280-0148. arXiv: 2310.06396 [cs.LG].

[29] Axel Laborieux et al. “Scaling Equilibrium Propagation to Deep ConvNets by Drastically Reducing Its Gradient Estimator Bias”. In: *Frontiers in Neuroscience* 15 (2021), p. 633674. DOI: 10.3389/fnins.2021.633674.

[30] Serge Massar and Bortolo Matteo Mognetti. “Equilibrium Propagation: the Quantum and the Thermal Cases”. In: *Quantum Studies: Mathematics and Foundations* 12.1 (2025), p. 6. DOI: 10.1007/s40509-024-00351-6. arXiv: 2405.08467 [quant-ph].

[31] Víctor López-Pastor and Florian Marquardt. “Self-Learning Machines Based on Hamiltonian Echo Backpropagation”. In: *Physical Review X* 13.3 (2023), p. 031020. DOI: 10.1103/PhysRevX.13.031020. arXiv: 2103.04992 [cs.LG].

[32] Piero Caldirola. “Forze non conservative nella meccanica quantistica”. In: *Il Nuovo Cimento* 18 (1941), pp. 393–400. DOI: 10.1007/BF02960144.

[33] Eiichi Kanai. “On the Quantization of the Dissipative Systems”. In: *Progress of Theoretical Physics* 3.4 (1948), pp. 440–442. DOI: 10.1143/PTP/3.4.440.

[34] T. Konstantin Rusch and Siddhartha Mishra. “UnICORNN: A Recurrent Model for Learning Very Long Time Dependencies”. In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 9168–9178. URL: https://proceedings.mlr.press/v139/rusch21a.html.

[35] Wayne W. Zachary. “An Information Flow Model for Conflict and Fission in Small Groups”. In: *Journal of Anthropological Research* 33.4 (1977), pp. 452–473. DOI: 10.1086/jar.33.4.3629752.

[36] Petar Veličković et al. “Graph Attention Networks”. In: *International Conference on Learning Representations*. 2018.

[37] Constantino Tsallis. “Possible Generalization of Boltzmann-Gibbs Statistics”. In: *Journal of Statistical Physics* 52.1–2 (1988), pp. 479–487. DOI: 10.1007/BF01016429.

[38] Daniel Liberzon. *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton, NJ: Princeton University Press, 2012.

[39] Marc G. Bellemare, Will Dabney, and Rémi Munos. "A Distributional Perspective on Reinforcement Learning". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 449–458.

[40] Marc G. Bellemare et al. "The Cramer Distance as a Solution to Biased Wasserstein Gradients". In: *arXiv preprint arXiv:1705.10743* (2017). DOI: 10.48550/arXiv.1705.10743. arXiv: 1705.10743.

[41] Mark Rowland et al. "An Analysis of Categorical Distributional Reinforcement Learning". In: *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*. Vol. 84. Proceedings of Machine Learning Research. PMLR, 2018, pp. 29–37.

[42] Ziming Liu et al. "KAN: Kolmogorov–Arnold Networks". In: *International Conference on Learning Representations*. 2025. URL: https://openreview.net/forum?id=Ozo7qJ5Fn7.

[43] Lynton Ardizzone et al. "Guided Image Generation with Conditional Invertible Neural Networks". In: *arXiv preprint arXiv:1907.02392* (2019). DOI: 10.48550/arXiv.1907.02392. arXiv: 1907.02392 [cs.CV].

[44] Laurent Dinh, David Krueger, and Yoshua Bengio. "NICE: Non-linear Independent Components Estimation". In: *International Conference on Learning Representations*. 2015. arXiv: 1410.8516. URL: https://arxiv.org/abs/1410.8516.

[45] Ivan Kobyzev, Simon J. D. Prince, and Marcus A. Brubaker. "Normalizing Flows: An Introduction and Review of Current Methods". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), pp. 3964–3979. DOI: 10.1109/TPAMI.2020.2992934. arXiv: 1908.09257.

[46] Ming Jin et al. "Multivariate Time Series Forecasting With Dynamic Graph Neural ODEs". In: *IEEE Transactions on Knowledge and Data Engineering* 35.9 (2023), pp. 9168–9180. DOI: 10.1109/TKDE.2022.3221989.

[47] Robert S. Liptser and Albert N. Shiryaev. *Statistics of Random Processes: I. General Theory*. 2nd ed. Vol. 5. Stochastic Modelling and Applied Probability. Springer, 2001. DOI: 10.1007/978-3-662-13043-8.

[48] Haiyang Liu et al. "Attention-Based Spatial-Temporal Graph Convolutional Recurrent Networks for Traffic Forecasting". In: *Advanced Data Mining and Applications*. Vol. 14176. Lecture Notes in Computer Science. Springer, 2023, pp. 630–645. DOI: 10.1007/978-3-031-46661-8_42.

[49] Shengnan Guo et al. "Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33. 01. 2019, pp. 922–929. DOI: 10.1609/aaai.v33i01.3301922.

[50] Antonio Longa et al. "Graph Neural Networks for Temporal Graphs: State of the Art, Open Challenges, and Opportunities". In: *Transactions on Machine Learning Research* (2023), pp. 1–24.

[51] Si Zhang et al. “Graph Convolutional Networks: A Comprehensive Review”. In: *Computational Social Networks* 6.1 (2019), p. 11. DOI: 10.1186/s40649-019-0069-y.

[52] Bin Wang et al. “AAGCN: A Graph Convolutional Neural Network with Adaptive Feature and Topology Learning”. In: *Scientific Reports* 14 (2024), p. 10134. DOI: 10.1038/s41598-024-60598-2.

[53] R. L. Stratonovich. “A New Representation for Stochastic Integrals and Equations”. In: *SIAM Journal on Control* 4.2 (1966), pp. 362–371. DOI: 10.1137/0304028.

[54] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. 6th ed. Springer, 2003. DOI: 10.1007/978-3-662-05084-4.